Contents

Analysis....2
Problem....2
Stakeholders....2
Research of the Problem....3
Essential Features....6
Limitations of Proposed Solution....6
Hardware and Software Requirements....6
Success Criteria....7
Design....8
Problem Decomposition....8
Structure of the Game....9
Forming a Complete Solution with Algorithms....11
Main Menu code:....11
Draw board subroutine:....12
Main Loop function:....13
Take Turn method:....14
Key Variables, Data Structures and Classes....15
Players class:....15
Tiles class:....16
Usability Features....17
Test plan....19
Development....25
Evidence of iterative development....25
First Iteration:....25
Break Down of First Iteration:....25
Testing of First Iteration:....52
Remedial Action:....72
Review of First Iteration:....74
Second Iteration:....75
Break Down of Second Iteration:....75
Testing of Second Iteration:....104
Review of Second Iteration:....117
Third Iteration:....118
Break Down of Third Iteration:....118
Testing of Third Iteration:....143

Analysis

Problem

Monopoly is a very popular board game around the world, however you are unable to play it digitally without giving money to purchase it, in addition most computational solutions are proprietary so you are unable to study and learn from the code or change it to suit your needs. My project intends to provide this functionality for consumers by being open source. It is also simple and lightweight so you don't have to purchase expensive hardware to be able to run it.

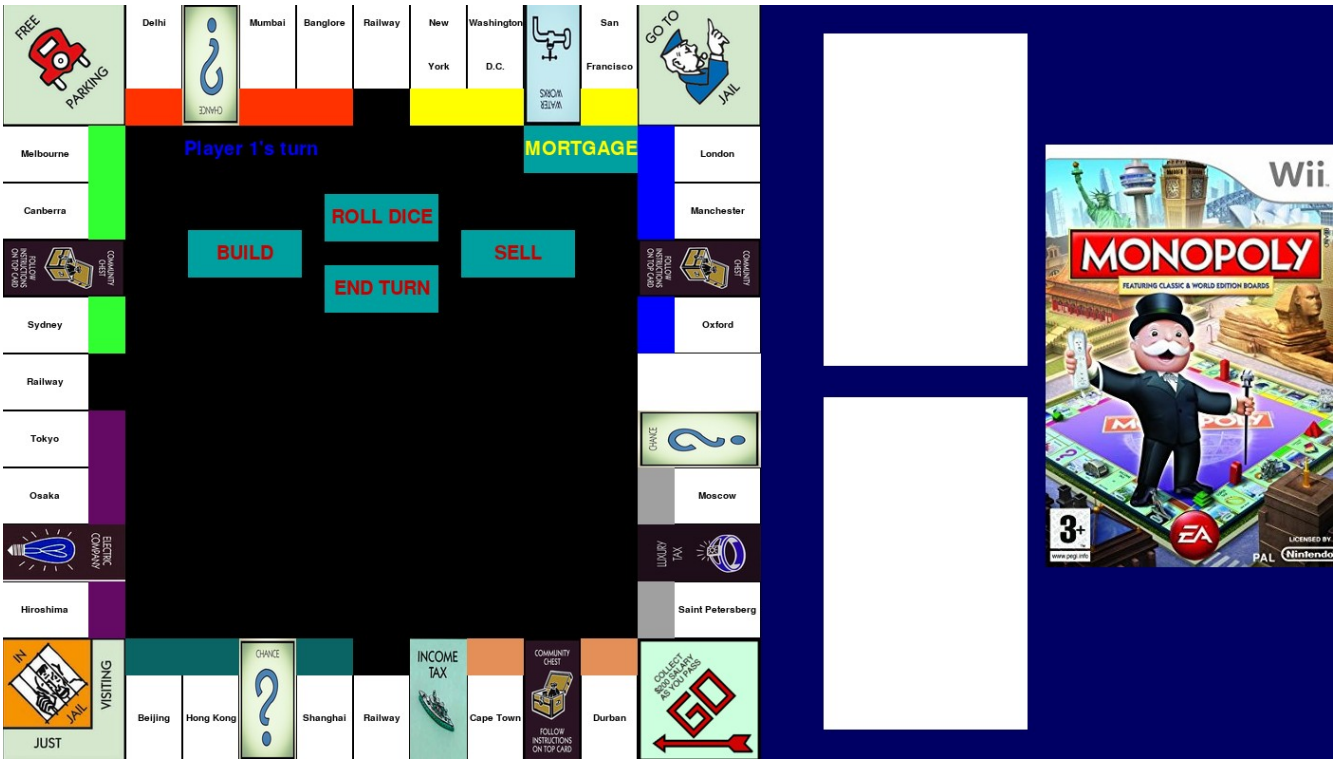
I have given this project a computational approach because much of the game involves simple arithmetic, like calculating how much to pay someone, which a computer is very efficient at doing, making the game quicker to play. There isn't any aspects of monopoly that would hinder a computational approach. In addition, a computational approach allows the game to have computer players meaning that even a person on their own can still enjoy a game. It also allows the game to be easily played portably, such as on a laptop, as in the original board game it was difficult to do so.

Stakeholders

Since this game is going to be free, the main group I am targeting is people with less money such as teenagers and young adults. Unlike many other free versions of monopoly which are full of advertisements and incomplete, my product will provide most of the features which make up the original game. To appeal to this audience I will ensure that the system can run on cheaper hardware and is easy to understand. I am also trying to appeal to programmers who might want to view the source code and extend it, which is possible due to the open source nature.

The main stakeholder is myself. As I am producing the application independently, I wish to see the program become a success in the sense that the game is feature complete and is bug free. In order to achieve this I will dedicate lots of time to planning out the development cycle to ensure the code is successful. Another stakeholder to consider is the user which will mostly consist of teenagers and young adults. I need to appeal to this audience to achieve a larger install who can benefit from my program. To do so I will ensure the game is easy to understand and play, and that it has all the features that would make up a regular game of monopoly.

Research of the Problem



In order to gain some insight on what a Monopoly game would look like in pygame I looked at other similar programs that I found online.

The first of which was simply titled Monopoly and was a recreation of Monopoly using the pygame and tkinter libraries. It used a main menu on a different window to select the names of the players, which was made with tkinter. This could be a good feature to include in my system. When the game started it was unfortunately broken as the screen would flicker and the buttons were unresponsive. In addition the rules weren't well explained so I was unfortunately unable to play much. Due the the broken nature I couldn't study most of the program but I could still look at the UI which was similar to my own having the text and options displayed in the middle of the board. In order to avoid problems I will make sure I test my system rigorously.

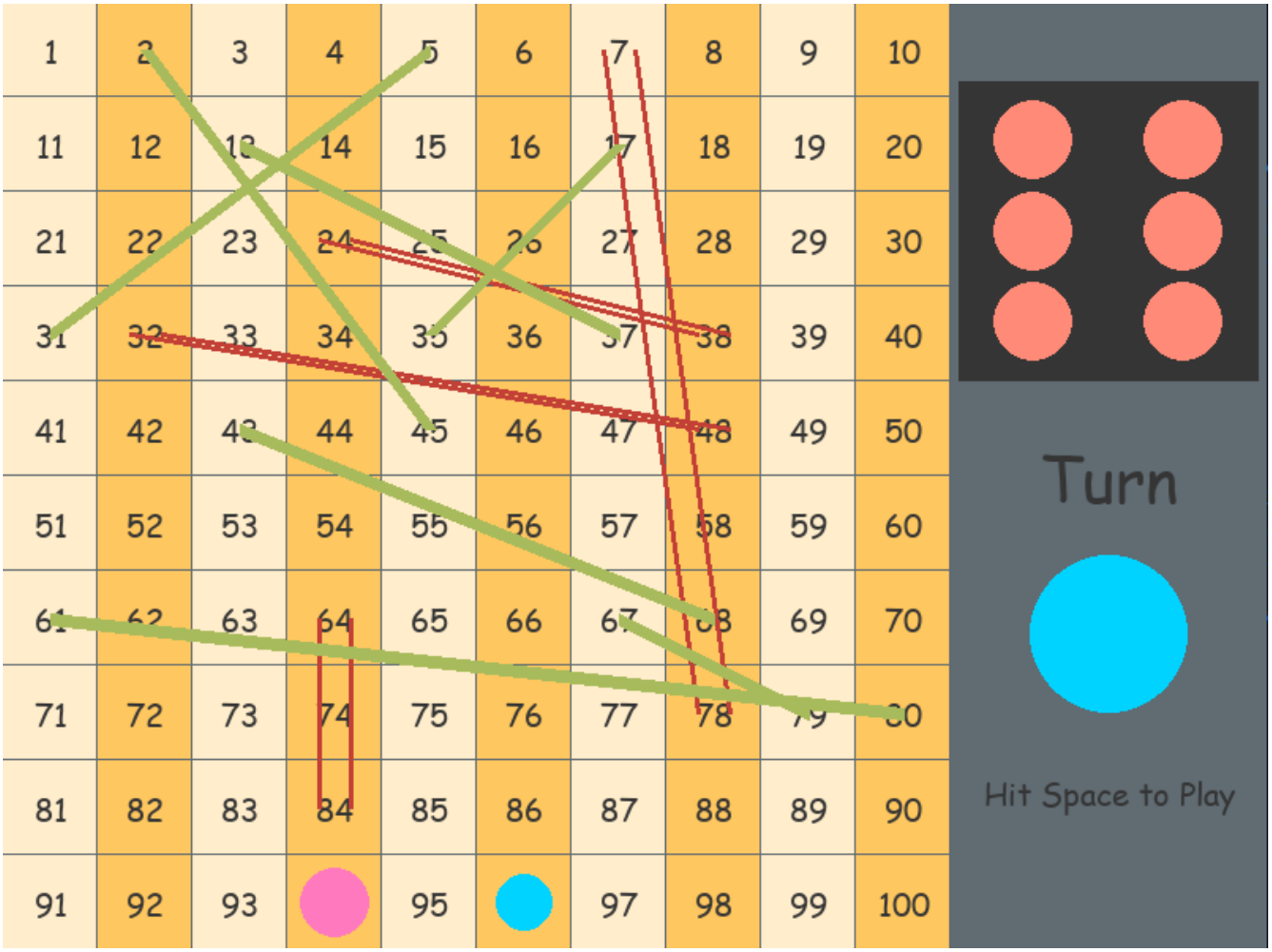
The UI for this game was also somewhat ugly with many clashing colours due to images from the monopoly board that have been blitted onto the game. The game also has the box art from a version of monopoly for the Nintendo Wii, which I found to look strange and out of place. In order to ensure my game's UI doesn't have similar issues I will gain feedback from users.

```
class Player:
    def __init__(self,color,no):  #each player initialised with its data
        self.cash = 300000
        self.posx = display_height-card_length/2
        self.posy = display_height-card_length/2
        self.total_wealth = 300000
        self.properties = []
        self.getoutofjailcard = 0
        self.color = color
        self.no = no
        self.no_of_railways = 0
        self.released = 1
    def draw(self):  #function to draw circular player on the board
            _font = pygame.font.Font('freesansbold.ttf',20)
            pygame.draw.circle(functions.gameDisplay,self.color,[(int)(self.posx),(int)(self.posy)],20)
            textSurface = _font.render(self.no, True, black)
            textRect = textSurface.get_rect()
            textRect.center = (self.posx,self.posy)
            gameDisplay.blit(textSurface, textRect)
            pygame.display.update()
```

Upon viewing the code, they appear to be using a class structure for the players which is useful as it allows for a lot of code reuse and creates a highly modular system.

Overall I found the code quite hard to understand as there were very few classes and the each class did many different things. Overall the code didn't feel very structured. There was also a lot of global variables which are bad practice to use.

Link: https://github.com/AniketSanghi/Monopoly-game



I also looked at a snakes and ladders game made in pygame as it would give me some insight into what a board game with some similar mechanics to monopoly would look like in pygame.

It didn't feature any menu screen so the only option was to play with two human players. When the dice is rolled an image of the dice switches between the different faces to represent the rolling of a dice. This makes the game more interesting to play and ensures the game isn't to fast paced by having instant dice rolls. This would be good feature to add to my program.

When the dice has finished rolling the player icon instantly teleports the the position it needs to be in. I found that this made the game hard to follow and I frequently lost track of it. In my program I will make sure the player moves in steps to help make the game more playable.

Link: https://github.com/manan-d8/snakes-and-ladders-pygame

I also conducted a questionnaire with many potential users to gain insight into how I should design the game.

.....

Essential Features

In order to make a functional program I have identified some essential features needed for the game. Firstly it will need a board that the players can go around with properties and spaces to land on. It also needs a main menu screen that allows the user to select the number of players and computer players. The game also needs a system to display information to the player such as showing that board, informing the player of what is happening in the game and displaying what each player owns. The players need to be able to purchase properties and level them up, and there needs to be a system where a player can go bankrupt, allowing the final player standing to win.

Limitations of Proposed Solution

As Monopoly is a board game there will be some limitations of a computational approach to the game. For example the game will not be able to be played with multiple players from different computers, as the networking is outside of the scope of the project.

Hardware and Software Requirements

In order develop the solution I will require certain hardware and software.

Hardware:

- Modern CPU
- About 10MB of disk space
- About 100MB of unused RAM
- At least 1080x1920 resolution display
- A keyboard and mouse

Software:

- A modern python supported OS (Windows, MacOS, Linux)

- Python3 and Pygame
- An IDE or code editor

For the user to run the solution they won't require Python3 or Pygame as the code will be compiled into machine code. They also won't need an IDE or code editor.

Success Criteria

1. The game must have a main menu screen
    - There must be an option to choose the number of players and number of computer players
    - There must be a way to load a previous game save
    - 100% of users must find the interface usable and easily understandable
2. The whole game of monopoly must be playable
    - The players must go around the board and have the ability to purchase properties and houses
    - There should be chance and community chest cards
    - When the player rolls a double they get another go and three doubles means they go to jail
    - When the player passes go they get £200
    - When a player runs out of money they go bankrupt and are out of the game.

3. There should be additional UI elements while playing the game to improve the user experience
    - The game should display whose turn it is at all times along with how much money the player has
    - There should be a place to display all players balance along with the properties they own
    - When the user clicks on a space on the board a card should show on the screen that displays information about the space such as the name or the owner if it is a property
    - When the user rolls the dice an animation should play to show the dice rolling
4. There should be optional AI players so the user can play without other people

- The AI must be able to purchase properties with some level of logic behind their decision making
- They must be able to buy houses for their properties when they can afford it
- The AI must roll individually and make any other decisions with logic behind it

5. There needs to be a way to save the state of the game
    - All objects and variables in the game need to be saved to a file
    - The data should be able to be loaded from the main menu screen of the game

Design

Problem Decomposition

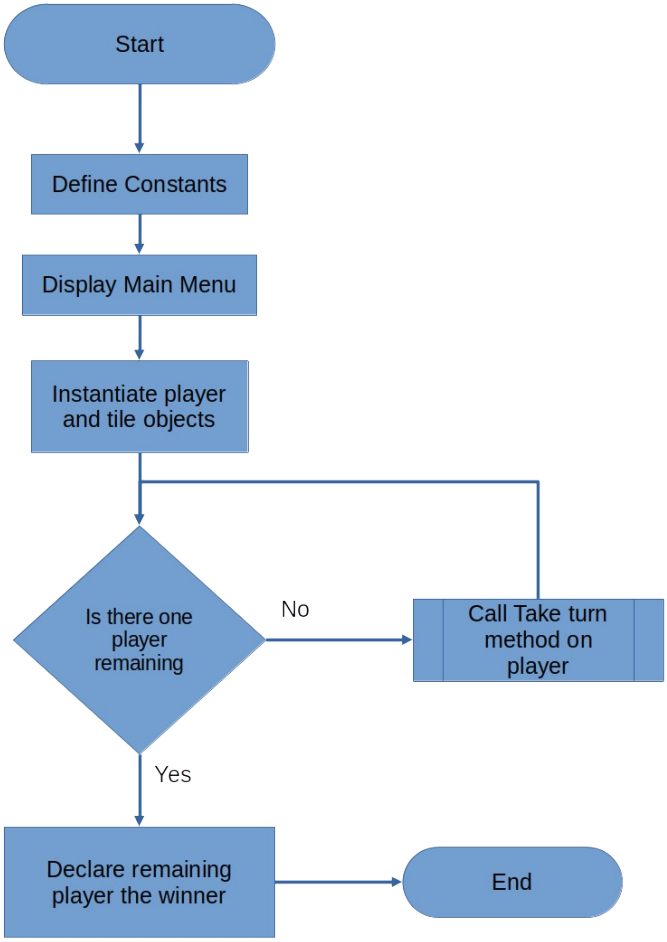
The game can be broken down into to separate main parts: the main menu screen and the game screen.

The main menu screen allows the users to select options before the game. The player will select options using the mouse and clicking on the buttons. The user can select the number of players and AI players. At all points during the game, the user is able to close the window and have the pygame library quit and the code stop running, this ensures that the program user is able to quit when they want. If that were not the case, the program would be exhibiting behaviours similar to malware.

After the main menu screen is the game screen. This has multiple elements:

- The code responsible for drawing the board
- The player class which houses all of the attributes and methods the player uses.
- The computer class which will house the AI's attributes and methods
- The tile class which is responsible for all of the spaces the player can land
- The main loop where all of the other functions and methods are called to make up the game

Structure of the Game

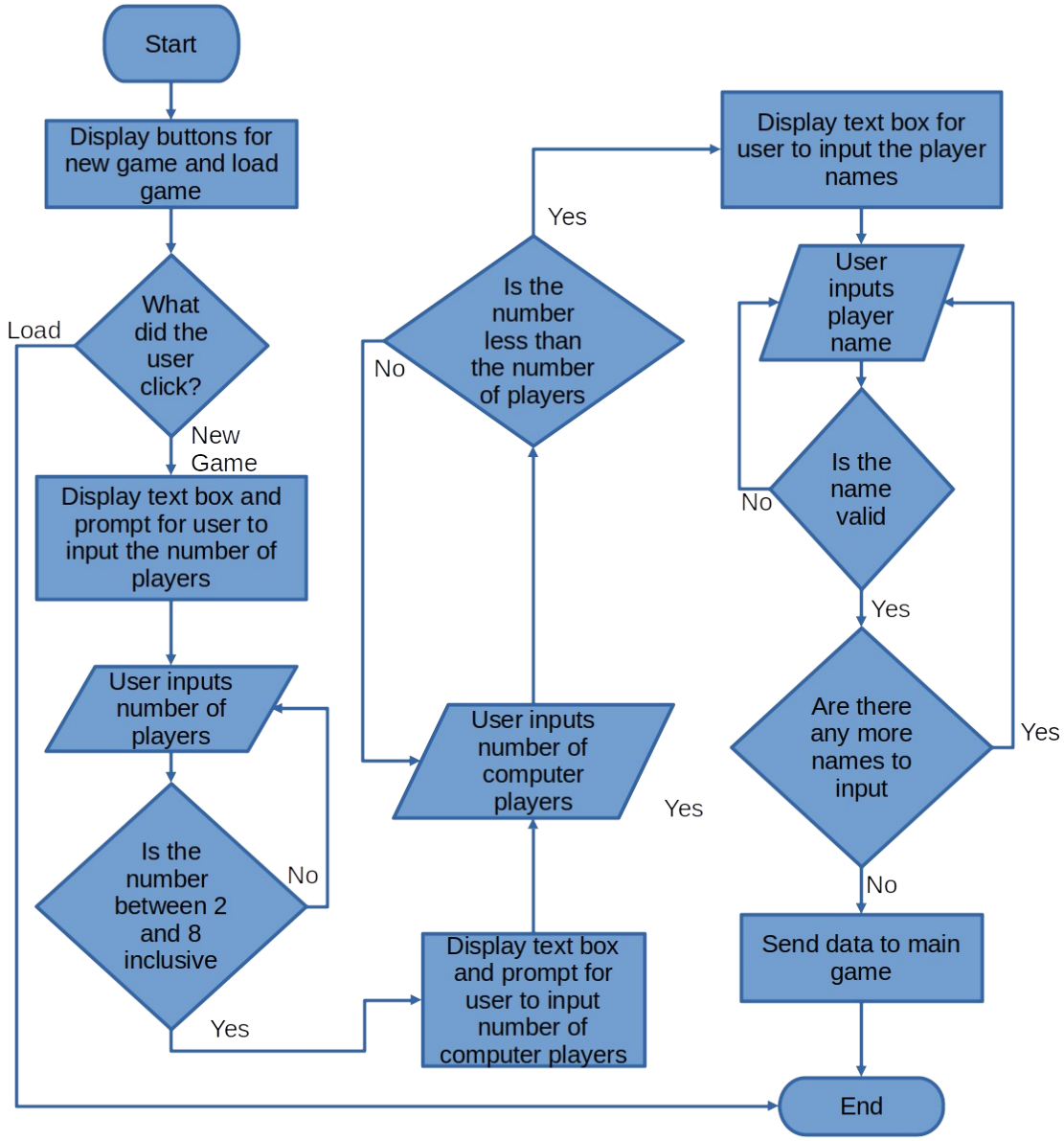


1. The game will start with all of the variables, constants and objects being created for use in the main game, this only happens once.
2. Then the main menu where the player and AI count will be selected by the user then be sent to main game.
3. Before the game starts, all the player and tile objects are instantiated from their class to be used throughout the game
4. A while loop is used for the game which cycles through each of the player objects and will call a method on each of them one at a time which allows them to take their turn.
5. Within the take turn method, anytime the player needs information or to make a decision a function is called which outputs to the player

6. This function is a while loop which is capable of displaying text or buttons to the user depending on the parameters. It also updates the board each frame.
7. While this loop is going, the player will be able to click on spaces on the board to receive information about that space.
8. When a player lands on tile a method will be called for that tile. All the different types of tiles will inherit from one superclass, allowing polymorphism to be used when calling the method
9. Once a player goes bankrupt, the game must take that player out of the while loop so they cant take their turn again.
10. Once only one player remains the game must end and display the winner.

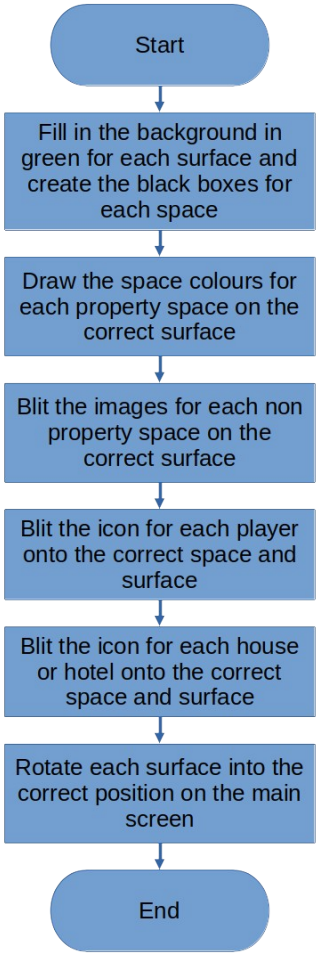
Forming a Complete Solution with Algorithms

Main Menu code:



Whenever the game is launched, the main menu will show. This lets the user choose the number of players and computer players and choose the names of the players. All of the inputs will be done using a text box and they will have to be validated. The menu also allows the user to load a game that has been previously saved which will cause all of the variables and classes to be created with the correct values from a file, essentially loading the games previous state.

Draw board subroutine:



The board and all the elements will be created and drawn in a subroutine that can be called at anytime. This means that the board can easily be redrawn in case something else is blitted over it. In order to create the board graphic, I will use pygame's built in draw functions to draw the board with code. This allows me to know the exact pixel size of each space on the board which makes it easier to move players around. It also allows the board to be scalable with window size which could be useful in case I want the window size to be different. Each property space will be coloured in its respective colour and the other spaces will have an image blitted to them which will be scaled accordingly.

Each side of the board will act as its own surface. This means that when I need to add things to the spaces, I don't need to use complicated maths to position everything correctly as I can just rotate the surface into position at the end.

Each player will be a coloured square that will sit in the bottom of each square. When another player moves onto the same square, they will be moved next the player before them so they don't overlap.

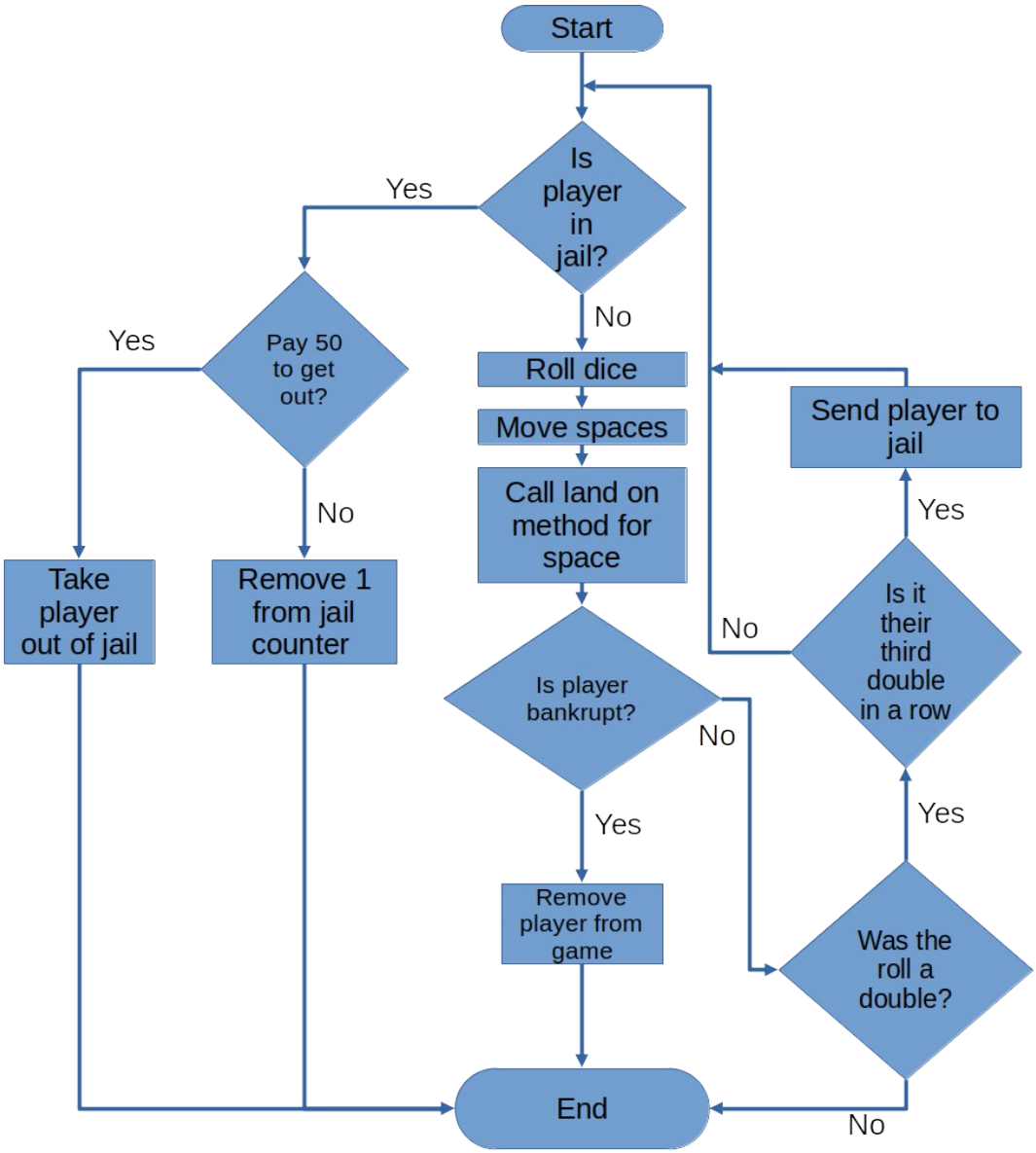
Main Loop function:

```
function main loop(text, prompt, button1, button2, save)
     while true:
          draw board and players and houses
          draw player information
          if in text mode:
               display text and next button
               if next button is pressed:
                    endloop
          if in input mode:
               display prompt and buttons
               if button1 is pressed:
                    return 1
               if button2 is pressed:
                    return 2
          if tile is clicked on:
               show info about tile
          if save:
               display save button
               if save button is clicked on
                    save game state to file
 endfunction
```

The main loop is a function that has the while true loop which is responsible for generating the input and output. As monopoly is not a real time game, the loop will be limited to running 30 times a second which reduces unnecessary CPU load. Everytime something new needs to be displayed on the screen, the main loop will be called with specific parameters which can decide whether it displays text or choices.

At the start of each loop, the board will be drawn and the players and houses will be added to it using the previous function. After, there will be a space to the right of the board will be drawn, which displays information about players such as what properties they own and how much money they have. Then the mode is checked. If it is in text mode, it will blit text parsed through the parameter onto the screen, allowing the user to keep up with the game, and also a next button. The loop will repeat until it detects that the user has clicked the next button which causes the loop to end. If is input mode, a prompt and two buttons will be drawn on screen. The text for the prompt and the text on the buttons will be parsed as parameters into the function. When the user clicks one of the buttons, the loop will end and a value will be returned so the choice can be processed by the program. At any point during the loop, the player will be able to click on one of the tiles on the board, which will display some information about the space.

Take Turn method:

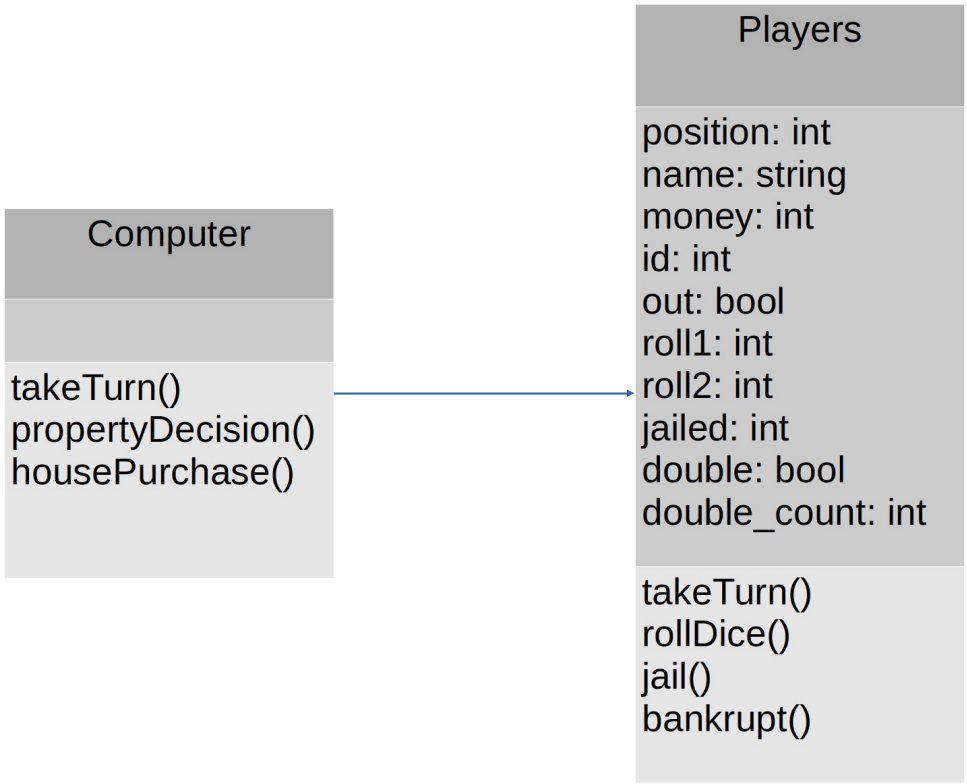


The take turn method goes through an entire turn of monopoly for a player. It is a part of the player class which allows it to easily access the players attributes in response to actions during the turn. For each time the method needs to output some text to the user to explain something, the main loop function is called. Everytime the player lands on a space, the land on method for that space will be called.

Key Variables, Data Structures and Classes

There will be two main classes in the game, the player class and the tile class. Each will have many subclasses.

Players class:

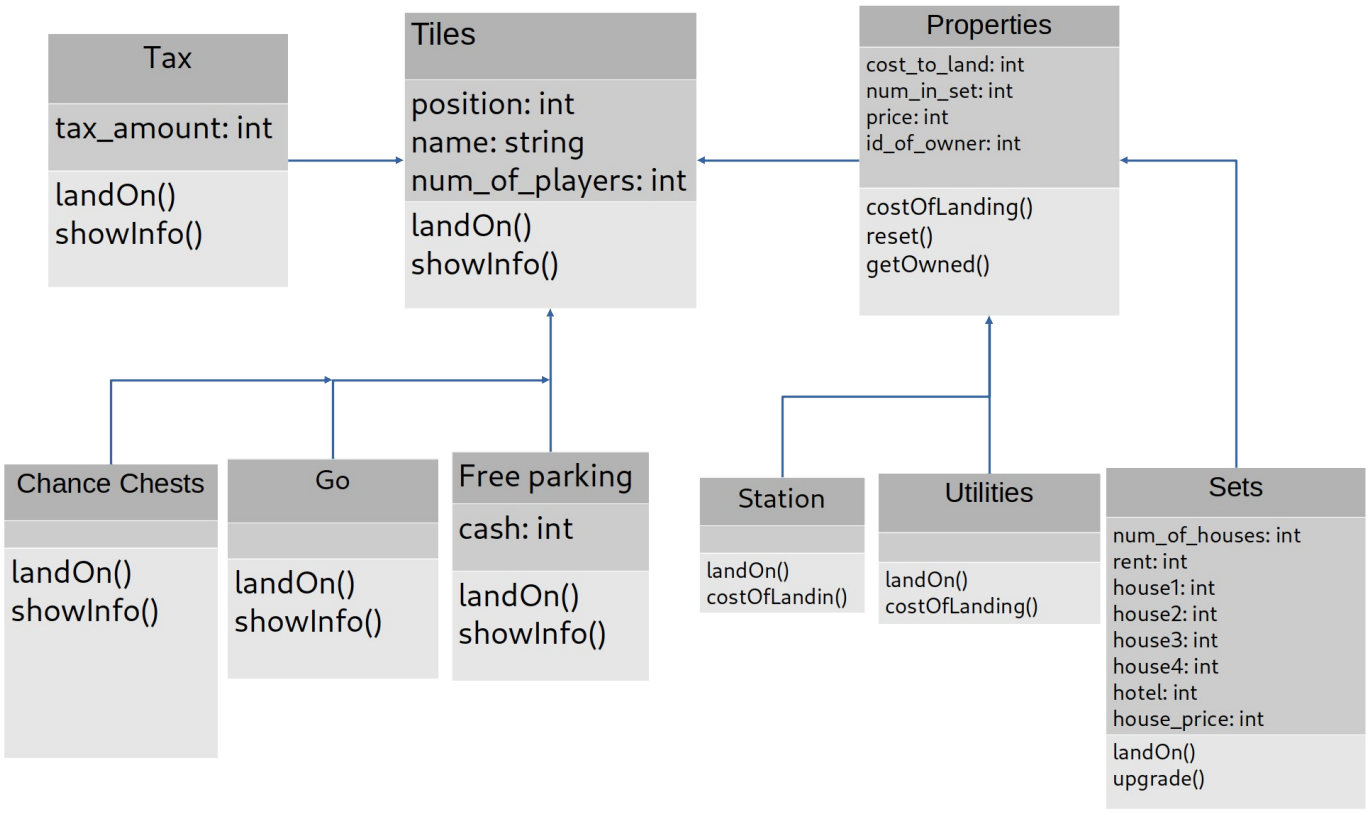


The Players class will hold many important attributes for the player such as their name, position, wealth, their rolls, if they are in jail and more. The takeTurn() method lets a player have their turn. The rollDice() method will generate two random numbers from 1 to 6 and put the result into the roll1 and roll2 attributes. The jail() method will put a player in jail, so they cant have their go for three turns, and move the player to the jail space. The bankrupt() method will be called if the players money reaches 0 and it will cause the player to be removed from the game.

The Computer class is a subclass of the Players class. It will inherit all of the attributes from the superclass as they are still a player and the attributes only affect the gameplay. The

computer class however will overwrite the takeTurn() method as it needs to be able to take its turn autonomously. The method propertyDecision() will be called when a computer lands on a property and it will produce an output of true or false on whether the computer wants to purchase it. The decision will be made based on how much money the computer player has and on a number in a file which lists the worth of the property. The housePurchase() method will be called at the end of the computer's takeTurn() method. It will make the decision based on the price of houses, the wealth they have and how much more money they will make from the upgrade. It will assess each possible house placement until it has rejected all of them which will end the method.

Tiles class:



Add GoToJail class

The Tiles class is an abstract class which holds the common attributes and methods that each tile will have. Each tile will have a position and a name. They will each need a landOn() method

to do the consequences of landing on a space, and a showInfo() method to decide what information is shown when a player clicks on the tile.

The Properties class is another abstract class that holds all of the common methods between tiles which can be bought. cost_to_land will describe how much you will have to pay if you land on this space if someone else owns it. num_in_set will show how many properties in the same set are owned by the same player, it is useful as it can affect the cost_to_land or the ability to build houses. The attribute price stores how much it will cost to buy the property.

The properties class has three subclasses: Station, Utilities and Sets. Station and Utilities just edits the landOn() and showInfo() methods to make it work like a monopoly station or utility and it inherits all of the attributes. The Sets subclass accounts for the main properties you buy in monopoly which you can put houses on. It has attributes to show the number of houses it has, the cost of landing on it with rent and with each number of houses, and the cost per house. It overwrites the landOn() method and has an upgrade() method which increases the number of houses on a set if they are evenly spread out.

The tiles class also has several miscellaneous subclasses which all just change the landOn() method to account for some of the other spaces to land on.

All of the tiles will be created once at the start of the game and put into an array. They will be put in order of position so the index of the tile matches its position. In addition a 2D array will be created which will show which properties are part of the same set. This will be used to find the num_in_set.

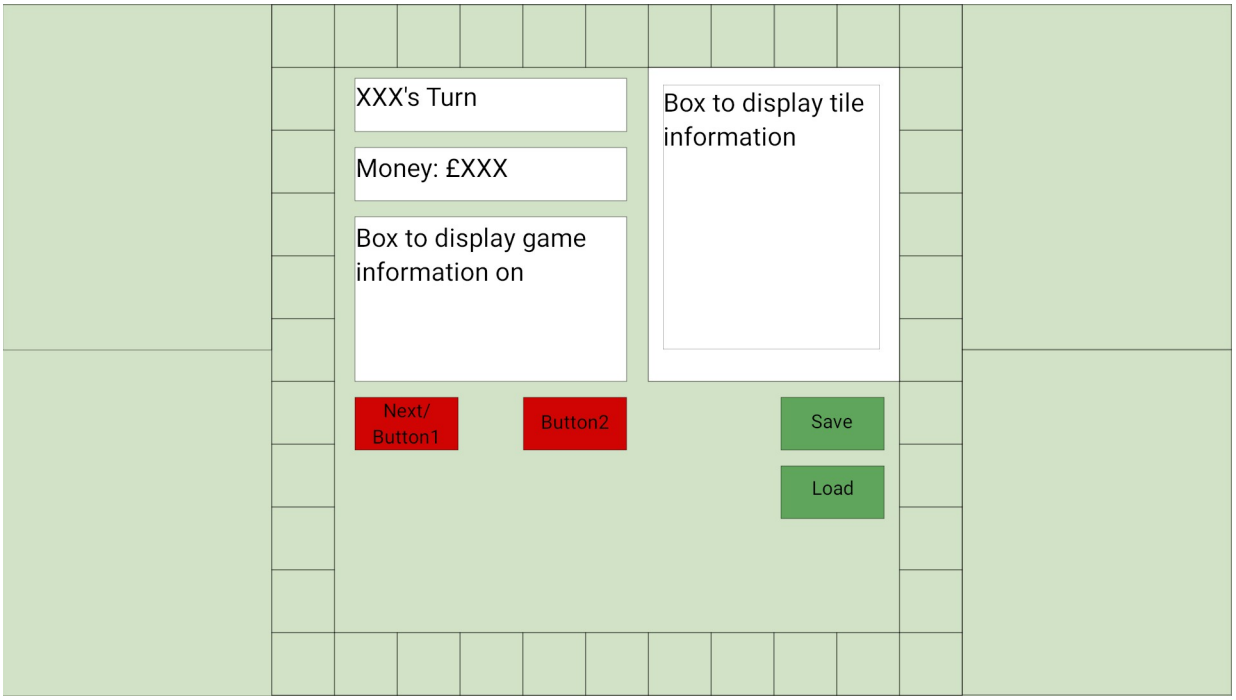
Since it will be made in pygame, I will need to import many important data structures from the module, such as the surface and rect classes for creating elements on the screen, and the event class for handling user inputs. These will be crucial for allowing the game to be played with a GUI. In addition I will need to import the random module in order to create a random object that can generate random numbers which will be helpful when doing things like dice rolls.

Usability Features

In order to maximise user satisfaction, I will ensure that my code includes many usability features.

For the game to be playable, the user must understand what is happening in the game at all times. To achieve this, I must include UI elements to make this happen.

The UI design without any detail should look like this:



Firstly, the main board will be in the centre of the screen as it is the focus of the game. The status cards will be placed around the board with two on the left and two on the right of it and they will display information about the players in the game. All other information will be displayed in the centre of the board as there is a lot of empty space there. The game will also display whose turn it is and how much money they have as these are very important things to know. When the player clicks on a tile, information about that tile will be shown in the board at the top right. When no tile is displayed I will show text in its place which tells the player to click on a tile to show information about it.

When the dice is rolled an animation should play similar to the one from the snakes and ladders game. It should be skippable by clicking if the player doesn't want to wait. When the player icon moves around the board it should move one space at a time so it is clear where it has moved. These two features should ensure that the game is not too fast paced confusing to follow.

In order for the game to run, it will require a 1920x1080 screen as the creation of the board is based around that resolution. Any lower resolution would cause it to be too big for the screen

For the inputs, the user will use the mouse to click on buttons to select what they want to do. This makes more sense than a keyboard as it is simpler to understand and the game doesn't require fast precise inputs that the keyboard provides.

Test plan

In order to make sure my program works as intended I will use black box alpha testing to check that it follows the spec and has no errors. I will also use beta testing by providing the software to my family and friends so they can give feedback.

Test plan for gameplay

Test Number	Test data	Expected results	Justification	Type of test data
101	Click on button 1 during the main loop	Function should return True	To test if the button works correctly	Normal
102	Click on button 2 during main loop	Function should return False	To test if the button works correctly	Normal
103	Closing the window while program is running	Pygame should stop and the window should close	To test if the user can exit the game correctly	Normal
104	Clicking on all the different tiles on the screen	A box with information about the property clicked on should appear on the screen	To test if the tile cards are blitted correctly	Normal
105	Clicking on a tile which is part of a set that cant have houses on it	There should not be a box allowing the user to upgrade their set	To test that the user cant buy houses when they don't meet the requirements	Normal

106	Clicking on a tile which is a set that is upgradable	A button should appear in the box which allows the user to upgrade their house	To test if the user is able to upgrade their houses	Normal
107	Clicking on the upgrade button for a tile	The space should have an extra house	To test if the user is able to upgrade houses	Normal
108	Upgrading a set more than 5 times	The user should be prevented from doing so	To test that the program prevents upgrading a house more than 5 times	Erroneous
109	Try to buy a property with enough money	The property should become owned by the player	To test if the player is able to purchase properties	Normal
110	Try to buy a property without enough money	The player should be unable to purchase the property	To test that the user is unable to purchase property when they cant afford it	Normal
111	Land on a property owned by someone else	The player who landed on it should pay the owner the amount specified	To test if payment to other players works properly	Normal

112	Land on a property owned by another player without enough money to pay them	The player should go bankrupt and be out of the game and all their properties should stop being owned and the owner should get what is left of their money	To test if a player can successfully go bankrupt	Normal
113	Land on a utility where both utilities are owned by the same player	The player who landed on it should pay the owner 10 times their previous dice roll	To test if the utilities work properly	Normal
114	Land on a station owned by someone who owns multiple stations	The player who landed on it should have to pay more depending on how many the owner owns	To test if the stations work properly	Normal
115	Roll a double	The player should get another roll	To test if the program correctly identifies doubles	Normal
116	Roll 3 doubles in a row	The player should go to jail	To test if the program can identify 3 doubles and correctly respond	Normal

117	Land on a chance or community chest space	The player should receive a chance or community chest card with instructions on it, those instructions should be followed and work correctly	To test if chance and community chest spaces work	Normal
118	Player moves past Go	They should receive £200	To test if the Go space works correctly	Normal
119	Player lands on a tax space	They must pay money to free parking	To test if the tax space works correctly	Normal
120	Player lands on free parking	They gain what ever money is stored in the space	To test if free parking works correctly	Normal
121	Player lands on Go to Jail	They are immediately sent to jail	To test if go to jail works correctly	Normal
122	All but one players going bankrupt	The remaining player should be the winner and be congratulated	To test if the game ends correctly	Normal

Test plan for main menu

Test Number	Test data	Expected results	Justification	Type of test data
201	Number of players: 3	3 players are created	To test if normal data works correctly	Normal
202	Number of players: 2	2 players are created	To test lower boundary data as there must be at least 2 players	Boundary
203	Number of players: 4	4 players are created	To test upper boundary data as there must be at most 4 players	Boundary
204	Number of players: 1	Invalid number as there is not enough players	To test the minimum of 2 players	Invalid
205	Number of players: 5	Invalid number as there is too many players	To test the limit of 8 players	Invalid
206	Number of players: abcd	Invalid input as number of players must be a number	To test that the user cant enter letters instead of numbers	Erroneous
207	Number of players: 4 Number of computers: 2	4 players are created and 2 are computers	To test if normal data works correctly	Normal

208	Number of players: 4 Number of computers: 4	4 players are created and all 4 are computers	To test if the upper boundary data works correctly as the number of computers must be equal or less than the number of players	Boundary
209	Number of players: 4 Number of computers: 0	4 players are created and none are computers	To test if the lower boundary data works correctly as the number of computers must be equal or less than the number of players	Boundary
210	Number of players: 4 Number of computers: -1	Invalid input as number of players must be a positive number from 2 to 4	To test minimum of 0 computer players	Invalid
211	Number of players: 3 Number of computers: 4	Invalid number as there cant be more computers than players	To test the maximum of no more computers that the number of players	Invalid
212	Number of players: 4 Number of computers: abcd	Invalid input as number of players must be a number	To test that the user is unable to enter non-numbers	Erroneous

213	Name: Red	Player is given that name	To test if normal data works correctly	Normal
214	Name: abcdefghijkl	Player is given that name	To test the upper boundary of the 12 character limit	Boundary
215	Name: a	Player is given that name	To test the lower boundary of the 12 character limit	Boundary
216	Name: [BLANK]	Player is told to enter a name	To ensure that the name is required	Invalid
217	Name: 'abcdefghijklmnopqrst'	Player is told that the name is too long	To test if that name has a 12 character limit	Invalid
218	Number of players: 2 Number of computers: 0 Name of player 1: AA Name of player2: AA	Player2 is told that the name is already taken	To test that the program limits two players from having the same name	Invalid

Development

Evidence of iterative development

First Iteration:

Break Down of First Iteration:

```
import random
import pygame
from pygame.locals import *

#Initialises pygame library
pygame.init()
```

I started by importing the required modules I will need to use. I first import pygame as it is the main thing I will use to produce graphical outputs and to receive inputs. I will also import random as it is used for random number generation which I need for the dice rolls and the computer players decision making. I also make sure to call the 'pygame.init()' function which initialises all of the imported pygame modules allowing me to use the things it provides

```
#Constants:

#Property Colours
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
BROWN = (120, 78, 60)
SILVER = (186, 220, 238)
PINK = (170, 59, 126)
ORANGE = (214, 138, 55)
RED = (186, 42, 45)
YELLOW = (255, 242, 57)
GREEN = (95, 166, 92)
BLUE = (44, 103, 161)
LIGHTGREEN = (210, 226, 198)

#Player colours:
P_RED = (255, 0, 0)
P_GREEN = (0, 255, 0)
P_BLUE = (0, 0, 255)
P_YELLOW = (255, 255, 0)
P_TEAL = (0, 255, 200)
P_PURPLE = (128, 0, 255)

#Sizes of screen
SCREENWIDTH = 1920
SCREENHEIGHT = 1080

#Sizes of Board
BOARDWIDTH = 1078
BOARDHEIGHT = 1078

#Useful sizes
SQUARE_SIZE = BOARDWIDTH / 11
PLAYER_SIZE = SQUARE_SIZE / 4
CARD_WIDTH = SQUARE_SIZE * 4
CARD_HEIGHT = SQUARE_SIZE * 5
STATUS_CARD_WIDTH = ((SCREENWIDTH - BOARDWIDTH) / 2) - 1
STATUS_CARD_HEIGHT = (BOARDHEIGHT / 2)

#Thickness of the lines
LINEWIDTH = 1

#Whether anti aliasing is used
ANTI_ALIASING = True

#Different font sizes
FONT40 = pygame.font.Font("freesansbold.ttf", 40)
FONT30 = pygame.font.Font("freesansbold.ttf", 30)
FONT20 = pygame.font.Font("freesansbold.ttf", 20)
```

Next I define many of the constants that I will be using. I firstly create a bunch of the colours which I will be using regularly throughout the code. I could just write the tuple of the colour value when it is used but this way makes the code more readable and allows me to adjust specific colours easily. Then I define the screen sizes and board sizes and many other useful sizes. It is good practice as with the colours to define these separately as it is useful to have

them easily available and changeable. I also created an anti aliasing boolean which affects whether all of the text created on the screen by pygame in the program uses anti aliasing. I also create the three font sizes I will be using throughout.

```
#Creates the pygame display
screen = pygame.display.set_mode((SCREENWIDTH, SCREENHEIGHT), pygame.FULLSCREEN)
pygame.display.set_caption("Monopoly")

#Creating the random object
random = random.Random()

#Creating the clock object used to control fps
clock = pygame.time.Clock()

#Keeps track of which tile card is blitted
blitted_card = None

#Array of all players
players = []

#Keeps track of which players turn it is
current_player = 0
```

Afterwards I create some important objects and global variables that will be useful. The screen is what pygame uses to display graphics on and it is a window that can have its sizes changed to what I need. I also set the window caption of the pygame screen to monopoly. The random object is used to generate random numbers and it comes from the random module. Clock is a pygame class that can be used to control the fps of the code. I will use it to ensure that the game isn't running faster than it needs to. 'blitted_card' is an important variable that tracks which tile card is currently displayed. This is useful as when the main loop function ends and a new one begins, it is important to keep the same card displayed and not have it disappear. 'players' is an array which stores all of the player objects. 'current_player' is used to keep track of which players turn it is by storing their index in the 'players' array.

```
#Class for players
class Player():
    def __init__(self, given_name, given_id):
        #What space they are on
        self.position = 0
        #What name they were given
        self.name = given_name
        #How much money they have
        self.money = 1500
        #Identifies which player they are outside of the name and matches the index of the players array
        self.id = given_id
        #If they are bankrupt and out of the game
        self.dead = False
        #What their rolls were
        self.roll1 = None
        self.roll2 = None
        #The number of turns they have left in jail
        self.jailed = 0
        #If they just got a double
        self.double = False
        #How many doubles they got in a row
        self.double_count = 0
        #Chooses colour
        if self.id == 0:
            self.colour = P_RED
        elif self.id == 1:
            self.colour = P_GREEN
        elif self.id == 2:
            self.colour = P_BLUE
        elif self.id == 3:
            self.colour = P_YELLOW
```

Afterwards I define the classes starting with the player class. The player class has many important attributes that will be defined in the constructor method. Most of them are self explanatory such as position, name and money. ‘id’ is used to identify the player and it also matches a the index of that player on the ‘players’ array which is an array that stores all of the players in the game. ‘dead’ is a boolean which is determines if a player is out of the game. Roll1 and 2 are the values for each dice that is rolled. ‘jailed’ is used to determine if a player is in jail and it is an int with the number signifying how many turns they have left. ‘double’ shows if a player just rolled a double and ‘double_count’ is the number of doubles a player rolled in a row. ‘colour’ is the colour that the player icon will be when it is blitted to the board. For now it is just determined by the id.

```
def takeTurn(self):
    #Ends turn immediatly if player is out of the game
    if self.dead:
        return
    self.double_count = 0
    mainLoop(f"It is {self.name}'s go")
    #Checks if player is in jail and gives them option to pay to get out, if not the jail counter goes down by one
    if self.jailed > 0:
        if mainLoop(f"You are in jail with {self.jailed} turns left, you can pay 50 to get out", "Pay 50", "Dont pay 50"):
            self.jailed = 0
            board[20].free_parking_cash += 50
        else:
            self.jailed -= 1
    #Repeats turn while player has a double
    self.double = True
    while self.double:
        self.double = False
        #Gives input for player to roll dice
        mainLoop("Click to roll", "Roll")
        self.rollDice()
        mainLoop(f"You got a {self.roll1} and a {self.roll2}")
        #If the player gets 3 doubles in a row then they are sent to jail and the turn ends
        if self.double_count == 3:
            mainLoop("You got 3 doubles in a row and are now sent to jail")
            self.jail()
            return
        #Function is used to increase the position as animation can be added within the function later
        self.increasePos(self.roll1 + self.roll2)
        mainLoop(f"You landed on {board[self.position].name}")
        #Calls land on method for the space of the board where the player is
        board[self.position].landOn(self)
        #Checks if player is bankrupt
        if self.money < 0:
            mainLoop("You are bankrupt")
            self.dead = True
            return

#Creates 2 random dice rolls
def rollDice(self):
    self.roll1 = random.randint(1, 6)
    self.roll2 = random.randint(1, 6)
    if self.roll1 == self.roll2:
        self.double_count += 1
        self.double = True
    #add animation here later
```

The ‘takeTurn’ function is used to go through a players typical monopoly turn using the ‘mainLoop’ function as the output and inputs for it. It starts by checking if the player is out of the game and if they are then the turn is immediately ended. Then it resets the ‘double_count’ attribute so it is correct. Afterwards the ‘mainLoop’ function is called to output whose go it is. Since in monopoly when you roll a double you roll the dice again, I made the rest of the function in a while loop that repeats if the ‘double’ is true. If the player rolls a double three times in a row they will be sent to jail and there turn will end. When a player lands on any space then that spaces ‘landOn’ method will be called with the player as a argument. Due to the way python passes arguments, changing the parameter in method will affect the original value. At the end of the function it will check to see if the players money has gone below 0 and if so they are out of the game.

The ‘rollDice’ method creates two random numbers for each dice value. Later on in the development cycle there will be a dice animation here.

```
    #Increases the players position
    def increasePos(self, pos):
        self.position += pos
        if self.position > 39:
            mainLoop("You passed Go and get £200")
            self.money += 200
            self.position -= 40
        #Create animation later for this

    #Sets player status to jailed for 3 turns and move player to jail position
    def jail(self):
        self.jailed = 3
        self.position = 10
```

The ‘increasePos’ method adds the inputted number to the players position and uses logic to make it so when the player reaches the end of the board they go back to the start.

The ‘jail’ method sets the players ‘jailed’ value to 3 so they have 3 turns in jail and sets their position to the jail spot.

```
#Base super class that all tiles will inherit from
class Tiles():
    def __init__(self, given_name, given_pos):
        self.name = given_name
        self.position = given_pos

    def landOn(self, player):
        pass


    #All other spaces override it to add their own stuff then they call this method to complete common thinks like adding x button
    def showInfo(self, surface):
        #Creates x button at top left of card
        pygame.draw.rect(surface, RED, (CARD_WIDTH - 25 - LINEWIDTH, 0 + LINEWIDTH, 25, 25))
        surface.blit(FONT30.render("X", ANTI_ALIASING, BLACK), (CARD_WIDTH - 25, 0))
        screen.blit(surface, (421 + (6 * SQUARE_SIZE), SQUARE_SIZE + 1))
```

The ‘Tiles’ class is an abstract class which all of the spaces on the board inherit from as they all have similar attributes and methods.

The constructor for this class only assigns the common attributes that all tiles hold which is a name and a position. This means that for the subclass constructors I can call the superclass constructor inside it to avoid reusing code.

The ‘landOn’ method is an abstract method which is overwritten by the subclasses to do what needs to happen when a player lands on a space. The parameter is the player object which is passed by reference so the method can affect the players data.

The ‘showInfo’ method is used to create a surface which shows information about that specific space. The method is overwritten by the subclasses so they can create the correct information then the superclass method is called which does the work of blitting the surface to the main screen and creating an x button for the user to get rid of the blitted card.

```
#Class for jail tile
class Jail(Tiles):
    def __init__(self, given_name, given_pos):
        super().__init__(given_name, given_pos)


    def showInfo(self):
        offset = 0
        surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
        surface.fill((255, 255, 255))
        pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
        surface.blit(FONT30.render(self.name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11))
        surface.blit(FONT20.render("Players in jail:", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * 2))
        for i in range(len(players)):
            if players[i].jailed > 0:
                surface.blit(FONT20.render(players[i].name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * (3 + offset)))
                offset += 1
        super().showInfo(surface)
```

The Jail class has no ‘landOn’ method as nothing happens when you land on jail. The ‘showInfo’ method creates a list of all of the players in jail.

```
#Class for go tile
class Go(Tiles):
    def __init__(self, given_name, given_pos):
        super().__init__(given_name, given_pos)


    def showInfo(self):
        surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
        surface.fill((255, 255, 255))
        pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
        surface.blit(FONT30.render(self.name, True, BLACK), (10, CARD_HEIGHT /11))
        drawText(surface, "If you pass here you will get £200", BLACK, (10, CARD_HEIGHT / 11 * 2, CARD_WIDTH - 20, CARD_HEIGHT / 11 * 9), FO
        super().showInfo(surface)
```

The Go class also has nothing happen when you land on it and the ‘showInfo’ just explains how the space works.

```
#Class for free parking tile
class FreeParking(Tiles):
    def __init__(self, given_name, given_pos):
        super().__init__(given_name, given_pos)
        self.free_parking_cash = 0

    #Gives player the money stored on the tile
    def landOn(self, player):
        mainLoop(f"You get £{self.free_parking_cash}")
        player.money += self.free_parking_cash
        self.free_parking_cash = 0

    #Shows how much money is stored on the tile
    def showInfo(self):
        surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
        surface.fill((255, 255, 255))
        pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
        surface.blit(FONT30.render(self.name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11))
        surface.blit(FONT20.render(f"CASH £{self.free_parking_cash}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 *
        super().showInfo(surface)
```

The 'FreeParking' class has an attribute that stores how much money it has stored. When a player lands on the space the player gets the amount of money stored added to their money attribute and the money value of the space is emptied. The 'showInfo' method displays how much money is stored.

```
class Tax(Tiles):
    def __init__(self, given_name, given_pos):
        super().__init__(given_name, given_pos)

    #Player must pay money to free parking space
    def landOn(self, player):
        if self.position == 4:
            mainLoop(f"You must pay £200 tax")
            board[20].free_parking_cash += 200
            player.money -= 200
        else:
            mainLoop("You must pay £300 super tax")
            board[20].free_parking_cash += 300
            player.money -= 300

    #Shows how much money you will have to pay
    def showInfo(self):
        surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
        surface.fill((255, 255, 255))
        pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
        surface.blit(FONT30.render(self.name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11))
        surface.blit(FONT20.render(f"If you land here you must pay £{200 if self.position == 4 else 300}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /
        super().showInfo(surface)
```

The 'Tax' class is a class for the two tax spaces on the board. When a player lands on them they must pay the required amount which depends on which space it is. The money then goes to the free parking cash pile. The 'showInfo' method displays how much a player would have to pay if they landed on the space.

```
#Class for chance cards and community chest
class ChanceChests(Tiles):
    def __init__(self, name, position):
        super().__init__(name, position)

    def showInfo(self):
        pass
```

The ‘ChanceChests’ class is a class for handling chance and community chest cards. It will be completed next iteration.

```
#Class for go to jail space
class GoToJail(Tiles):
    def __init__(self, name, position):
        super().__init__(name, position)

    def landOn(self, player):
        mainLoop(f"You are sent to jail")
        player.jail()

    def showInfo(self):
        surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
        surface.fill((255, 255, 255))
        pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
        surface.blit(FONT30.render(self.name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11))
        surface.blit(FONT20.render(f"If you land here you are sent to jail", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * 2))
        super().showInfo(surface)
```

The ‘GoToJail’ class is for the go to jail space of the board which when a player lands on is sent to jail. The ‘showInfo’ methods simply displays how the space works.

```
#Class all buyable tiles inherit from
class Properties(Tiles):
    def __init__(self, given_name, given_pos, given_price):
        super().__init__(given_name, given_pos)
        self.price = given_price
        self.id_of_owner = None
        self.num_in_set = 1
        self.cost_to_land = 0

    def calcCostOfLanding(self):
        pass

    def reset(self):
        self.id_of_owner = None
        self.owned = False
        self.num_in_set = 1

    def getOwned(self, player):
        self.id_of_owner = player.id
        checkForSets()
```

The ‘Properties’ class is another abstract class which inherits from the ‘Tiles’ class. It is used for all the spaces which you can buy and has common attributes. The ‘price’ attribute is how much it costs to purchase, ‘id_of_owner’ stores who owns the property, ‘num_in_set’ stores how many other properties in the set as the target are owned by the same person and the ‘cost_to_land’ attributes dictates how much a player will have to pay to the owner when they land on the space. The ‘calcCostOfLanding’ method is used to calculate what the ‘cost_to_land’ attribute is. It is an abstract method so it will be inherited and overwritten by the subclasses. The ‘reset’ method is used to set all of the attributes of a property back to their default values so it can be repurchased after the owner player becomes bankrupt. The ‘getOwned’ method sets the ‘id_of_owner’ to a players id after that player purchases the property. It also calls the ‘checkForSets’ function which updates the ‘num_in_set’ value for all of the properties.

```python
#Class for all standard coloured sets
class Sets(Properties):
    def __init__(self, given_name, given_pos, given_price, given_rent, given_house1, given_house2, given_house3, given_house4, given_hotel, size_of_set):
        super().__init__(given_name, given_pos, given_price)
        self.rent = given_rent
        self.house1 = given_house1
        self.house2 = given_house2
        self.house3 = given_house3
        self.house4 = given_house4
        self.hotel = given_hotel
        self.num_of_houses = 0
        self.size_of_set = size_of_set
        if self.position < 10:
            self.house_price = 50
        elif self.position < 19:
            self.house_price = 100
        elif self.position < 29:
            self.house_price = 150
        else:
            self.house_price = 200

    #Method for when player lands on the space
    def landOn(self, player):
        if self.id_of_owner != None:
            if self.id_of_owner == player.id:
                mainLoop("You own this property")
            else:
                self.calcCostOfLanding()
                mainLoop(f"It is owned by {players[self.id_of_owner].name}")
                mainLoop(f"You must pay {self.cost_to_land}")
                player.money -= self.cost_to_land
                players[self.id_of_owner].money += self.cost_to_land

        elif self.price <= player.money:
            if mainLoop(f"Do you want to buy it for £{self.price}?", "Yes", "No"):
                self.getOwned(player)
                player.money -= self.price
                mainLoop(f"You purchased {self.name}")
            else:
                mainLoop(f"You didn't purchase {self.name}")
        else:
            mainLoop("It is unowned but you are too poor to afford it")

    def upgrade(self):
        if self.house_price > players[current_player].money:
            mainLoop("You are too poor to buy it")
        else:
            self.num_of_houses += 1
            players[current_player].money -= self.house_price
            mainLoop("You upgraded your property")
```

The ‘Sets’ class is for all of the coloured sets that can have houses on in the game. It inherits from properties. It has an attribute for all of the rent prices for the different numbers of

houses on it and an attribute for the number of houses on it. The attribute ‘size_of_set’ determines what the size of the set that the property belongs to is and the attribute ‘house_price’ determines how much it costs to buy houses for that property which is dependent on the position on the board. The ‘landOn’ method either lets the player buy the property or pay the owner of the property the required amount. The ‘upgrade’ method will add a house to the property if the player has enough money to afford it.

```
#Calculates how much you will have to pay if you land on the space
def calcCostOfLanding(self):
    if self.num_of_houses == 0:
        if self.num_in_set == self.size_of_set:
            self.cost_to_land = self.rent * 2
        else:
            self.cost_to_land = self.rent
        self.cost_to_land = self.rent
    elif self.num_of_houses == 1:
        self.cost_to_land = self.house1
    elif self.num_of_houses == 2:
        self.cost_to_land = self.house2
    elif self.num_of_houses == 3:
        self.cost_to_land = self.house3
    elif self.num_of_houses == 4:
        self.cost_to_land = self.house4
    elif self.num_of_houses == 5:
        self.cost_to_land = self.hotel

def showInfo(self):
    surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
    surface.fill((255, 255, 255))
    pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
    surface.blit(FONT30.render(self.name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11))
    surface.blit(FONT20.render(f"RENT: £{self.rent}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 2))
    surface.blit(FONT20.render(f"1 HOUSE: £{self.house1}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 3))
    surface.blit(FONT20.render(f"2 HOUSES: £{self.house2}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 4))
    surface.blit(FONT20.render(f"3 HOUSES: £{self.house3}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 5))
    surface.blit(FONT20.render(f"4 HOUSES: £{self.house4}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 6))
    surface.blit(FONT20.render(f"HOTEL: £{self.hotel}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 7))
    surface.blit(FONT20.render(f"OWNED BY: {players[self.id_of_owner].name} at level: {self.num_of_houses}" if self.id_of_owner != None else "Not owned by anybody", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 8))
    if self.num_in_set == self.size_of_set and self.id_of_owner == current_player:
        surface.blit(FONT20.render(f"Add house for £{self.house_price}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * 9))
        pygame.draw.rect(surface, RED, (5, (CARD_HEIGHT / 11 * 10) - 5, 100, 30))
        surface.blit(FONT20.render("Yes", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * 10))
    super().showInfo(surface)
```

‘calcCostOfLanding’ works out what the ‘cost_to_land’ value should be by looking at how many houses a property has and what the different rent values are for those numbers. The ‘showInfo’ method will show all of the different rents values for the different numbers of houses and who owns the property. If the it the owners turn and they own the whole of the set, then a button will appear allowing them to purchase a house on the property.

```python
#Class for all station spaces
class Station(Properties):
    def __init__(self, given_name, given_pos, given_price):
        super().__init__(given_name, given_pos, given_price)
        self.cost_to_land = 50
    def landOn(self, player):
        if self.id_of_owner != None:
            if self.id_of_owner == player.id:
                mainLoop("You own this station")
            else:
                self.calcCostOfLanding()
                mainLoop(f"It is owned by {players[self.id_of_owner].name}")
                mainLoop(f"You must pay them {self.cost_to_land}")
                player.money -= self.cost_to_land
                players[self.id_of_owner].money += self.cost_to_land

        elif self.price <= player.money:
            if mainLoop(f"Do you want to buy it for £{self.price}?", "Yes", "No"):
                self.getOwned(player)
                player.money -= self.price
                mainLoop(f"You purchased {self.name}")
            else:
                mainLoop(f"You didn't purchase {self.name}")
        else:
            mainLoop("It is unowned but you are too poor to afford it")

    def calcCostOfLanding(self):
        if self.num_in_set == 1:
            self.cost_to_land = 50
        elif self.num_in_set == 2:
            self.cost_to_land = 100
        elif self.num_in_set == 3:
            self.cost_to_land = 150
        else:
            self.cost_to_land = 200

    def showInfo(self):
        surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
        surface.fill((255, 255, 255))
        pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
        surface.blit(FONT30.render(self.name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11))
        surface.blit(FONT20.render(f"1 STATION: £50", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 2))
        surface.blit(FONT20.render(f"2 STATION: £100", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 3))
        surface.blit(FONT20.render(f"3 STATION: £150", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 4))
        surface.blit(FONT20.render(f"4 STATION: £200", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 5))
        surface.blit(FONT20.render(f"OWNED BY: {players[self.id_of_owner].name if self.id_of_owner != None else 'Nobody'}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 8))
        super().showInfo(surface)
```

The ‘Station’ class also inherits from properties. It is for the 4 stations that are on the board. They work like the sets but they cant have houses on and the 4 stations are part of a single set. As more are owned the rent will increase. The ‘landOn’ method is similar to the one for the

```
#Class for all utility spaces
class Utilities(Properties):
    def __init__(self, given_name, given_pos, given_price):
        super().__init__(given_name, given_pos, given_price)

    def landOn(self, player):
        if self.id_of_owner != None:
            if self.id_of_owner == player.id:
                mainLoop("You own this Utility")
            else:
                pay = (4 if self.num_in_set == 1 else 10) * (player.roll1 + player.roll2)
                mainLoop(f"It is owned by {players[self.id_of_owner].name}")
                mainLoop(f"You must pay them {(4 if self.num_in_set == 1 else 10)} times your last dice roll which is £{pay}")
                player.money -= pay
                players[self.id_of_owner].money += pay
        elif self.price <= player.money:
            if mainLoop(f"Do you want to buy it for £{self.price}?", "Yes", "No"):
                self.getOwned(player)
                player.money -= self.price
                mainLoop(f"You purchased {self.name}")
            else:
                mainLoop(f"You didn't purchase {self.name}")
        else:
            mainLoop("It is unowned but you are too poor to afford it")

    def showInfo(self):
        surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
        surface.fill((255, 255, 255))
        pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
        surface.blit(FONT30.render(self.name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11))
        drawText(surface, "If one Utility is owned then the rent will be 4 times the previous dice roll", BLACK, (10, CARD_HEIGHT / 11 * 2, CARD_WIDTH - 20, CARD_HEIGHT / 11 * 4), FONT20, ANTI_ALIASING)
        drawText(surface, "If both Utilities are owned then the rent will be 10 times the previous dice roll", BLACK, (10, CARD_HEIGHT / 11 * 4, CARD_WIDTH - 20, CARD_HEIGHT / 11 * 6), FONT20, ANTI_ALIASING)
        surface.blit(FONT20.render(f"OWNED BY: {players[self.id_of_owner].name if self.id_of_owner != None else 'Nobody'}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 8))
        super().showInfo(surface)
```

‘Sets’ class. ‘calcCostOfLanding’ uses the ‘num_in_set’ value to decide the ‘cost_of_landing’. The ‘showInfo’ method displays the different rent values for different numbers of stations owned and the owner of the station.

The ‘Utilities’ class inherits from properties like the other two and it is very similar to them. When a player lands on the property, the amount they pay to the owner is based on the previous roll. If one property is owned it is 4 times the roll, if both then 10. The ‘showInfo’ method explains this and displays the owner.

```
#Draws the detail for each section of the board and blits it onto the main screen surface
def drawBoard():
    #Fills the background
    screen.fill(LIGHTGREEN)
    #Creates the 4 surfaces that make up the board
    bottom_surf = pygame.surface.Surface((SQUARE_SIZE * 10, SQUARE_SIZE))
    left_surf = pygame.surface.Surface((SQUARE_SIZE * 10, SQUARE_SIZE))
    top_surf = pygame.surface.Surface((SQUARE_SIZE * 10, SQUARE_SIZE))
    right_surf = pygame.surface.Surface((SQUARE_SIZE * 10, SQUARE_SIZE))

    #Creates the common base features for each surface
    def drawBoardBase(surf):
        surf.fill((LIGHTGREEN))
        pygame.draw.rect(surf, BLACK, (0, 0, SQUARE_SIZE * 10, SQUARE_SIZE), LINEWIDTH)
        for i in range(0, 10):
            pygame.draw.rect(surf, BLACK, (SQUARE_SIZE * i, 0, SQUARE_SIZE, SQUARE_SIZE), LINEWIDTH)

    #Adds the squares the 4 surfaces
    drawBoardBase(bottom_surf)
    drawBoardBase(left_surf)
    drawBoardBase(top_surf)
    drawBoardBase(right_surf)

    #Draws all the coloured parts for each property and blits the images
    pygame.draw.rect(screen, (0,0,0), (SQUARE_SIZE + 421, SQUARE_SIZE + 1, BOARDWIDTH - (2 * SQUARE_SIZE), BOARDWIDTH - (2 * SQUARE_SIZE)), LINEWIDTH)
    #Old Kent Road
    pygame.draw.rect(bottom_surf, BROWN, ((SQUARE_SIZE * 8) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Community Chest 1
    #Whitechapel Road
    pygame.draw.rect(bottom_surf, BROWN, ((SQUARE_SIZE * 6) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Income Tax
    #Kings Cross Station
    #The Angel Islington
    pygame.draw.rect(bottom_surf, SILVER, ((SQUARE_SIZE * 3) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Chance 1
    #Euston Road
    pygame.draw.rect(bottom_surf, SILVER, ((SQUARE_SIZE * 1) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Pentonville Road
    pygame.draw.rect(bottom_surf, SILVER, ((SQUARE_SIZE * 0) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Jail
    #Pall Mall
    pygame.draw.rect(left_surf, PINK, ((SQUARE_SIZE * 8) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Electric Company
    #Whitehall
    pygame.draw.rect(left_surf, PINK, ((SQUARE_SIZE * 6) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Northumberland Avenue
    pygame.draw.rect(left_surf, PINK, ((SQUARE_SIZE * 5) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Marylebone Station
```

```
    #Marylebone Station
    #Bow Street
    pygame.draw.rect(left_surf, ORANGE, ((SQUARE_SIZE * 3) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Community Chest 2
    #Marlborough Street
    pygame.draw.rect(left_surf, ORANGE, ((SQUARE_SIZE * 1) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Vine Street
    pygame.draw.rect(left_surf, ORANGE, ((SQUARE_SIZE * 0) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Free Parking
    #Strand
    pygame.draw.rect(top_surf, RED, ((SQUARE_SIZE * 8) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Chance 2
    #Fleet Street
    pygame.draw.rect(top_surf, RED, ((SQUARE_SIZE * 6) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Trafalgar Square
    pygame.draw.rect(top_surf, RED, ((SQUARE_SIZE * 5) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Fenchurch Street Station
    #Leicester Square
    pygame.draw.rect(top_surf, YELLOW, ((SQUARE_SIZE * 3) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Coventry Street
    pygame.draw.rect(top_surf, YELLOW, ((SQUARE_SIZE * 2) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Water Works
    #Piccadilly
    pygame.draw.rect(top_surf, YELLOW, ((SQUARE_SIZE * 0) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Go To Jail
    #Regent Street
    pygame.draw.rect(right_surf, GREEN, ((SQUARE_SIZE * 8) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Oxford Street
    pygame.draw.rect(right_surf, GREEN, ((SQUARE_SIZE * 7) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Community Chest 3
    #Bond Street
    pygame.draw.rect(right_surf, GREEN, ((SQUARE_SIZE * 5) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Liverpool Street Station
    #Chance 3
    #Park Lane
    pygame.draw.rect(right_surf, BLUE, ((SQUARE_SIZE * 2) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Super Tax
    #Mayfair
    pygame.draw.rect(right_surf, BLUE, ((SQUARE_SIZE * 0) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
```

Here is the function that is used to create the board graphic. It starts by filling in the background of the screen in green and creating the 4 separate surfaces that will be used to create the 4 sides of the board. I then define an inner function that draws the squares along all of the surfaces as they all have the same layout of squares. I use an inner function as I only need to use it within this 'drawBoard' function so I don't want it to be accessible from outside.

After creating the base of all of the surfaces I must add the colours to all of the set spaces, I haven't added any images or other detail this iteration and that will be done later as it isn't an important feature. Since there is no pattern to the placement of these sets I must manually draw each colour to the correct surface in the correct space. I also label, with a comment, what each space is that I am detailing to make it more readable.

Different surfaces before they have been put into position:

Afterwards I create another inner function for use in the code for adding the player icons onto the board. To ensure that multiple players didn't overlap when on the same space I had to use some mathematics to prevent this. The 'getNumOnSpaceBefore' inner function takes a position input and a player id input and returns how many players are on that space which have a lower id than the starting player. This value is then used to decide how offset the player must be to ensure it doesn't overlap with the earlier players. There is also an if statement to decide which surface to blit the player to.

```
#Counts the number of players on a space with an id lower than the chosen player
def getNumOnSpaceBefore(pos,id):
    count = 0
    for i in range(id+1):
        if pos == players[i].position:
            count += 1
    return count

#Code used to blit player squares onto board
for player in players:
    #Gets the number of players before it in the player list to decide how much the current player needs to be moved by to make room
    num_on_space = getNumOnSpaceBefore(player.position, player.id)
    #Value of how much player must be moved by
    offset = 0
    if num_on_space == 2:
        offset = PLAYER_SIZE
    elif num_on_space == 3:
        offset = PLAYER_SIZE * 2
    elif num_on_space == 4:
        offset = PLAYER_SIZE * 3

    pos = player.position

    #If statement to choose which surface the player needs to be blitted on
    if pos < 10:
        pygame.draw.rect(bottom_surf, player.colour, (((SQUARE_SIZE * (9 - pos)) + offset), SQUARE_SIZE - PLAYER_SIZE, PLAYER_SIZE, PLAYER_SIZE), 0)
    elif pos < 20:
        pos -= 10
        pygame.draw.rect(left_surf, player.colour, (((SQUARE_SIZE * (9 - pos)) + offset), SQUARE_SIZE - PLAYER_SIZE, PLAYER_SIZE, PLAYER_SIZE), 0)
    elif pos < 30:
        pos -= 20
        pygame.draw.rect(top_surf, player.colour, (((SQUARE_SIZE * (9 - pos)) + offset), SQUARE_SIZE - PLAYER_SIZE, PLAYER_SIZE, PLAYER_SIZE), 0)
    else:
        pos -= 30
        pygame.draw.rect(right_surf, player.colour, (((SQUARE_SIZE * (9 - pos)) + offset), SQUARE_SIZE - PLAYER_SIZE, PLAYER_SIZE, PLAYER_SIZE), 0)
```

```
#Code for creating player status cards
status_cards = [None] * num_of_players
#Creates one status card for each player
for i in range(num_of_players):
    #Creates surface for the text to be blitted onto
    status_cards[i] = pygame.surface.Surface((STATUS_CARD_WIDTH, STATUS_CARD_HEIGHT))
    status_cards[i].fill(LIGHTGREEN)
    pygame.draw.rect(status_cards[i], BLACK, (0, 0, STATUS_CARD_WIDTH, STATUS_CARD_HEIGHT), LINEWIDTH)
    #Blits the players name and their money count
    status_cards[i].blit(FONT30.render(players[i].name, ANTI_ALIASING, BLACK), (10, STATUS_CARD_HEIGHT / 28 * 1))
    status_cards[i].blit(FONT20.render(f"Money: £{players[i].money}", ANTI_ALIASING, BLACK), (10, (STATUS_CARD_HEIGHT / 28 * 2) + 10))
    pos = 3
    #Loops through all properties to blit the ones that the player owns, when it is blitted the pos value increases so the next one wont overlap
    for j in board:
        if isinstance(j, Properties):
            if j.id_of_owner == i:
                status_cards[i].blit(FONT20.render(f"{j.name} level: {j.num_of_houses}" if isinstance(j, Sets) else f"{j.name}", ANTI_ALIASING, BLACK), (10, (STATUS_CARD_HEIGHT / 28 * pos) + 10))
                pos += 1
    #If the player is out of the game the text dead will appear over their card
    if players[i].dead:
        dead_text = FONT40.render("DEAD", ANTI_ALIASING, RED)
        dead_text = pygame.transform.scale2x(dead_text)
        dead_text = pygame.transform.scale2x(dead_text)
        dead_text = pygame.transform.rotate(dead_text, -45)
        status_cards[i].blit(dead_text, (0,0))
#blitting the cards onto the screen
screen.blit(status_cards[0], (1, 1))
screen.blit(status_cards[1], (1, STATUS_CARD_HEIGHT + 1))
if num_of_players > 2:
    screen.blit(status_cards[2], (421 + BOARDWIDTH, 1))
if num_of_players > 3:
    screen.blit(status_cards[3], (421 + BOARDWIDTH, 1 + STATUS_CARD_HEIGHT))


#Rotating and blitting all the surfaces onto the main display
screen.blit(bottom_surf, (STATUS_CARD_WIDTH + 1 + SQUARE_SIZE, (SQUARE_SIZE * 10) + 1))
left_surf_rot = pygame.transform.rotate(left_surf, 90)
left_surf_rot = pygame.transform.flip(left_surf_rot,  True, True)
screen.blit(left_surf_rot, (STATUS_CARD_WIDTH + 1, (SQUARE_SIZE) + 1))
top_surf_rot = pygame.transform.flip(top_surf, True, True)
screen.blit(top_surf_rot, (STATUS_CARD_WIDTH + 1, 1))
right_surf_rot = pygame.transform.rotate(right_surf, -90)
right_surf_rot = pygame.transform.flip(right_surf_rot, True, True)
screen.blit(right_surf_rot, (STATUS_CARD_WIDTH + 1 + SQUARE_SIZE * 10, 1))
pygame.draw.rect(screen, BLACK, (0, 0, 1920, 1080), 1)
```

After there is code that creates the status cards for each player which provide information about the players properties and money. A list of surfaces is created the size of the number of players in the game and each one is blitted with text about the name of the player and the amount of money they have. A for loop is then used to go through all of the properties and blit each one owned by that player along with how many houses it has. These status cards are then blitted onto their correct places on the main screen.

Finally the 4 surfaces are rotated and blitted into the correct positions on the main screen.

```
#Main game loop with different parameters to do different things
def mainLoop(text, input1="Next", input2=None):
    #bliited card global keeps track of which card is blitted
    global blitted_card
    drawBoard()
    if blitted_card != None:
        board[blitted_card].showInfo()
    # blitted_card = None
    # genTilePos()
    #Draws boxes for all the text
    pygame.draw.rect(screen, WHITE, (551, 116, 424, 83)) #Box and outline for  player turn
    pygame.draw.rect(screen, BLACK, (551, 116, 424, 83), LINEWIDTH)
    pygame.draw.rect(screen, WHITE, (551, 224, 424, 83)) #Box and outline for money
    pygame.draw.rect(screen, BLACK, (551, 224, 424, 83), LINEWIDTH)
    pygame.draw.rect(screen, WHITE, (551, 332, 424, 257)) #Box and outline for text
    pygame.draw.rect(screen, BLACK, (551, 332, 424, 257), LINEWIDTH)
    #Blits all the text
    drawText(screen, f"{players[current_player].name} Turn", BLACK, (551, 116 + 5, 424, 83), FONT40, ANTI_ALIASING)
    drawText(screen, f"Money: {players[current_player].money}", BLACK, (551, 224 + 5, 424, 83), FONT40, ANTI_ALIASING)
    drawText(screen, text, BLACK, (551, 332 + 5, 424, 257), FONT40, ANTI_ALIASING)
    next_rect = pygame.rect.Rect(551, 614, 161, 82)
    button2_rect = pygame.rect.Rect(814, 614, 161, 82)
    #First button is blitted
    pygame.draw.rect(screen, RED, next_rect)
    screen.blit(FONT30.render(input1, ANTI_ALIASING, BLACK), (551, 614))
    #Creates second button if existence of input2 arguement
    if input2 != None:
        pygame.draw.rect(screen, RED, button2_rect)
        screen.blit(FONT30.render(input2, ANTI_ALIASING, BLACK), (814, 614))
    #Updates the screen so all of the changes appear
    pygame.display.flip()
    while True:
```

The main loop function is one of the most important functions in the code, as it is in charge of handling all of the inputs and outputs. There are three parameters: the text that will be displayed, the text on the first input and the text on the other input. When the user clicks on the first input, the function will return true and if they click on the other it will return false.

```
while True:
    #Event loop used to collect inputs
    for event in pygame.event.get():
        #If the user tries to close the window, pygame and python will stop
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()
        #If the user clicks the mouse button the code will check if the mouse is hovered over a button
        if event.type == MOUSEBUTTONDOWN:
            #Checks each pixel if mouse is over it
            for i in range(len(pixels)):
                tile_rect = pygame.rect.Rect(pixels[i][0], pixels[i][1], SQUARE_SIZE, SQUARE_SIZE)
                if tile_rect.collidepoint((pygame.mouse.get_pos())):
                    #Shows info about the clicked on tile
                    board[i].showInfo()
                    #Updates the part of the screen where the tile is so the rest isnt affected
                    pygame.display.update((421 + (6 * SQUARE_SIZE), SQUARE_SIZE + 1, CARD_WIDTH, CARD_HEIGHT))
                    blitted_card = i
            if blitted_card != None:
                #Check if the blitted card tile is a property
                if isinstance(board[blitted_card], Sets):
                    #Check if the blitted card property is owned by the player and can have houses
                    if board[blitted_card].num_in_set == board[blitted_card].size_of_set and board[blitted_card].id_of_owner == current_player:
                        #Check if the player clicked to buy another house
                        if pygame.Rect(5 + 421 + (6 * SQUARE_SIZE), (CARD_HEIGHT / 11 * 10) - 5 + SQUARE_SIZE + 1  , 100, 30).collidepoint((pygame.mouse.get_pos())):
                            board[blitted_card].upgrade()
                #If the x button is clicked the card will stop being blitted
                if pygame.Rect((CARD_WIDTH - 25 - LINEWIDTH + 421 + (6 * SQUARE_SIZE), 0 + LINEWIDTH + SQUARE_SIZE + 1, 25 , 25)).collidepoint(pygame.mouse.get_pos()):
                    drawBoard()
                    pygame.display.update((421 + (6 * SQUARE_SIZE), SQUARE_SIZE + 1, CARD_WIDTH, CARD_HEIGHT))
                    blitted_card = None

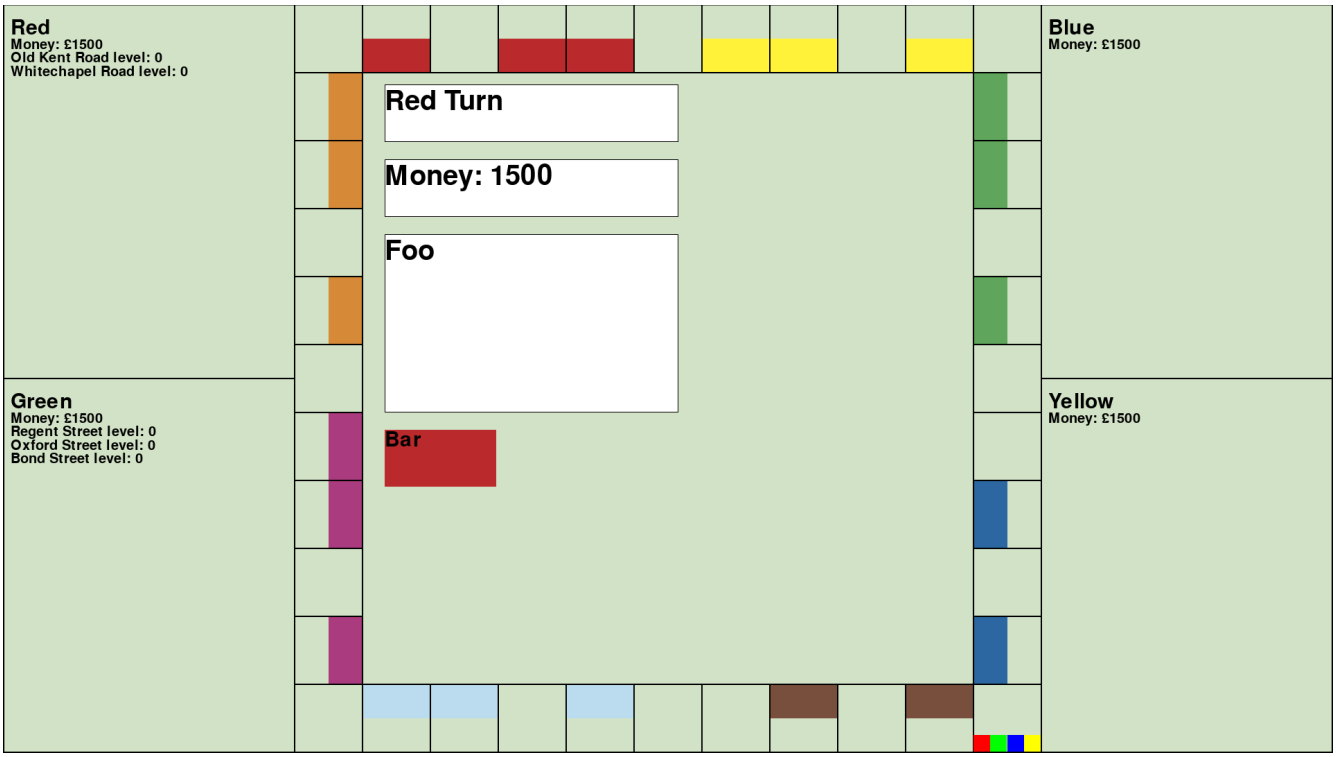
            #Check if one of the button was clicked to decide what to return
            if next_rect.collidepoint(pygame.mouse.get_pos()):
                return True
            if input2 != None:
                if button2_rect.collidepoint(pygame.mouse.get_pos()):
                    return False
    #Limits the framerate to 30 frames per second
    clock.tick(30)
```

This means the function can be used to create inputs. The main way it will be used is to display text information to the player and the first button will default to saying next allowing the player to confirm they have read it. The second button only appears if it is given a value.

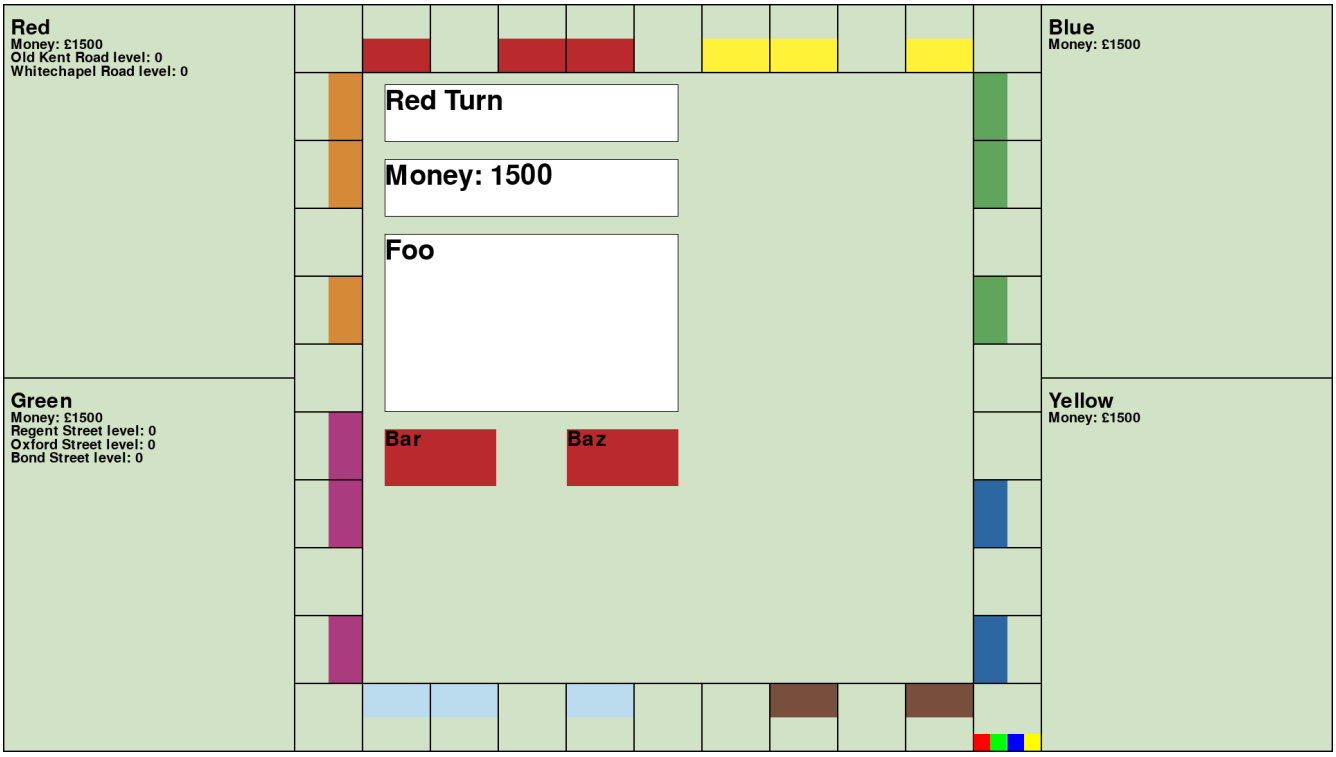
The function uses a while true loop and an event loop to wait for and gather user inputs to process them. Before the loop begins, all of the text and important information is blitted to the screen and the screen is updated so that it is all displayed. During the while loop, the event loop collects user inputs. If it detects that a mouse button has been clicked, it will check if the mouse was on top of any buttons to decide if they were clicked. It also uses another for loop to check through all instances of a 2D array called ‘pixels’ which has the top left corner position stored for each tile. This is used to find out if the mouse if clicking on a tile on the board and if so which one. If the mouse has clicked on a tile then the ‘showInfo’ method for that tile is called which displays some information about it on the screen.

When a card is blitted, the code also checks if the x button has been clicked which will stop showing the card. If the blitted card is able to have houses be bought on it and it is the owners turn then a button will appear on it allowing the player to buy a house. At the end on the while loop the function ‘clock.tick()’ is called which limits the number of while loops per second to the argument given. It is set to 30 as the game isn’t a real time game so differences in frame rate aren’t noticeable therefore a lower frame rate will save on the CPU usage.

If I called the code 'mainLoop("Foo", "Bar")' It would output:



And if I call 'mainLoop("Foo", "Bar", "Baz") it will output:



If the player clicks on a tile and they can buy houses on it it will display:




```python
#Used to make the text fit into boxes
def drawText(surface, text, color, rect, font, aa=False, bkg=None):
    rect = pygame.rect.Rect(rect)
    y = rect.top
    lineSpacing = -2
    #get the height of the font
    fontHeight = font.size("Tg")[1]
    while text:
        i = 1
        #determine if the row of text will be outside our area
        if y + fontHeight > rect.bottom:
            break
        #determine maximum width of line
        while font.size(text[:i])[0] < rect.width and i < len(text):
            i += 1
        #if we've wrapped the text, then adjust the wrap to the last word
        if i < len(text):
            i = text.rfind(" ", 0, i) + 1
        #render the line and blit it to the surface
        if bkg:
            image = font.render(text[:i], 1, color, bkg)
            image.set_colorkey(bkg)
        else:
            image = font.render(text[:i], aa, color)
        surface.blit(image, (rect.left, y))
        y += fontHeight + lineSpacing
        #remove the text we just blitted
        text = text[i:]
    return text
```

In order to make text wrap so it didn't overflow I found a function online called 'drawText' that fitted text to a specific size.

Link: https://www.pygame.org/wiki/TextWrap

```
#Finds out the number of properties owned by the same player as the target property and puts values as num_in_set
def checkForSets():
    number = 0
    for i in sets:
        for j in range (0, len(i)):
            for k in range (0, len(i)):
                if board[i[j]].id_of_owner != None:
                    if board[i[j]].id_of_owner == board[i[k]].id_of_owner:
                        number += 1
            board[i[j]].num_in_set = number
            number = 0
```

To calculate the 'num_in_set' value for the properties there is a function called 'checkForSets'. It loops through a tuple called 'sets', which contains multiple tuples for each set which contain the indexes of each member of the set, and checks each member of the set against each other to find out the number of others that have the same 'id_of_owner'. This number then becomes that properties 'num_in_set' value.

```
#Is a function as it causes all separate tiles to be local variables and not be able to be used in global scope
def createTiles():

    go = Go(                        "Go"                        , 0)
    old_kent_road = Sets(           "Old Kent Road"             , 1, 60 , 2, 10,    30,  90,  160,   250,  2)
    communityChest1 = ChanceChests("Community Chest"            , 2)
    whitechapel_road = Sets(        "Whitechapel Road"          , 3 , 60, 4, 20,    60,  180,  320,  450,  2)
    income_tax = Tax(               "Income Tax"                , 4)
    kings_cross = Station(          "Kings Cross Station"       , 5 , 200)
    the_angel_islington = Sets(     "The Angel Islington"       , 6 , 100, 6, 30,   90,  270,  400,  550,  3)
    chance1 = ChanceChests(         "Chance"                    , 7)
    euston_road = Sets(             "Euston Road"               , 8 , 100, 6, 30,   90,  270,  400,  550,  3)
    pentonville_road = Sets(        "Pentonville Road"          , 9 , 120, 8, 40,   100, 300,  450,  600,  3)
    jail = Jail(                    "Jail"                      , 10)
    pall_mall = Sets(               "Pall Mall"                 , 11, 140, 10, 50,  150, 450,  625,  750,  3)
    electric_company = Utilities(   "Electric Company"          , 12, 150)
    whitehall = Sets(               "Whitehall"                 , 13, 160, 10, 50,  150, 450,  625,  750,  3)
    northumberland_avenue = Sets(   "Northumberland Avenue"     , 14, 160, 12, 60,  180, 500,  700,  900,  3)
    marylebone = Station(           "Marylebone Station"        , 15, 200)
    bow_street = Sets(              "Bow Street"                , 16, 180, 14, 70,  200, 550,  750,  950,  3)
    communityChest2 = ChanceChests("Community Chest"            , 17)
    marlborough_street = Sets(      "Marlborough Street"        , 18, 180, 14, 70,  200, 550,  750,  950,  3)
    vine_street = Sets(             "Vine Street"               , 19, 200, 16, 80,  220, 600,  800,  1000, 3)
    free_parking = FreeParking(     "Free Parking"              , 20)
    strand = Sets(                  "Strand"                    , 21, 220, 18, 90,  250, 700,  875,  1050, 3)
    chance2 = ChanceChests(         "Chance"                    , 22)
    fleet_street = Sets(            "Fleet Street"              , 23, 220, 18, 90,  250, 700,  875,  1050, 3)
    trafalgar_square = Sets(        "Trafalgar Square"          , 24, 240, 20, 100, 300, 750,  925,  1100, 3)
    fenchurch_street = Station(     "Fenchurch Street Station", 25, 200)
    leicester_square = Sets(        "Leicester Square"          , 26, 260, 22, 110, 330, 800,  975,  1150, 3)
    coventry_street = Sets(         "Coventry Street"           , 27, 260, 22, 110, 330, 800,  975,  1150, 3)
    water_works = Utilities(        "Water Works"               , 28, 150)
    piccadilly = Sets(              "Piccadilly"                , 29, 280, 24, 120, 360, 850,  1025, 1200, 3)
    go_to_jail = GoToJail(          "Go to Jail"                , 30)
    regent_street = Sets(           "Regent Street"             , 31, 300, 26, 130, 390, 900,  1100, 1275, 3)
    oxford_street = Sets(           "Oxford Street"             , 32, 300, 26, 130, 390, 900,  1100, 1275, 3)
    communityChest3 = ChanceChests("Community Chest"            , 33)
    bond_street = Sets(             "Bond Street"               , 34, 320, 28, 150, 450, 1000, 1200, 1400, 3)
    liverpool_street = Station(     "Liverpool Street Station", 35, 200)
    chance3 = ChanceChests(         "Chance"                    , 36,)
    park_lane = Sets(               "Park Lane"                 , 37, 350, 35, 175, 500, 1100, 1300, 1500, 2)
    super_tax = Tax(                "Super Tax"                 , 38)
    mayfair = Sets(                 "Mayfair"                   , 39, 400, 50, 200, 600, 1400, 1700, 2000, 2)

    board = (
```

```
    board = (
        go,
        old_kent_road,
        communityChest1,
        whitechapel_road,
        income_tax,
        kings_cross,
        the_angel_islington,
        chance1,
        euston_road,
        pentonville_road,
        jail,
        pall_mall,
        electric_company,
        whitehall,
        northumberland_avenue,
        marylebone,
        bow_street,
        communityChest2,
        marlborough_street,
        vine_street,
        free_parking,
        strand,
        chance2,
        fleet_street,
        trafalgar_square,
        fenchurch_street,
        leicester_square,
        coventry_street,
        water_works,
        piccadilly,
        go_to_jail,
        regent_street,
        oxford_street,
        communityChest3,
        bond_street,
        liverpool_street,
        chance3,
        park_lane,
        super_tax,
        mayfair
    )
    sets = (
        (1, 3),
        (5, 15, 25, 35),
        (6, 8, 9),
        (11, 13, 14),
        (16, 18, 19),
        (21, 23, 24),
        (26, 27, 29),
        (31, 32, 34),
        (37, 39)
    )

    return board, sets
```

The ‘createTiles’ function is used to instantiate each tile on the board. All of the data for the tiles is written here. These tiles are then put into an array called ‘board’ and the tuple ‘sets’ is

also created here. Both are then returned and given to values outside the function. The reason this was made a function was to make the objects within the function local variables and to break up the code to make it more readable.

```
#Generates top left square_size for every board position, used to find rects to make tiles clickable
pixels = [None] * 40
for i in range (0, 10):
    pixels[i] = (((10 - i) * SQUARE_SIZE) + 421, BOARDHEIGHT - SQUARE_SIZE)
    if i == 10:
        continue
    pixels[i + 10] = (421, SQUARE_SIZE * (10 - i))
    pixels[i + 20] = ((SQUARE_SIZE * i) + 421, 0)
    pixels[i + 30] = (BOARDWIDTH - SQUARE_SIZE + 421, (SQUARE_SIZE * i) + 1)
```

To find the values for ‘pixels’, which holds the top left corner position for each tile on the board, I create some code to calculate it. A for loop is used that goes from 1 to 10 and each pass the next tile location is found for each of the four sides of the board.

```
#Creates all the board tiles and the array of sets
board, sets = createTiles()
#The names of the different players, will be created by the menu
names = ["Red", "Green", "Blue", "Yellow"]
num_of_players = 4
#instantiates players using above data
for i in range(num_of_players):
    players.insert(i, Player(names[i], i))


#Loops through all players and make them take their turn
while True:
    players[current_player].takeTurn()
    current_player += 1
    current_player = current_player % num_of_players
    players_left = 0
    for i in players:
        if not i.dead:
            players_left += 1
    if players_left == 1:
        break

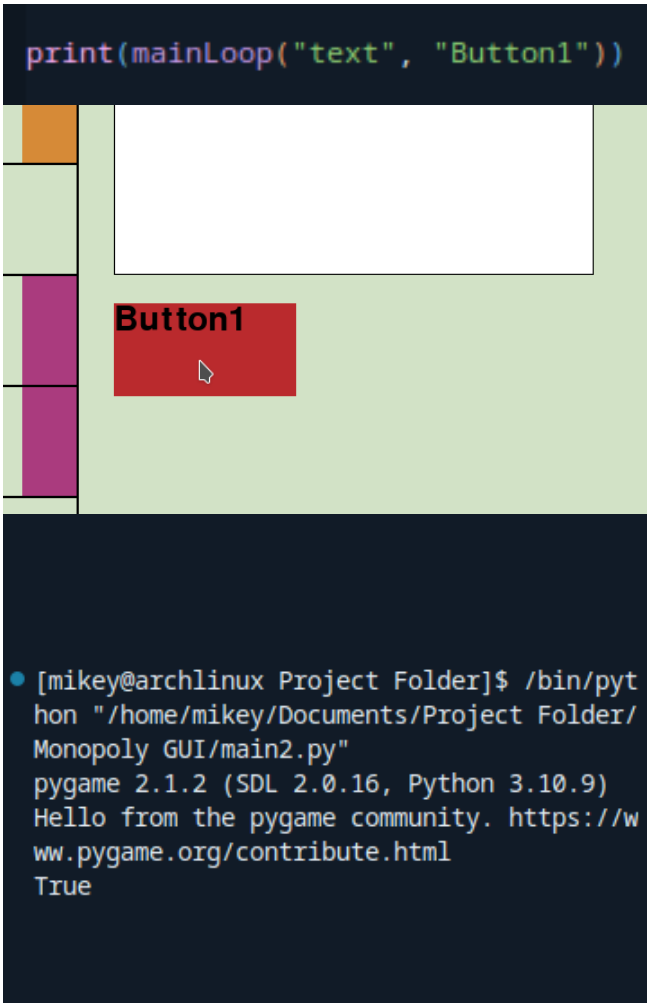
#Finds out who is left and congratulates them
for i in players:
    if not i.dead:
        mainLoop(f"{i.name} Wins!!")

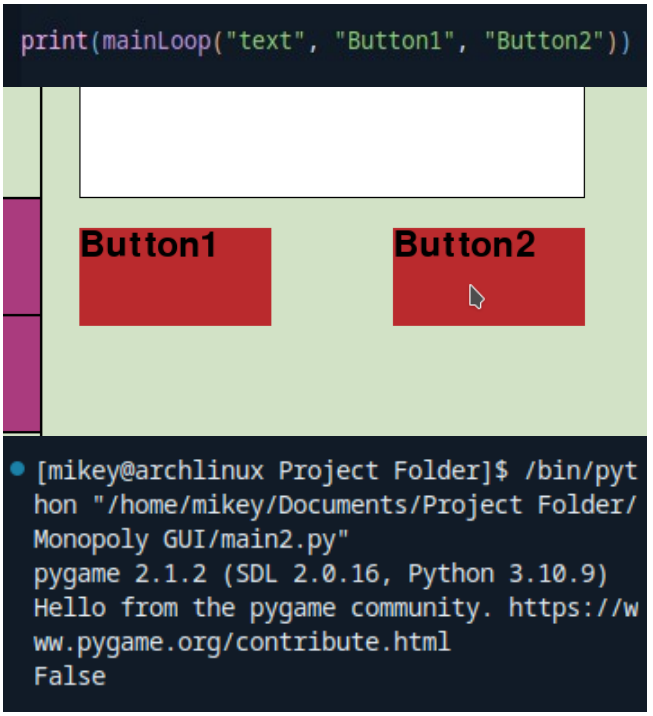
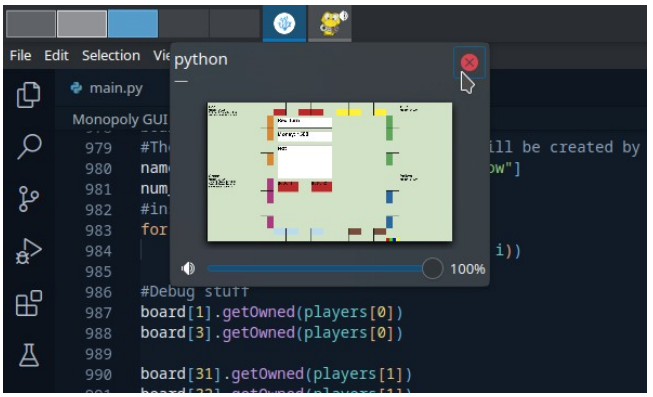
pygame.quit()
```

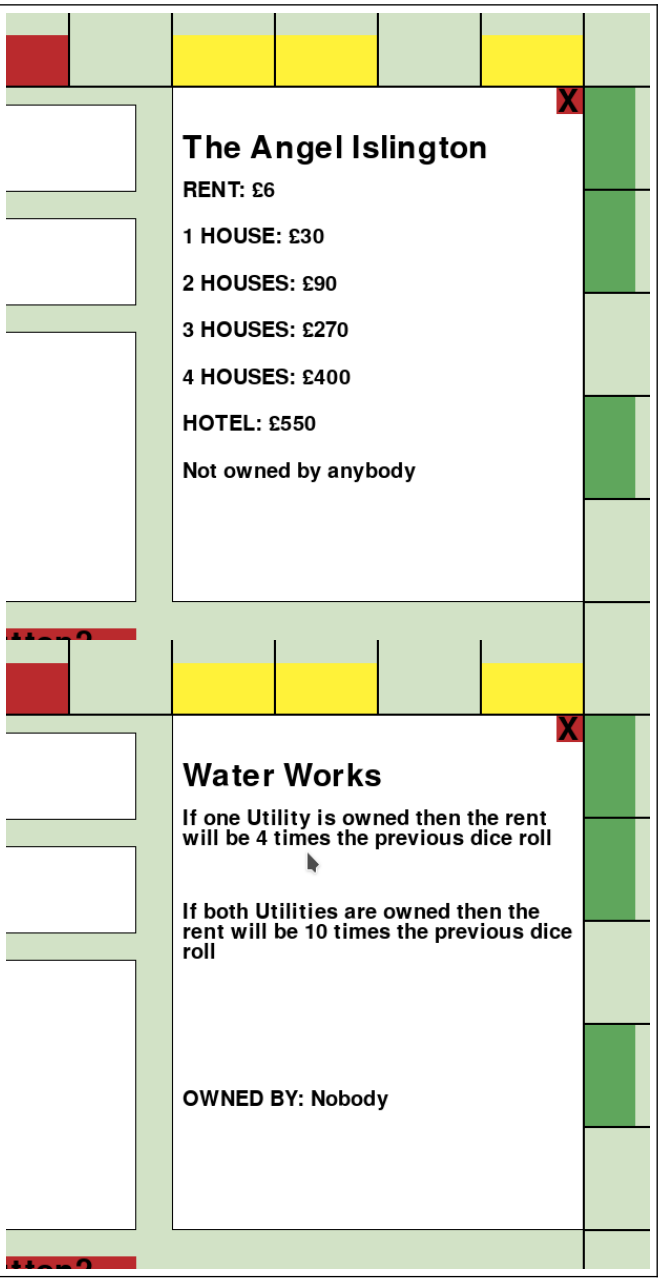
As I haven't created the main menu yet, the number and names of the players are created in the code in an array. A for loop goes through and instantiates each player. Each player is put into the 'players' array which makes it easy to access them.

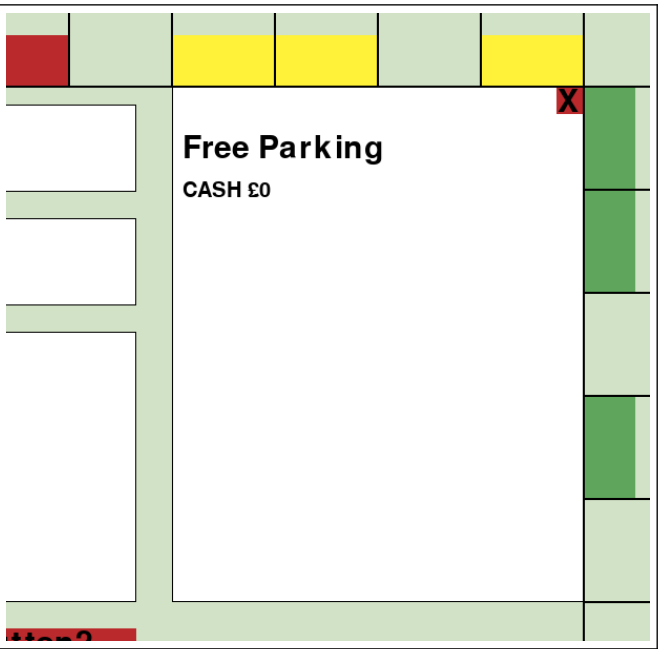
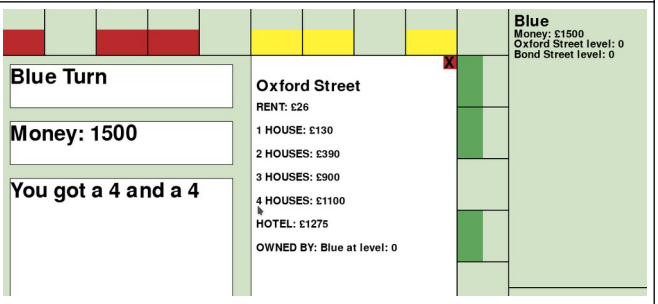
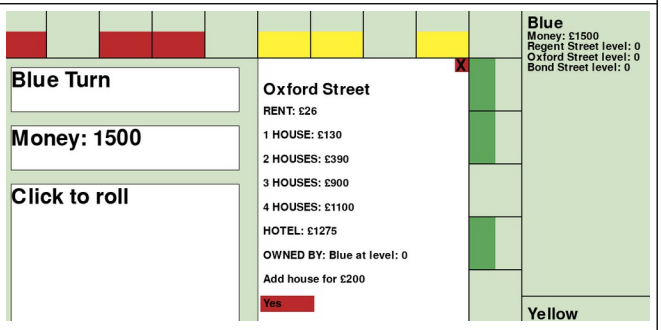
In order for the game to be played, a while true loop is used in order to cycle through each player giving each one a turn repeatedly. The lines of code: 'current_player += 1' and 'current_player % num_of_player' allow it to loop from 1 to the number of players repeatedly. At the end of each loop a for loop is used to check how many players are left. If only one remains then the while loop is broken. Another for loop is then used to find out who the winner is and congratulate them.

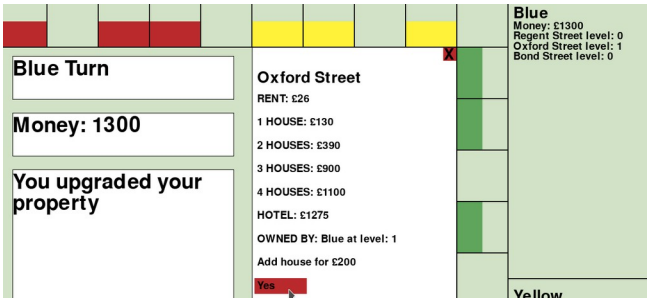
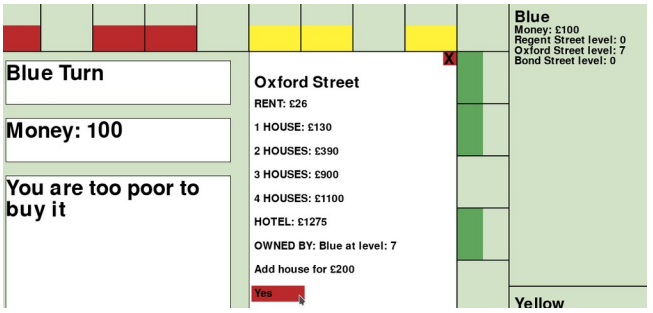
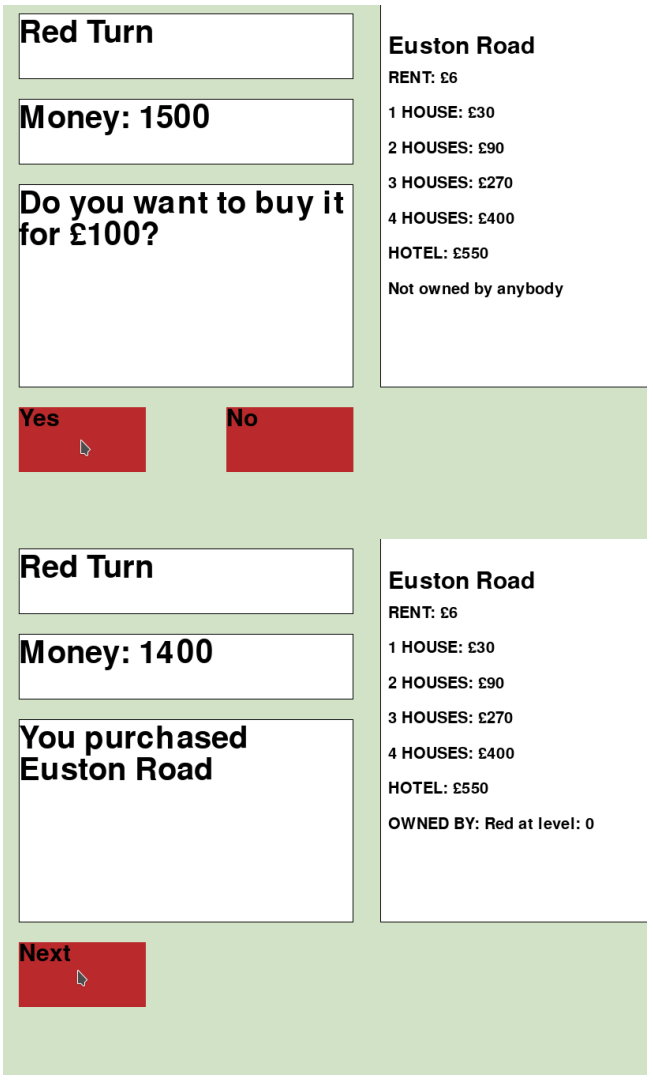
Testing of First Iteration:

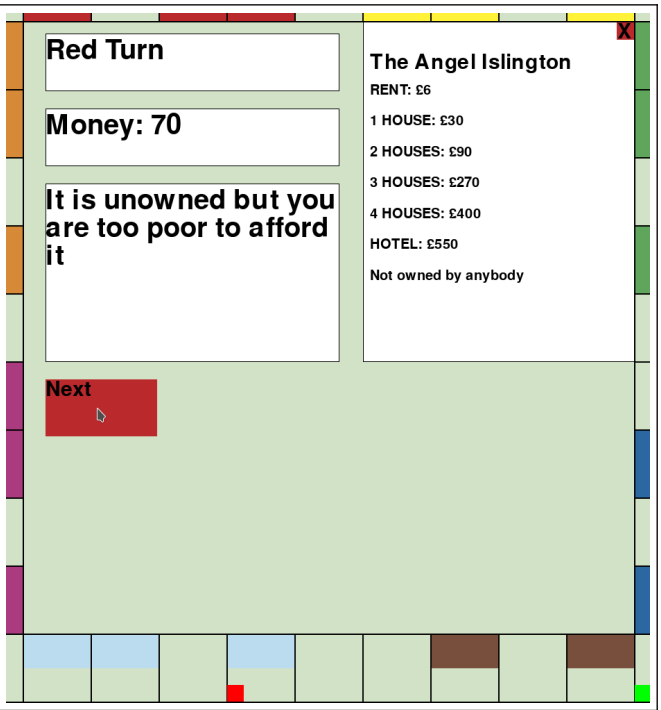
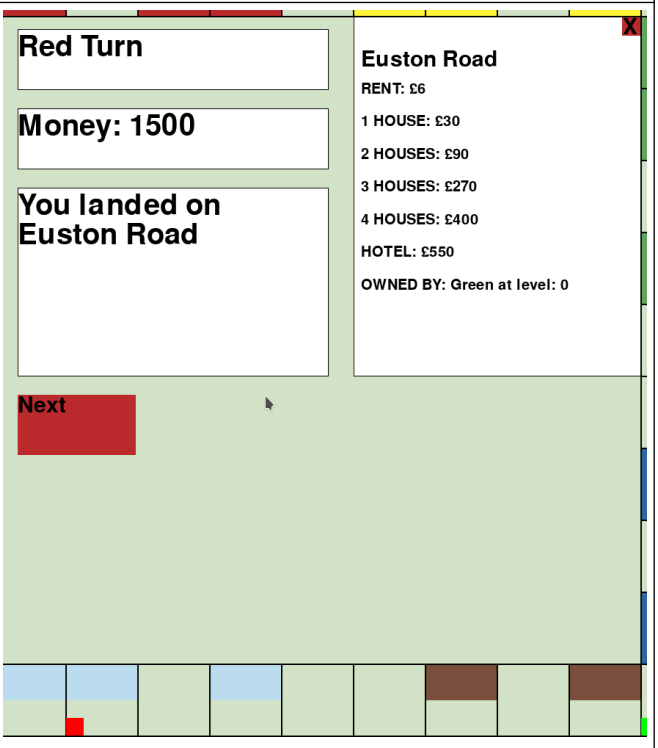
Test Number	Test Description	Pass?	Evidence Description	Evidence
101	When clicking the first button in the main loop the function should return True	Yes	The code outputted True as shown in the terminal	

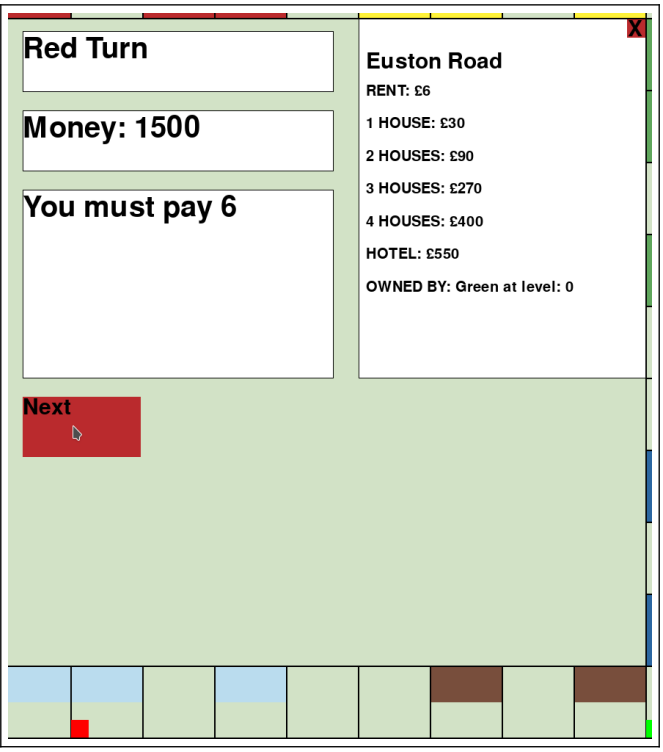
102	When clicking the second button in the main loop the function should return False	Yes	The code outputted as True as shown in the terminal	
103	When window is attempted to be closed, pygame stops running	Yes	Window closes when x button is clicked. However, the game runs in fullscreen so it requires use of keyboard shortcuts to close which may be unsuitable for less advanced users	

104	Clicking on all the different tiles on the screen should display a box with information about the tile	No	All work apart from the community chests and chance cards as they haven't been programmed yet	

				
105	When the player clicks on a property that they own but are not eligible to place houses on then they should not have the option to do so	Yes	It is Blue's turn and they own two of the properties in the set but they are unable to purchase any houses as they do not owned the final	
106	When the player clicks on a property that they own and are eligible to purchasing houses on it then they should have the option to do so	Yes	They have the option to purchase houses as they own the whole set.	

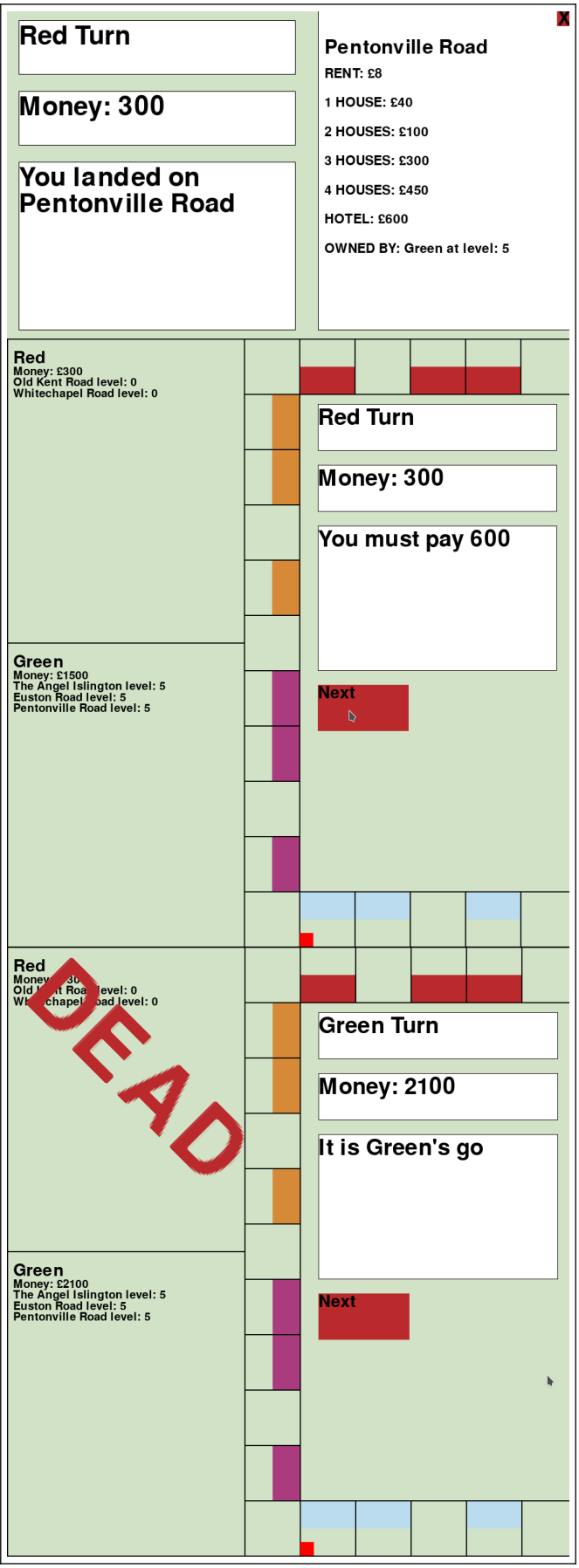
107	When the a player clicks the button to purchase a house, the property should gain an extra level for the extra house	Yes	The property is now at level 1 after the property was upgraded once	
108	The player should not be able to upgrade a set more than 5 times	No	There is no limit to the number of houses so I was able to purchase 7	
109	The player should be able to buy a property when it is unowned and they can afford to. When they purchase it the money should be taken from their wealth.	Yes	The Property ‘Euston Road’ was purchased by ‘Red’ for £100 when they had £1500. After the transaction ‘Red’ had £1400	

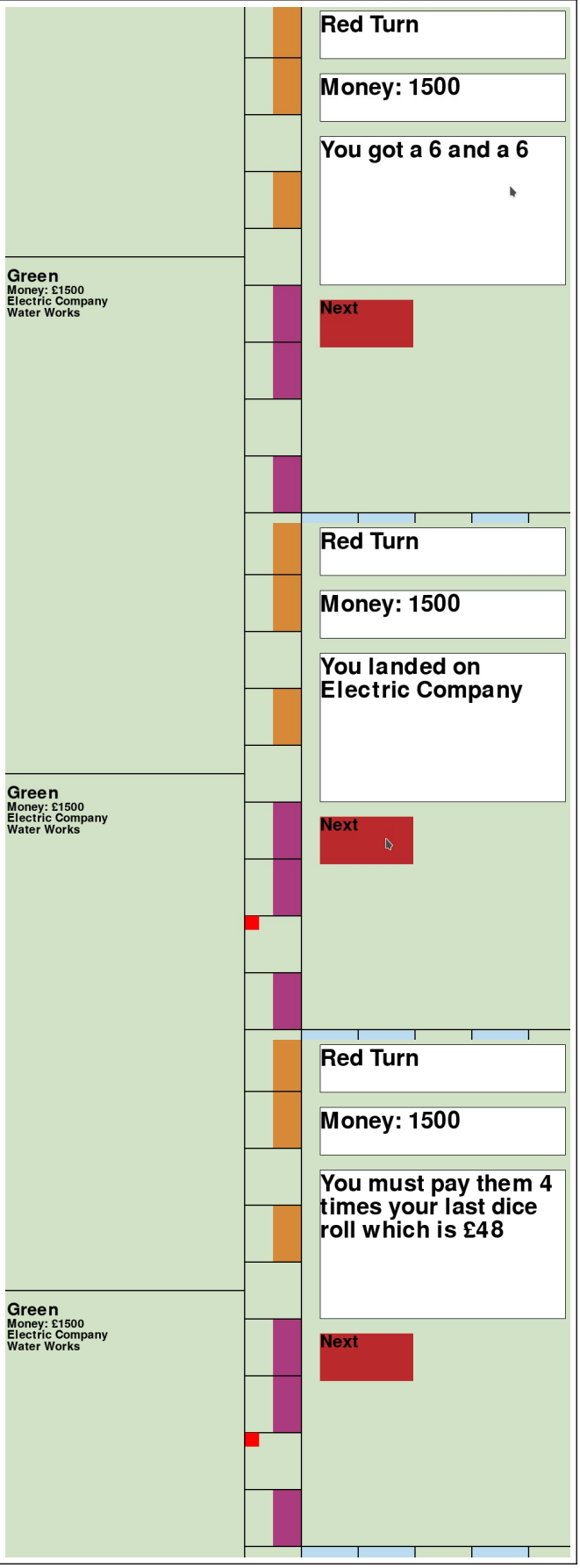
110	The player should be prevented from purchasing an unowned property if they cant afford it	Yes	The property costs £100 (though that info isn't displayed in the game yet) and the player has 70 so they are prevented from making the purchase	
111	When a player lands on a property owned by somebody else they should pay them the exact amount required	No	When 'Red' lands on the property they pay 'Green' £6 for rent which would normally be accurate but since 'Green' owns the whole set the rent should be doubled	

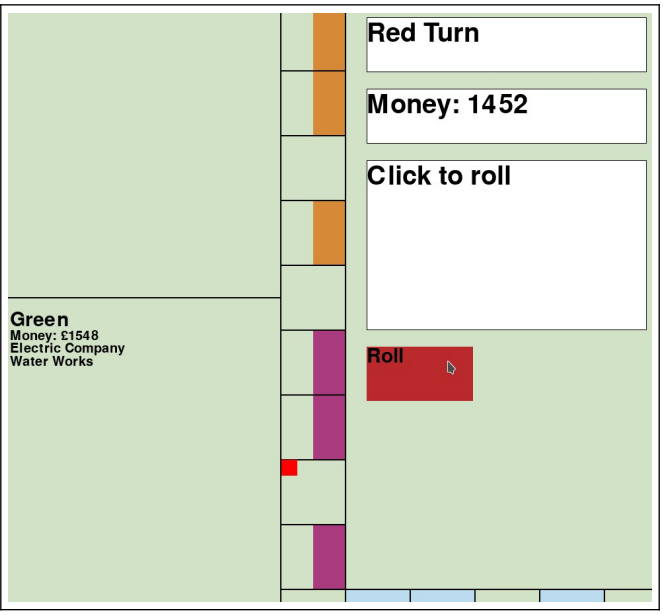
				

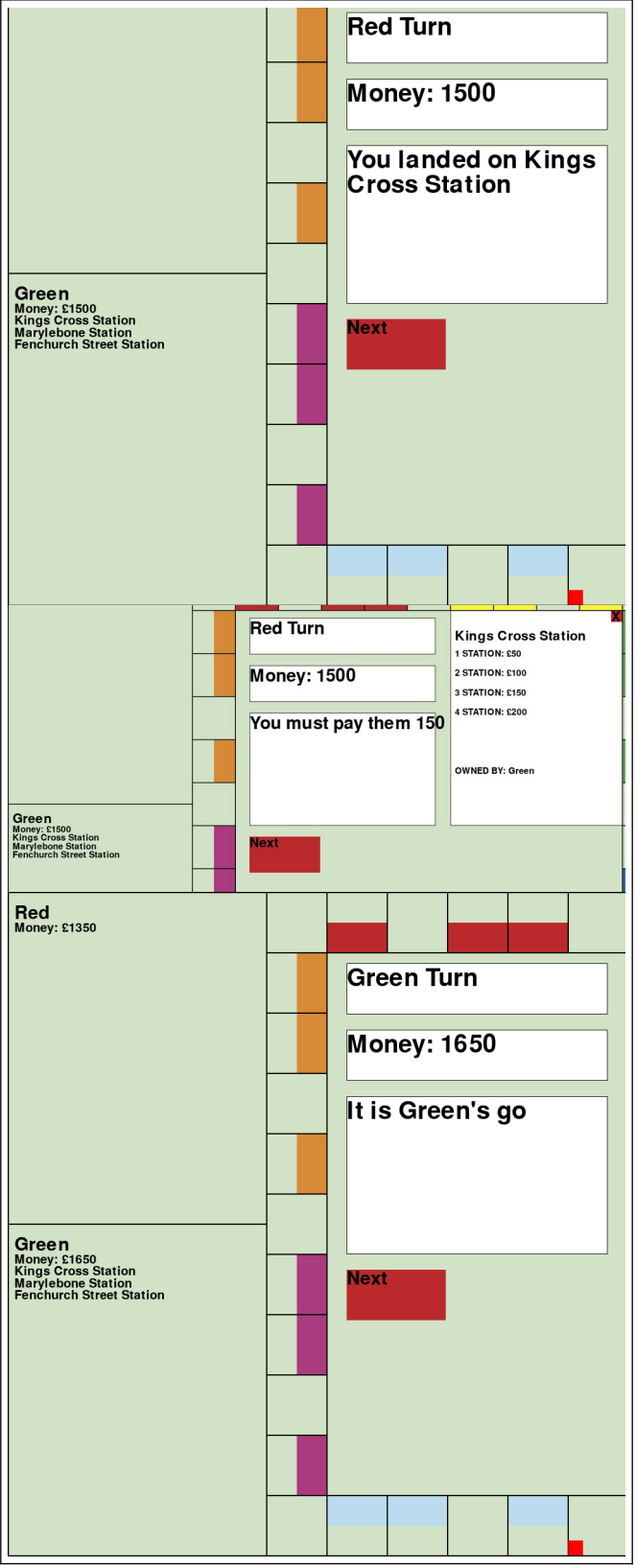
Red Turn
Money: 1500
You must pay 6
Next
X
Euston Road
RENT: £6
1 HOUSE: £30
2 HOUSES: £90
3 HOUSES: £270
4 HOUSES: £400
HOTEL: £550
OWNED BY: Green at level: 0

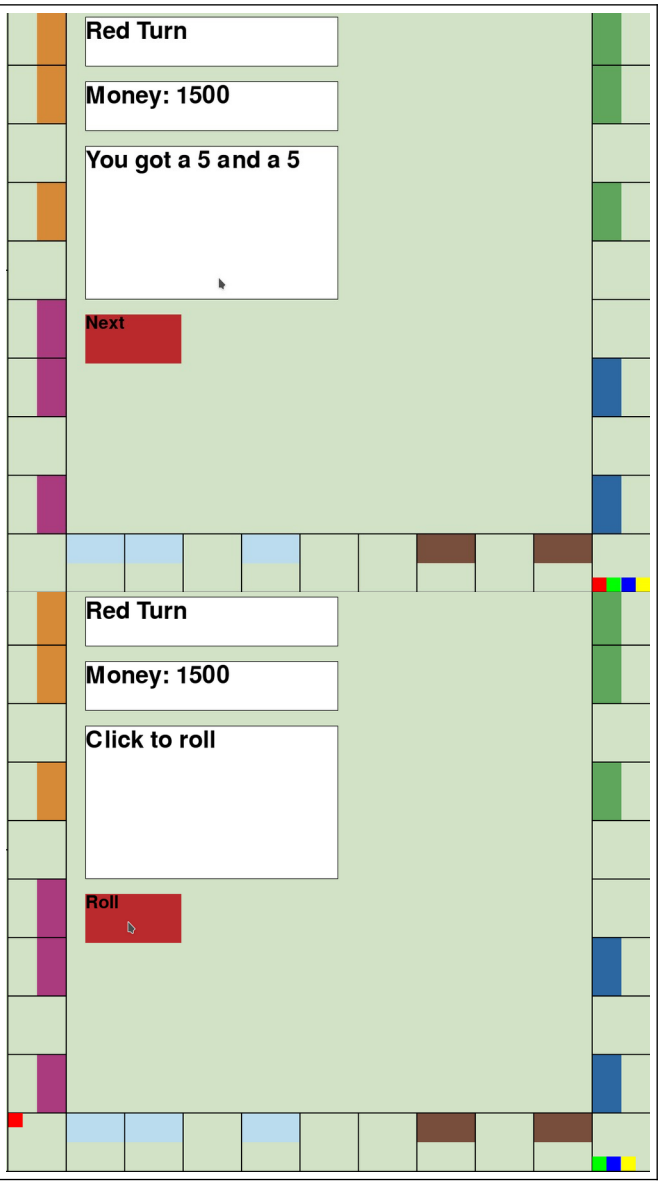
				Red **Money: £1494** **Old Kent Road level: 0** **Whitechapel Road level: 0** **Green** **Money: £1506** **The Angel Islington level: 0** **Euston Road level: 0** **Pentonville Road level: 0**

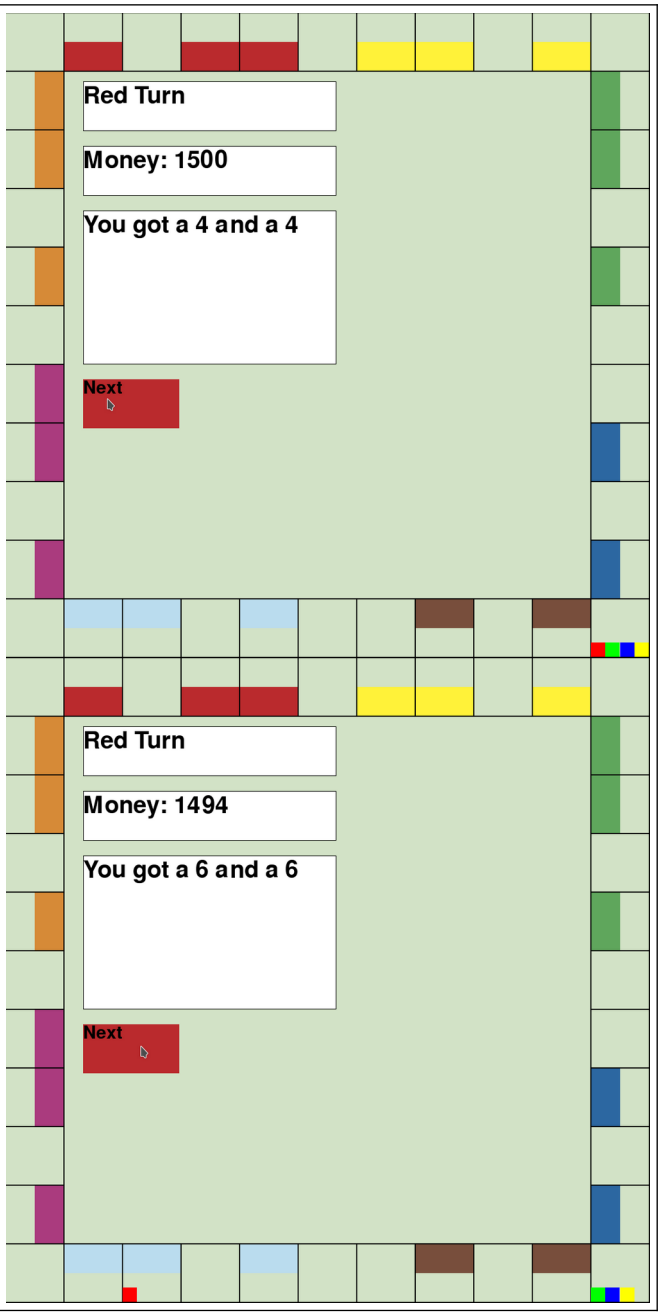
112	When a player lands on a property owned by someone else and they cant afford to pay the other person. The other person should get the remainder of their cash and the player should go bankrupt	No	'Red' needs to pay 'Green' £600 but they only have £300 so they go bankrupt. However, 'Green' gets paid the full £600 instead of the remaining £300	

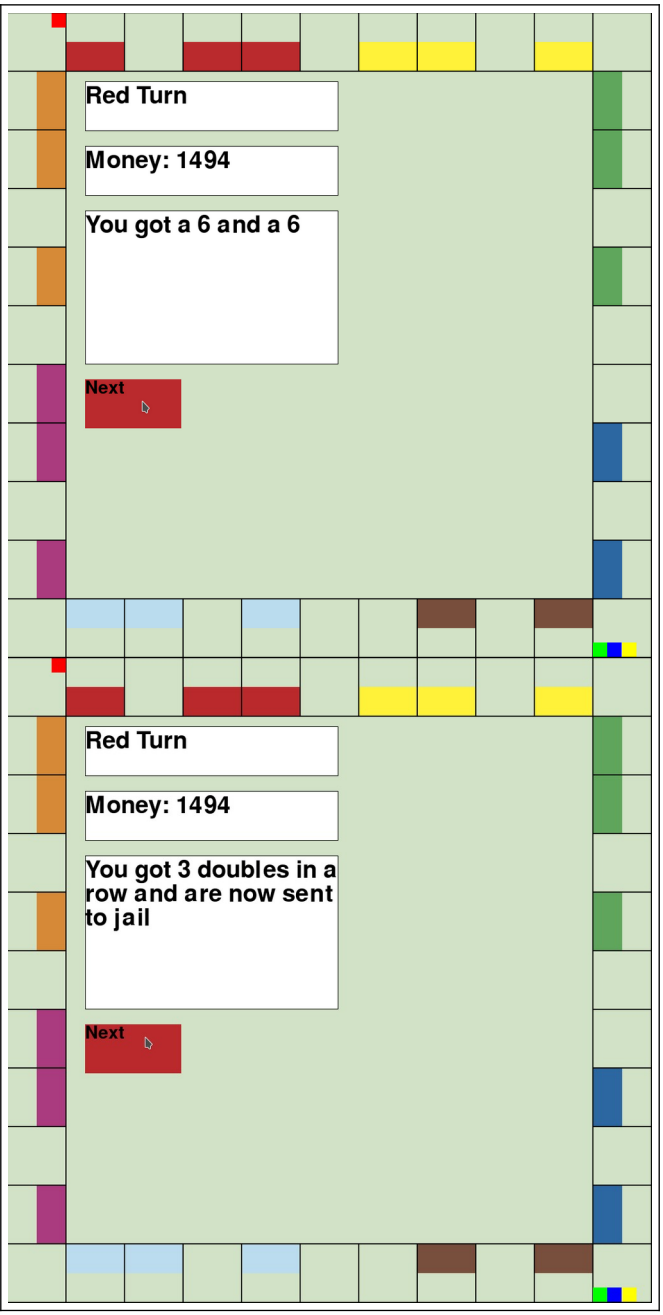
113	When the player lands on a utility where both are owned by the same person they should have to pay the owner 10 times their previous dice roll	No	After 'Red' lands on it they only have to pay 4 times their previous dice roll rather than 10 times given that 'Green' owns both. The 'num_in_set' value isn't updated correctly as the indexes for the utilities are not present in the 'sets' tuple.	

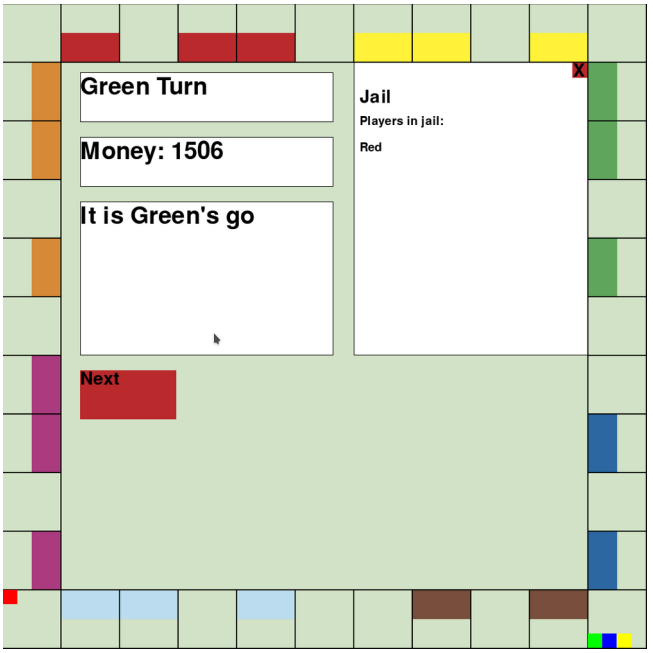
				 Red Turn Money: 1452 Click to roll Roll Green Money: £1548 Electric Company Water Works

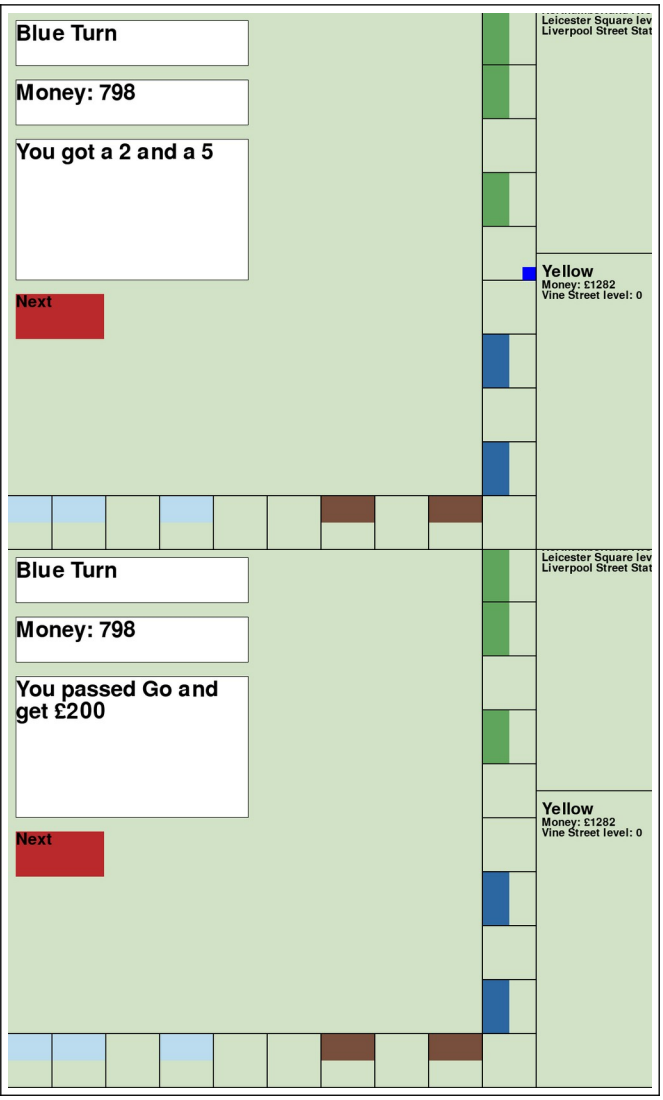
114	When the player lands on a station which is owned by another player who owns multiple stations they should have to pay them more depending on how many they own	Yes	‘Red’ pays ‘Green’ 150 as ‘Green’ owns 3 stations which corresponds to 150 to be payed	

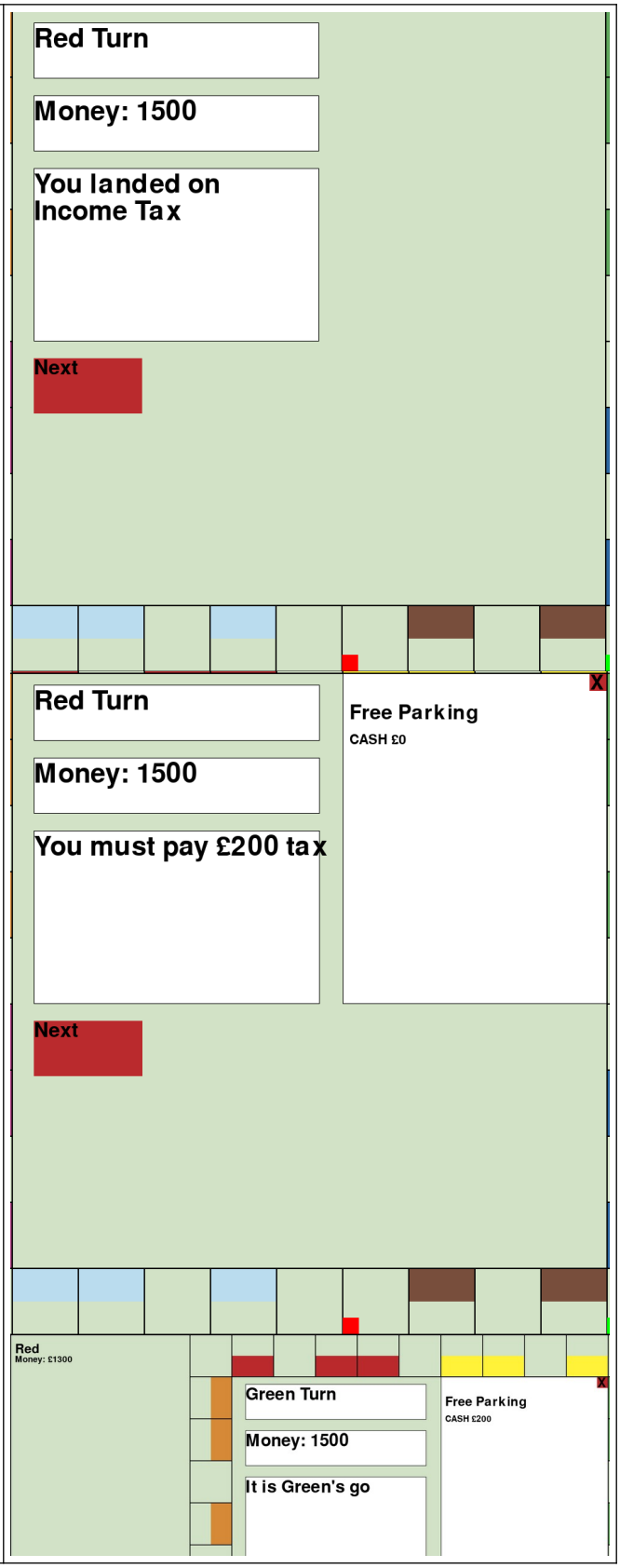
115	When the player rolls a double they should get an extra turn	Yes	After 'Red' rolls a double they get another roll before anyone else has moved	

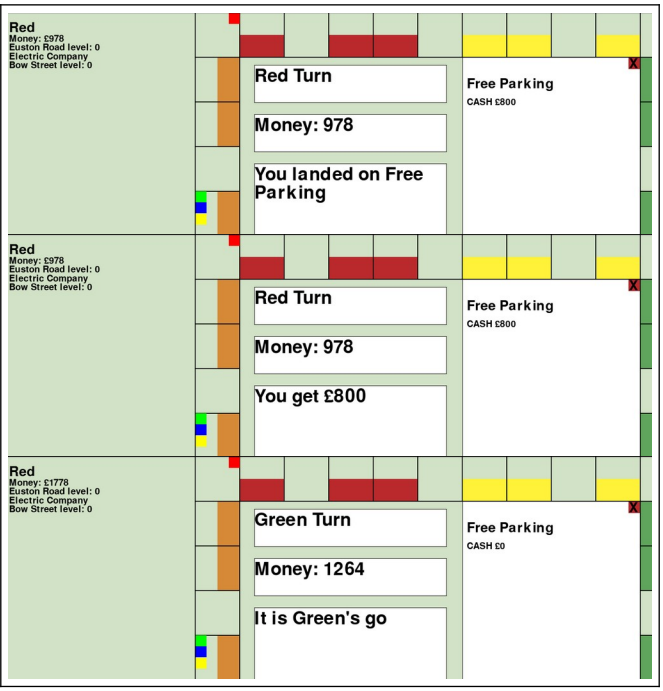
116	When the player rolls 3 doubles in a row they should go to jail	Yes	‘Red’ rolls a double three times in a row and on the third time they are sent to jail	

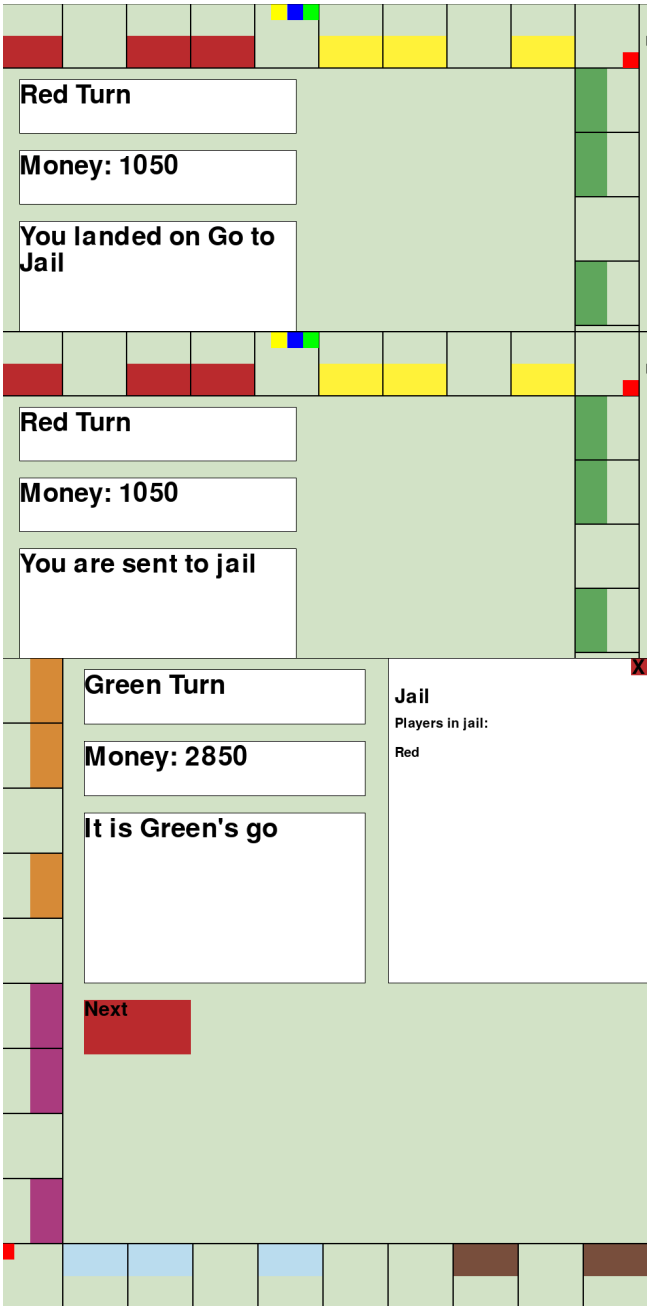
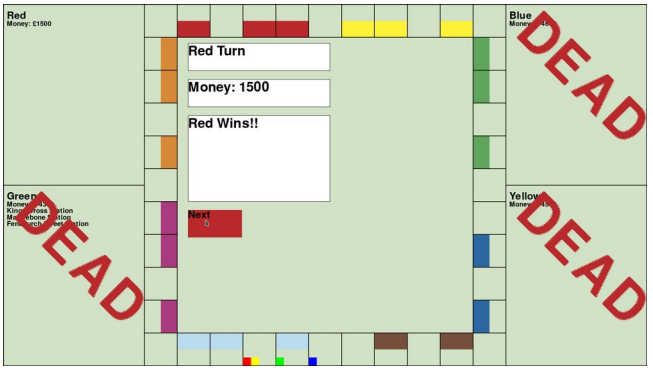
				 Red Turn Money: 1494 You got a 6 and a 6 Next Red Turn Money: 1494 You got 3 doubles in a row and are now sent to jail Next

				
117	When the player lands on a chance or community chest space they should receive a card with instructions on it which should be followed correctly by the program	No	Chance and community chest cards have not been implemented yet	

118	When the player moves past Go they should receive £200	Yes	‘Blue’ passes Go and receives £200. There is a bug however where the player icon disappears when they are shown that they pass Go	

119	When the player lands on a tax space they pay the money specified to the free parking space	Yes	‘Red’ lands on income tax and loses £200. The free parking cash increase by £200	

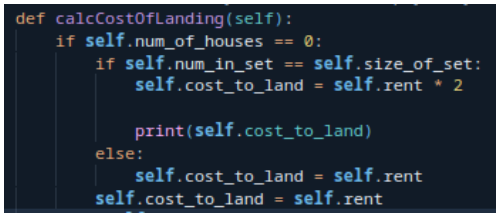
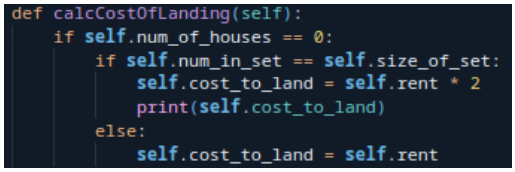
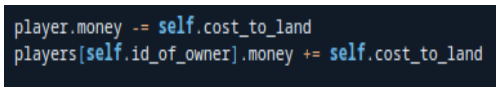
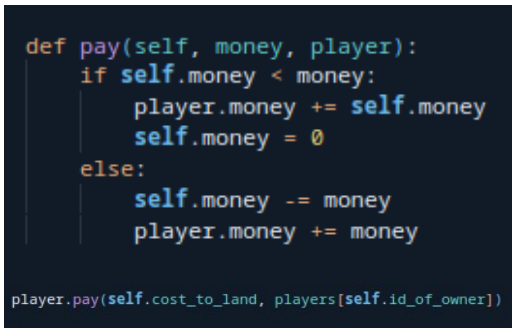
120	When the player lands on Free Parking they should get all of the money in the space and there should be £0 in free parking afterwards	Yes	‘Red’ lands on the space when there is £800 stored in it. ‘Reds money increases by £800 and the free parking cash resets to zero	

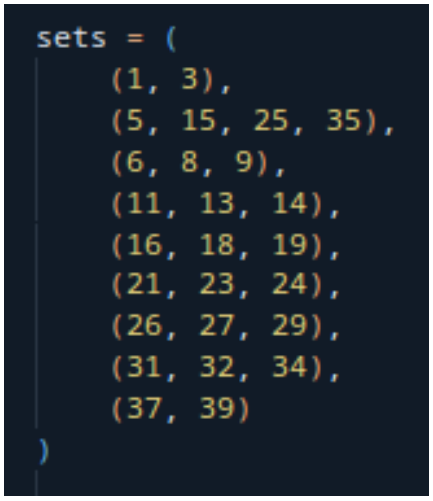
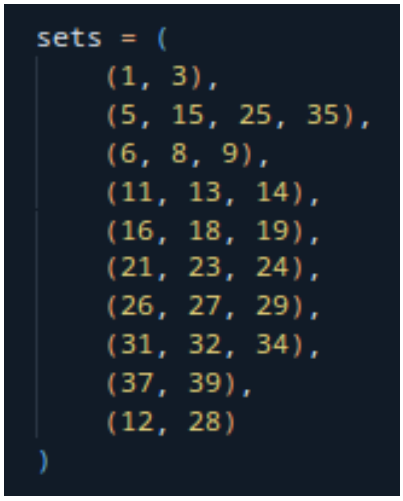
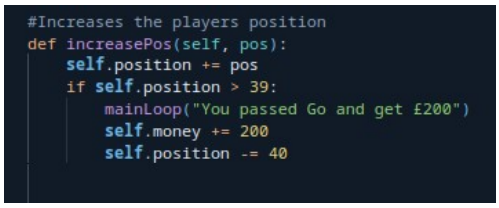
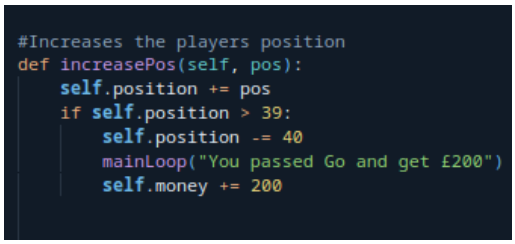
121	When the player lands on Go to Jail they should be immediately sent to jail	Yes	‘Red’ lands on Go to Jail and is sent to jail	
122	When all players but one go bankrupt the remaining player should be declared the winner and be congratulated	Yes	All other players have gone bankrupt so ‘Red’ is declared the winner	

Remedial Action:

Test Number	Original Code	Fixed Code	Explanation

108	`if self.num_in_set == self.size_of_set and self.id_of_owner == current_player:`
	
	`if board[blitted_card].num_in_set== board[blitted_card].size_of_set and board[blitted_card].id_of_owner == current_player:`
		`if self.num_in_set == self.size_of_set and self.id_of_owner == current_player and self.num_of_houses < 5:`
		
		`if board[blitted_card].num_in_set == board[blitted_card].size_of_set and board[blitted_card].id_of_owner == current_player and board[blitted_card].num_of_houses < 5:`
			I added an extra parameter for the if statement for the blitting of the button and the detection of the input to check if the number of houses of the property is less than 5

111			'self.cost_to_land' was accidentally set to being equal to rent regardless of the outcome of the if statement
112 (when bankrupted only pay remaining)			I made a new method of the player class called 'pay' which processes transactions and makes sure that you cant pay more than you have

113 (utilities don't work properly)			The sets tuple which provides all of the indexes for all of the sets didn't include the indexes for two utilities as I forgot to include it. This means that the property couldn't be recognised as being part of a complete set and thus the 10x dice roll cost of landing was unable to be achieved
118 (Icon dissappears when passing go)			The method detects when a player passes Go by checking if the players position has increased above 39, it then will then subtract 40 from it. When I called the main loop function, I called it before the position was reduced by 40 so the 'drawPos' function was blitting the player off of the screen as the position was too high. I fixed this by changing the order so the position is updated before

Review of First Iteration:

What has been done? I have completed much of the features that are required to play a full game of monopoly such as the basic graphics and the act of moving around the board and landing on spaces. All tiles, other than the chance and community chests, are fully functional and the game can end.

What went well? The inheritance and polymorphism associated with the all of the tiles worked well though some of the code is repeated a little. The drawing of the board also worked out well given that in its current state it is created entirely with code.

What went poorly? The code for the main loop could get a little messy especially when interacting with the blit card code.

What improvements to the code could be made?

- The fps number in the 'clock.tick(30)' function call should be changed to a constant to make it more easily readable and easier to change
- The create tiles function could be put outside of a function as it is unnecessary
- The 'size_of_set' attribute for properties can be gained from the 'sets' tuple instead of inputted into the function
- The rect I inputted into the 'drawText' function could be shrunken in some places as the text overflows from the box
- The 'showInfo' method shouldn't blit the information to the screen directly and instead it should return a surface with the information which the main loop could blit as the main loop should be the only place where the graphics changed
- The 'landOn' method for the 'Station' and the 'Sets' classes are effectively the same so the could be combined into one method in the superclass

What improvements to the usability could be made?

- The cost of a property should be shown in the 'showInfo' method of the property
- The status cards for each player should have the different owned properties in the list colour coded so it is easier to tell when you have got a full set
- The number of houses on a space should be shown on a the board

What success criteria were met?

- The game is playable apart from the chance and community chests
- There are additional UI elements to improve the user experience other than the dice roll animation

What success criteria were not met?

- There is no main menu
- There is are no computer players
- There is no way to save the game

Conclusion. Overall the first iteration was a success as there weren't too many large bugs and the game works quite well. The game did suffer some playability issues as it was hard to follow the game and the code could be improved a bit.

Second Iteration:

Break Down of Second Iteration:

There were many additions present in this iteration. Firstly I added computer players so the game doesn't need multiple people to be played. I also implemented chance and community chest cards. In addition I rewrote most of the graphics and input functions into a class and utilised an additional module to help with inputs. Animations now play when rolling the dice and moving the player.

```
import random
import pygame
from pygame.locals import *
import pygame_widgets
from pygame_widgets.button import Button
import math

#Initialises pygame library
pygame.init()
```

The first change made was to import the extra things I would need. For this iteration all of the buttons I made were replaced with buttons from the pygame_widgets module. This allowed the buttons to have more functionality and look better. In addition the module will be further utilised later on to allow for players to input text so they can write their names. I only use the button widget now so I only import that.

I also import the math module which is a built in python module that provides maths functions and constants. This is used later for the computer logic.

```
#The FPS of the game
FPS = 60

#Time it takes for the computer to progress text
COMPUTER_TIME = 60 # Default 60
#Time it takes per space to move in move animation
MOVE_TIME = 10 # Default 10
#Range of time for the dices to roll
ROLL_TIME_1 = 60 # Default 60
ROLL_TIME_2 = 90 # Default 90

#Size of dice images
IMAGE_SIZE = 200
```

Constants are useful as it allows important parameters in the program to be easily changed. For example if I want the computers to go faster I can easily change a single constant and I wont have to go through the whole code and find where it needs to be changed. The FPS value has been changed to a constant for this reason.

```
#Images:
#Images for each dice number
dice1 = pygame.transform.scale(pygame.image.load("Images/Dice1.png"), (IMAGE_SIZE, IMAGE_SIZE))
dice2 = pygame.transform.scale(pygame.image.load("Images/Dice2.png"), (IMAGE_SIZE, IMAGE_SIZE))
dice3 = pygame.transform.scale(pygame.image.load("Images/Dice3.png"), (IMAGE_SIZE, IMAGE_SIZE))
dice4 = pygame.transform.scale(pygame.image.load("Images/Dice4.png"), (IMAGE_SIZE, IMAGE_SIZE))
dice5 = pygame.transform.scale(pygame.image.load("Images/Dice5.png"), (IMAGE_SIZE, IMAGE_SIZE))
dice6 = pygame.transform.scale(pygame.image.load("Images/Dice6.png"), (IMAGE_SIZE, IMAGE_SIZE))

# Images for each space on the board that isnt a coloured set
free_parking_image = pygame.transform.flip(pygame.transform.scale(pygame.image.load("Images/free_parking.png"), (SQUARE_SIZE, SQUARE_SIZE)), True, True)
go_image = pygame.transform.scale(pygame.image.load("Images/go_3.png"), (SQUARE_SIZE, SQUARE_SIZE))
station_image = pygame.transform.scale(pygame.image.load("Images/train.png"), (SQUARE_SIZE, SQUARE_SIZE))
jail_image = pygame.transform.scale(pygame.image.load("Images/jail_2.png"), (SQUARE_SIZE, SQUARE_SIZE))
chance_image = pygame.transform.scale(pygame.image.load("Images/chance_2.png"), (SQUARE_SIZE, SQUARE_SIZE))
community_chest_image = pygame.transform.scale(pygame.image.load("Images/community_chest.png"), (SQUARE_SIZE, SQUARE_SIZE))
go_to_jail_image = pygame.transform.scale(pygame.image.load("Images/go_to_jail.png"), (SQUARE_SIZE, SQUARE_SIZE))
electric_company_image = pygame.transform.scale(pygame.image.load("Images/electric.png"), (SQUARE_SIZE, SQUARE_SIZE))
water_works_image = pygame.transform.scale(pygame.image.load("Images/water_works_2.png"), (SQUARE_SIZE, SQUARE_SIZE))
super_tax_image = pygame.transform.scale(pygame.image.load("Images/super_tax_2.png"), (SQUARE_SIZE, SQUARE_SIZE))
income_tax_image = pygame.transform.scale(pygame.image.load("Images/income_tax_2.png"), (SQUARE_SIZE, SQUARE_SIZE))

#Array of dice images
dices = (dice1, dice2, dice3, dice4, dice5, dice6)
```

The board display now has many image icons for each space so they must all be loaded here. I also need to scale most of them as otherwise pygame will just keep them into their original size which is usually too big. I also define all of the dice images here and put them into an array to make it easier to access them.

```
#Class for players
class Player():
    def __init__(self, given_name, given_id):
        #What space they are on
        self.position = 0
        #What name they were given
        self.name = given_name
        #How much money they have
        self.money = 1500
        #Identifies which player they are outside of the name and matches the index of the players array
        self.id = given_id
        #If they are bankrupt and out of the game
        self.dead = False
        #What their last rolls were
        self.roll1 = None
        self.roll2 = None
        #The number of turns they have left in jail
        self.jailed = 0
        #If they just got a double
        self.double = False
        #How many doubles they got in a row
        self.double_count = 0
        #Chooses colour
        if self.id == 0:
            self.colour = P_RED
        elif self.id == 1:
            self.colour = P_GREEN
        elif self.id == 2:
            self.colour = P_BLUE
        elif self.id == 3:
            self.colour = P_YELLOW
        #Keeps track of if they have passed go this game
        self.passed_go = False
```

There is now a new attribute for the player which keeps track of whether the player has passed go. This is due to a rule implemented where players cannot purchase houses until they have passed go once.

```
    def takeTurn(self):
        #Ends turn immediatly if player is out of the game
        if self.dead:
            return
        self.double_count = 0
        gc.mainLoop(f"It is {self.name}'s go")
        #Checks if player is in jail and gives them option to pay to get out, if not the jail counter goes down by one
        if self.jailed > 0:
            if gc.mainLoop(f"You are in jail with {self.jailed} turns left, you can pay 50 to get out", "Pay", input2="Dont Pay"):
                self.jailed = 0
                self.money -= 50
                board[20].free_parking_cash += 50
            else:
                self.jailed -= 1
                return
        #Repeats turn while player has a double
        self.double = True
        while self.double:
            self.double = False
            #Gives input for player to roll dice
            gc.mainLoop("Click to roll", "Roll")
            self.rollDice()
            #If the player gets 3 doubles in a row then they are sent to jail and the turn ends
            if self.double_count == 3:
                gc.mainLoop("You got 3 doubles in a row and are now sent to jail")
                self.jail()
                return
            #Function is used to increase the position as it provides animation
            self.increasePos(self.roll1 + self.roll2)
            #Checks if player is bankrupt
            if self.money < 0:
                self.bankrupt()
                return
        gc.mainLoop("Your turn has ended")
```

Made some small changes to the 'takeTurn' method. The dice roll confirmation where the game would show you what rolls you got has been moved to the 'rollDice' method as it would always be shown with it anyway.

An error with the jail code was fixed. I added a return statement if the player decides not to pay 50 to get out of jail as otherwise they would be able to continue with their turn when they shouldn't be able to.

The land on code has been moved to the 'increasePos' method. There is also a confirmation to show that a turn has ended when it ends. This is because before, after players completed a transaction their go would immediately and they would not be able to view the changes to their wealth easily or purchase houses.

In addition all of the 'mainLoop()' function calls have been replaced with 'gc.mainLoop()'. This is because the graphics components of the program have been bundled into a class with to object being called 'gc' for graphics class.

```
#Creates 2 random dice rolls
def rollDice(self):
    #Calls roll dice animation through main loop and when the dices are rolled the values are given to the player dice attributes
    self.roll1, self.roll2 = gc.mainLoop("Rolling Dice...", None, diceRoll=True)
    gc.mainLoop(f"You got a {self.roll1} and a {self.roll2}", "Next", diceRoll=False)

    if self.roll1 == self.roll2:
        self.double_count += 1
        self.double = True
```

Instead of creating the random dice rolls itself the 'rollDice' method now calls the main loop method with the parameter of 'diceRoll' set to true which means a dice rolling animation will play and the dice values will be returned. The second main loop function call sets 'diceRoll' to false which means that it wont roll the dice but simply display the previous roll so the player can view it.

```
#Increases the players position
def increasePos(self, pos):
    #Calculates where the plyaer will end up
    destination = self.position + pos
    if destination > 39:
        destination -= 40
    #Calls main loop to play an animation to move the player to its position
    gc.mainLoop(f"Move {pos} spaces", "Next", movePlayer = (destination, True if pos < 0 else False))
    gc.mainLoop(f"You landed on {board[self.position].name}")
    #Calls land on method for the space of the board where the player is
    board[self.position].landOn(self)
```

The 'increasePos' method has been changed so that it now just calculates the destination and adds that as a parameter to the main loop method so it can provide the animation and move the player. It also supports negative values where the player needs to move backwards. If this happens the second index of the tuple argument to the main loop will be true and the player will move backwards to the destination. The land on code is also moved here as players will always want to land on the space after moving. This means that if a chance card caused a player to move backwards then they would land on the space as the chance card code would call the 'increasePos' code. It also doesn't give extra money for players passing go here as that now happens in the main loop.

```
#Sets the players position
def setPos(self, pos):
    #Gives player money from passing go if they have to go past it to reach the position
    passed_go = False
    if pos < self.position:
        passed_go = True
        self.passed_go = True
    self.position = pos
    if passed_go:
        gc.mainLoop("You passed Go and get £200")
        self.money += 200
```

Often the players position will need to be set instead of them moving, often due to chance cards, so this method just immediately moves the player. The go money is given in this method as it doesn't use the main loop to move the player.

```
#Is called when player goes bankrupt, resets all properties owned by them
def bankrupt(self):
    gc.mainLoop("You are bankrupt")
    self.dead = True
    for i in sets:
        for j in i:
            if board[j].id_of_owner == self.id:
                board[j].reset()
```

When a play goes bankrupt a loop is used to cycle through every property owned by them and call the reset method on it which removes the owner making it free to purchase and removes all houses.

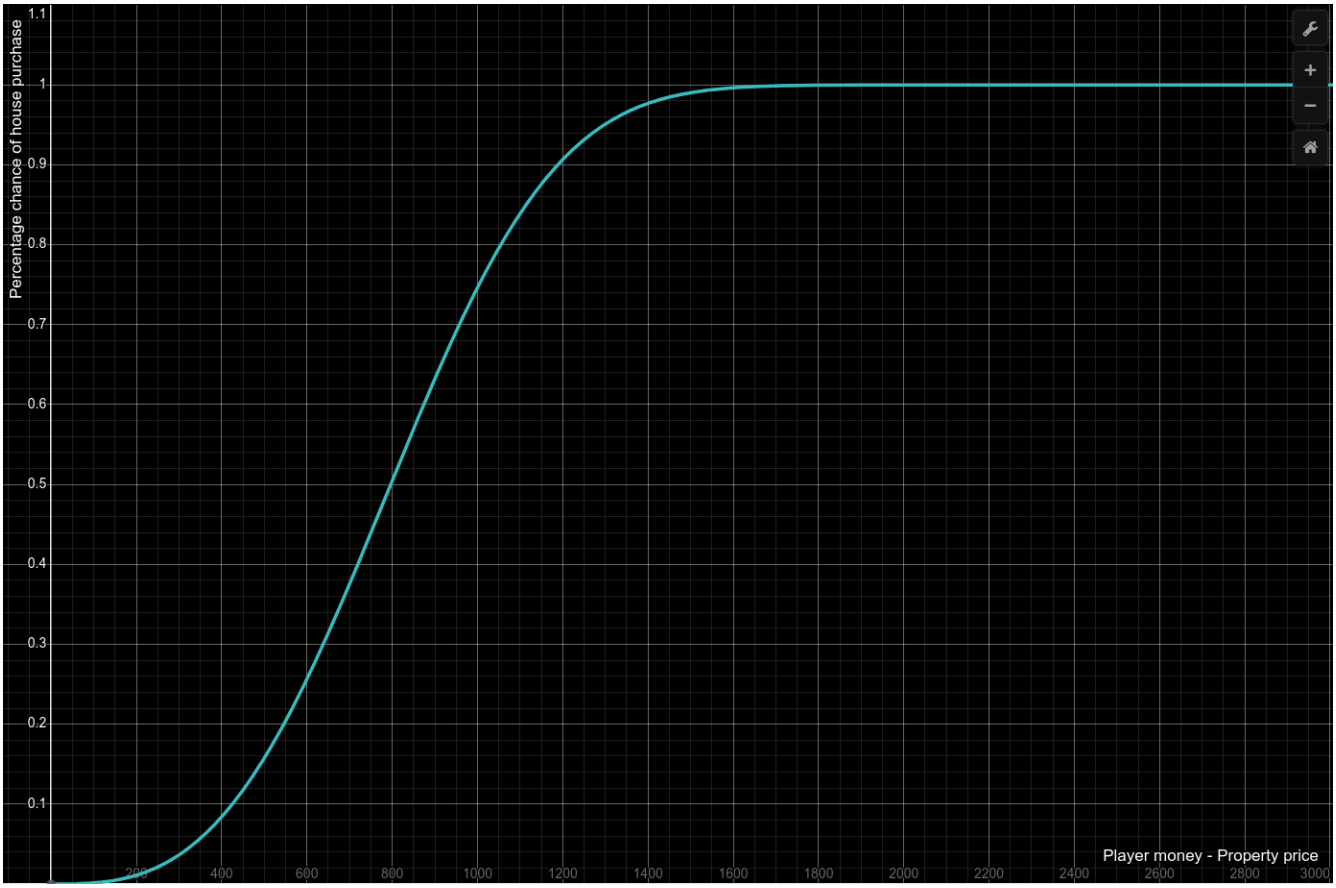
```
#Class for computer players
class Computer(Player):
    #Initialises the computer with the name being computer and the id of it
    def __init__(self, given_id):
        super().__init__(f"Computer {given_id + 1}", given_id)
```

The computer class is the class for all of the computer players. It inherits from the player class and reuses most of its methods such as the 'takeTurn' method. When it is created the name given is the word 'Computer' and one added to the id number afterwards.

```
#Uses mathematical function to decide the probablity of the computer buying the property
def propertyDecision(self, position):
    chance = (1 - math.e**-((self.money - board[position].price)/900)**3)
    #Returns whether the probability gave true or not
    return random.random() < chance
```

This method uses a mathematical function to provide a percentage chance of buying the property. It uses a graph with the x axis being the price of the house subtracted from the players wealth and the y axis being the decimal probability of the property being purchase. 'random.random()' is used to generate a random decimal from 0 to 1 which can be used to generate an outcome from the probability which is then returned.

The graph used:



The graph flattens out further one where the choice to purchase the property is obviously the correct one.

```
#Goes through all owned sets and uses a mathematical function to decide whether to buy a house on it
def buyHouses(self):
    sets2 = list(sets)
    random.shuffle(sets2)
    for i in sets2:
        houses_bought = True
        while houses_bought:
            for j in i:
                houses_bought = False
                if isinstance(board[j], Sets):
```

```
if isinstance(board[j], Sets):
    if board[j].id_of_owner == self.id and board[j].can_buy_houses and self.passed_go:
        x = (self.money - (2 * board[j].house_price)) * (((board[j].house_values[board[j].num_of_houses + 1] - board[j].house_values[board[j].num_of_houses])/1000) + 0.5) + random.randint(-300, 100)
        chance = (1 - math.e**-((x)/900)**3)
        if random.random() < chance:
            board[j].upgrade()
            houses_bought = True
```

The buy house function is called at the end of every main loop for computer players. It gives them the opportunity to purchases houses on their property. It works by looping through each set tuple infinitely until the computer decides not to buy any houses for a whole loop which means it then goes onto the next one.

This algorithm uses the same curve as the previous but the x parameter is now a value created from multiple different values. The values are: the wealth of the player, the price of the house, the increase in cost of landing for the property if it is purchased and a random value. After this is all calculated and the probability is chosen the 'random.random()' function is called again to decide whether the house should be upgraded.

```
#Base super class that all tiles will inherit from
class Tiles():
    def __init__(self, given_name, given_pos):
        self.name = given_name
        self.position = given_pos

    def landOn(self, player):
        pass

    #Creates a surface with info about the tile and returns it
    def showInfo(self):
        surface = pygame.surface.Surface((CARD_WIDTH, CARD_HEIGHT))
        surface.fill((255, 255, 255))
        pygame.draw.rect(surface, BLACK, (0, 0, CARD_WIDTH, CARD_HEIGHT), LINEWIDTH)
        surface.blit(FONT30.render(self.name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11))
        return surface
```

The 'showInfo' method for the base tile class has been changed so that it now creates the surface with all of the common elements of the tile card and then returns it to the subclass. Previously the superclass would finish off the elements of the surface but it made more sense to do it this way. In addition, instead of the method blitting the surface itself, the surface is now returned from all of the subclass 'showInfo' methods and it is blitted in the graphics class. This ensures that all graphical operations are contained within the single class which makes the code more modular.

The jail class now returns the surface from the 'showInfo' method instead.

```python
#Class for jail tile
class Jail(Tiles):
    #Shows who is currently in jail
    def showInfo(self):
        surface = super().showInfo()
        offset = 0
        surface.blit(FONT20.render("Players in jail:", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * 2))
        for i in range(len(players)):
            if players[i].jailed > 0:
                surface.blit(FONT20.render(players[i].name, ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * (3 + offset)))
                offset += 1
        return surface
```

```python
#Class for chance cards and community chest
class ChanceChests(Tiles):
    def __init__(self, name, position):
        super().__init__(name, position)
        #List of possible cards for chance and community chest
        self.chances = ["AdvToGo", "AdvToTrafalgar", "AdvToPall", "AdvNearUtil", "AdvNearRail", "Get50", "GoBack3", "GoToJail", "GeneralRepairs", "AdvToKingsCross", "AdvToMayfair", "PayEachPlayer50", "Get150"]
        self.com_chests = ["AdvToGo", "Get200", "Pay50", "Get50_2", "GoToJail", "Get100", "Get20", "Get10FromEach", "Get100_2", "Pay50_2", "Pay50_3", "Get25", "GeneralRepairs_2", "Get10", "Get100_3"]
```

The ‘chanceChests’ class is a subclass of the ‘Tiles’ class that is used to provide functionality to the chance and community chest spaces. When a player lands on them they will be given a random instruction usually involving moving or money transactions. Chance cards and community chests have separate decksof cards to pull from with instruction but they have a lot of similar cards and card types so they are in the same class. When the class is instantiated, two arrays full of keywords for each card are created.

```python
def landOn(self, player):
    #Switch case statement for random card to decide what happens
    match random.choice(self.chances if self.name == "Chance" else self.com_chests):
        #Chance Cards
        case "AdvToGo":
            self.advanceTo(board[0].name, 0, player)
        case "AdvToTrafalgar":
            self.advanceTo(board[24].name, 24, player)
        case "AdvToPall":
            self.advanceTo(board[11].name, 11, player)
        case "AdvNearUtil":
            self.advanceTo("Nearest Utility",self.findNearest(player.position, Utilities) , player)
        case "AdvNearRail":
            self.advanceTo("Nearest Railway Station",self.findNearest(player.position, Station) , player)
        case "Get50":
            self.getMoney("Bank pays you a dividend of £50", 50, player)
        case "GoBack3":
            player.increasePos(-3)
        case "GoToJail":
            gc.mainLoop("Go to Jail. Do not pass Go, do not collect £200")
            player.jail()
        case "GeneralRepairs":
            self.propertyRepairs(f"Make general repairs on all of your property. For each house pay £25, for each hotel pay £100", 25, 100, player)
        case "AdvToKingsCross":
            self.advanceTo(board[5].name, 5, player)
        case "AdvToMayfair":
            self.advanceTo(board[39].name, 39, player)
        case "PayEachPlayer50":
            self.moneyFromEach("You have been elected chairman of the board. Pay each player £50", -50, player)
        case "Get150":
            self.getMoney("Your loan matures. Recieve £150", 150 , player)

        #Community Chest cards
```

```
        #Community Chest cards
        case "Get200":
            self.getMoney("Bank Error in your favour. Collect £200", 200, player)
        case "Pay50":
            self.getMoney("Doctors fees. Pay £50", -50, player)
        case "Get50_2":
            self.getMoney("From sales of stocks you get £50", 50, player)
        case "Get100":
            self.getMoney("Holiday fund matures. Collect £100", 100, player)
        case "Get20":
            self.getMoney("Income tax refund. Collect £20", 20, player)
        case "Get10FromEach":
            self.moneyFromEach("It is your birthday. Collect £10 from every player", 10, player)
        case "Get100_2":
            self.getMoney("Life insurance matures. Collect £100", 100, player)
        case "Pay50_2":
            self.getMoney("Hospital Fees. Pay £50", -50, player)
        case "Pay50_3":
            self.getMoney("School fees. Pay £50", -50, player)
        case "Get25":
            self.getMoney("Recieve for services £25", 25, player)
        case "GeneralRepairs_2":
            self.propertyRepairs(f"You are assessed for street repairs. Pay £40 per house and £115 per hotel you own", 40, 115, player)
        case "Get10":
            self.getMoney("You won second prize in a beauty contest. Collect £10", 10, player)
        case "Get100_3":
            self.getMoney("You inherit £100", 100, player)
```

A large switch case statement is used to choose the event that happens for each card. The ‘random.choice’ function is used to randomly choose a card from one of the lists depending on which one the player landed on. As most cards do similar things, most of the options call a few methods.

```
#Used for all cards that involve giving the player money
def getMoney(self, message, amount, player):
    gc.mainLoop(message)
    player.money += amount
    #If money is negative then the money will go to free parking
    if amount < 0:
        board[20].free_parking_cash -= amount
```

This method is used for all of the cards that involve giving the player money or taking the players money. The ‘message’ parameter is used to say the message on the card and the ‘amount’ parameter show how much money is added to the players wealth. If the ‘amount’ value is negative then the player will lose money and it will go into the free parking space.

```
#For all the cards which involve all getting money from all players or giving money to all players
def moneyFromEach(self, message, amount, player):
    gc.mainLoop(message)
    for i in players:
        if not i.dead and i != player:
            player.money += amount
            i.money -= amount
```

Some cards involve having the player take money from each player or give money to each player. This method provides this to avoid reusing code and it displays a message. It works for giving money and taking money by using negative and positive values for the ‘amount’ parameter.

```
#Used for all cards that make the player advance to a position
def advanceTo(self, message, pos, player):
    gc.mainLoop(f"Advance to {message}")
    player.setPos(pos)
    board[pos].landOn(player)
```

This method is used for all of the cards that make the player advance to a specific space. It uses the 'setPos' method for the player to change their position.

```
#Used for all cards that make the player advance to the nearest of a type
def findNearest(self, player_pos, type):
    for i in range(player_pos, player_pos + 39):
        if i > 39:
            i -= 40
        if isinstance(board[i], type):
            return i
```

Some cards make the player move to the next station or utility. This method will find the next of a given type of tile and return it's position. This position is then used with the previous method to advance to the space.

```
#Used for all the cards that make the player pay a certain amount per house and hotel they own
def propertyRepairs(self, message, house_cost, hotel_cost, player):
    gc.mainLoop(message)
    to_pay = 0
    for i in sets:
        for j in i:
            if board[j].id_of_owner == player.id and isinstance(board[j], Sets):
                to_pay += board[j].num_of_houses * house_cost if board[j].num_in_set < 5 else hotel_cost
    gc.mainLoop(f"You must pay {to_pay}" if to_pay > 0 else "You own no houses or hotels so you dont need to pay")
    player.money -= to_pay
    board[20].free_parking_cash += to_pay
```

This method makes the player pay a certain amount of money for every house and hotel they own. It works like most of the other methods with a message and values to input as different cards make you pay different amounts.

```
def showInfo(self):
    surface = super().showInfo()
    drawText(surface, f"When you land here you will get special card with unique instructions", BLACK, (10, CARD_HEIGHT / 11 * 2, CARD_WIDTH - 20, CARD_HEIGHT / 11 * 9), FONT20, ANTI_ALIASING)
    return surface
```

The 'showInfo' message for the 'chanceChests' class simply explains how they work

```
#Class all buyable tiles inherit from
class Properties(Tiles):
    def __init__(self, given_name, given_pos, given_price, given_colour):
        super().__init__(given_name, given_pos)
        self.price = given_price
        self.id_of_owner = None
        self.num_in_set = 1
        self.cost_to_land = 0
        self.colour = given_colour

    def calcCostOfLanding(self):
        pass

    #Unowns the property and resets all the attributes
    def reset(self):
        self.id_of_owner = None
        self.num_in_set = 1

    def getOwned(self, player):
        self.id_of_owner = player.id
        #Updates the num in sets and house eligablity
        checkForSets()
        checkHouseEligability()
```

The ‘Properties’ class now has a colour value. This is the colour that each property has which is used to colour code the properties owned by the player in the player status card.

The ‘getOwned’ method now calls a method called ‘checkHouseEligability’ which checks if every coloured set property to see if it can have houses built on it.

```
#Performs common task of purchasing properties if unowned
def landOn(self, player):
    if self.id_of_owner == None:
        if self.price <= player.money:

            if isinstance(player, Computer):
                output = player.propertyDecision(self.position)
            else:
                output = None
            if gc.mainLoop(f"Do you want to buy it for £{self.price}?", "Yes", input2="No", output = output):
                self.getOwned(player)
                player.money -= self.price
                gc.mainLoop(f"You purchased {self.name}")
            else:
                gc.mainLoop(f"You didn't purchase {self.name}")
        else:
            gc.mainLoop("It is unowned but you are too poor to afford it")
    else:
```

Originally each type of property reused the same code when it came to purchasing the property if it was unowned. Now the 'Properties' superclass handles all of this. First it checks if it is unowned, if so it will allow the player to purchase the property which is the same process for coloured sets, stations and utilities. If it is owned then it will return true and the subclass method will carry out the more specific instructions when it comes to paying rent on the property. Also, if the player is a computer player then the output of whether they want to purchase the house is pre-calculated and fed into the main loop which will output it.

```
#Class for all standard coloured sets
class Sets(Properties):
    def __init__(self, given_name, given_pos, given_price, given_rent, given_house1, given_house2, given_house3, given_house4, given_hotel, given_colour):
        super().__init__(given_name, given_pos, given_price, given_colour)
        #Values for house prices are put in an array
        self.house_values = [None] * 6
        self.house_values[0] = given_rent
        self.house_values[1] = given_house1
        self.house_values[2] = given_house2
        self.house_values[3] = given_house3
        self.house_values[4] = given_house4
        self.house_values[5] = given_hotel

        self.num_of_houses = 0
        if self.position < 10:
            self.house_price = 50
        elif self.position < 19:
            self.house_price = 100
        elif self.position < 29:
            self.house_price = 150
        else:
            self.house_price = 200

        #Works out the size of the set
        for i in sets:
            for j in i:
                if self.position == j:
                    self.size_of_set = len(i)
        #Variable that shows if houses can be bought on the space
        self.can_buy_houses = False
```

Originally in this class, the values for each number of houses were stored as separate variables. It has now been changes to an array which makes it easier to access and order. Instead of the 'size_of_set' attribute being inputted manually, it is now inferred by checking the 'sets' tuple and looking at the length of the tuple containing the property. There is also a value that shows if houses can be bought on a property. This avoids having to recalculate this every time it was needed.

```
#Method for when player lands on the space
def landOn(self, player):
    #First checks if the player can buy it then goes through set specific actions
    if super().landOn(player):
        if self.id_of_owner == player.id:
            gc.mainLoop("You own this property")
        else:
            self.calcCostOfLanding()
            gc.mainLoop(f"It is owned by {players[self.id_of_owner].name}")
            gc.mainLoop(f"You must pay them {self.cost_to_land}")
            player.pay(self.cost_to_land, players[self.id_of_owner])
```

Due to the superclass doing some of the code, this method is now shorter. If the super method returns true then the property is owned and the player must pay the owner.

```
#Adds an extra house
def upgrade(self):
    self.num_of_houses += 1
    players[current_player].money -= self.house_price
    checkHouseEligability()
    #If a computer bought the house confirmation text is shown
    if isinstance(players[current_player], Computer):
        gc.mainLoop(f"You bought a house on {self.name}")
```

When the player adds a new house the 'checkHouseEligability' function is called. This is because houses are required to be spread out across each property is a set so buying a house could make you no longer able to buy houses on the space. When a player buys a house no confirmation text is shown because it is unnecessary but it is shown when a computer buys a house as otherwise you wouldn't be able to tell when a computer has bought a house.

```
#Calculates how much you will have to pay if you land on the space
def calcCostOfLanding(self):
    self.cost_to_land = self.house_values[self.num_of_houses]
    if self.num_of_houses == 0:
        if self.num_in_set == self.size_of_set:
            self.cost_to_land = self.house_values[0] * 2
```

This method has been simplified thanks to putting all of the house values in an array. Whereas before the cost of landing was calculated with a chain of if statements, now it just uses the number of houses attribute as an index for the array of house values. It also doubles the rent for zero houses if the whole set is owned.

```
#Shows price of property along with all house values and owner
def showInfo(self):
    surface = super().showInfo()
    surface.blit(FONT20.render(f"PRICE: £{self.price}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 2))
    surface.blit(FONT20.render(f"RENT: £{self.house_values[0]}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 3))
    surface.blit(FONT20.render(f"1 HOUSE: £{self.house_values[1]}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 4))
    surface.blit(FONT20.render(f"2 HOUSES: £{self.house_values[2]}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 5))
    surface.blit(FONT20.render(f"3 HOUSES: £{self.house_values[3]}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 6))
    surface.blit(FONT20.render(f"4 HOUSES: £{self.house_values[4]}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 7))
    surface.blit(FONT20.render(f"HOTEL: £{self.house_values[5]}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 8))
    surface.blit(FONT20.render(f"OWNED BY: {players[self.id_of_owner].name} at level: {self.num_of_houses}" if self.id_of_owner != None
    return surface
```

```
else "Not owned by anybody", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT /11 * 9))
```

The ‘showInfo’ method no longer displays the add house message as that is now added on in the graphics class. The price to buy the property is also shown for all of the properties now.

```
#Class to handle all pygame graphics processing
class Graphics():
    def __init__(self):
        #Keeps track of which card is blitted
        self.blitted_card = None
        #Variable to tell the main loop method to end and what to return
        self.out = None

        #Defining the pygame widget buttons so they dont have to be defined as global variables
        #adds 50 hue to colour for pressed colour and subtract 50 for the hover colour
        self.button1 = Button(screen, 551, 614, 161, 82, text = None, inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60), fontSize = 50,
        self.button2 = Button(screen, 814, 614, 161, 82, text = None, inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60), fontSize = 50,
        self.xButton = Button(screen, CARD_WIDTH - 25 - LINEWIDTH + (421 + (6 * SQUARE_SIZE)), LINEWIDTH + SQUARE_SIZE + 1, 25, 25, text = "X", inactiveColour=RED,
        self.upgradeButton = Button(screen, 425 + 200 + (6 * SQUARE_SIZE), 1 + SQUARE_SIZE + (CARD_HEIGHT / 11 * 10) - 5, 100, 30, text = "Yes", inactiveColour=RED,

        #Hides all buttons until they are needed
        self.button1.hide()
        self.button2.hide()
        self.xButton.hide()
        self.upgradeButton.hide()
```

```
 onClick = lambda: self.setOut(True))
 onClick = lambda: self.setOut(False))
hoverColour = (136, 31, 34), pressedColour = (236, 54, 60), fontSize = 50, onClick = lambda: self.stopCardBlit())
 hoverColour = (136, 31, 34), pressedColour = (236, 54, 60), fontSize = 40)
```

The graphics class is a class used to handle all graphical and user input based code. This was done in order to minimise the use of global variables that the previous way of doing it used.

The ‘blitted_card’ attribute is still used to keep track of which tile card is blitted but it is no longer a global variable. The ‘out’ attribute is used to tell the main loop when to stop and what to return.

The class also defines 4 buttons which come from the ‘pygame_widgets’ module. ‘Button1’ and ‘button2’ are used as basic outputs for the main loop which can change what they say. They aren’t given text immediately as that will be added during the loop as it can change. The ‘xButton’ is used to stop blitting the tile card just like it did before but it is now a pygame widget. The ‘upgradeButton’ is used by players to upgrade properties. There are many arguments that the buttons take. Most of them just involve the look of them and are self explanatory. The ‘onClick’ parameter takes a lambda statement and calls it when the button is

clicked. For this is used methods in the class to pass to the lambda statement. All of the buttons are hidden as soon as they are created and are then shown again when they are needed.

```
#Main game loop with different parameters to do different things
#Other args used to provide less used optional arguements that dont need to be a regular parameter
def mainLoop(self, text, input1="Next", **other_args):
    self.out = None
    #The input2 variable is gotten from an optional arguement
    input2 = other_args.get("input2", None)


    #A tuple with info about move player animation, the first element holds the destination, the second decides if it should go backwards or not
    movePlayer = other_args.get("movePlayer", None)
    #A variable which if true plays a dice roll animation and returns the result, if false it displays the previous roll, if none nothing happens
    diceRoll = other_args.get("diceRoll", None)
    #Variable which decides which output should be given by a computer player
    computerOutput = other_args.get("output", True)

    #shows how many frames have been displayed during the loop
    time = 0

    #Draws the board and blits the card if it needs to be blitted
    self.drawBoard(text)
    self.blitCard()

    #If input 1 exits then it is rendered and the button is shown
    if input1 != None:
        self.button1.text = FONT30.render(input1, ANTI_ALIASING, BLACK)
        self.button1.show()

    #If input 2 exits then it is rendered and the button is shown
    if input2 != None:
        self.button2.text = FONT30.render(input2, ANTI_ALIASING, BLACK)
        self.button2.show()

    #If a dice roll is happening, two random roll times are created and two roll variables are defined
    if diceRoll:
        roll_time1 = random.randint(ROLL_TIME_1, ROLL_TIME_2)
        roll_time2 = random.randint(ROLL_TIME_1, ROLL_TIME_2)
        roll1 = 0
        roll2 = 0
```

The main loop now uses kwargs (keyword arguments) to add parameters. This is because as more parameters were added, I thought it would be simpler to make most of them optional and hidden. The ‘time’ variable keeps track of how many frames have been displayed since the start of the method. It is used for animations that require timing. In order to save on CPU performance, the board is only blitted before the while loop starts. The ‘drawBoard’ function, which is now a method as a part of the graphics class, now draws in the the text that shows the current player, the players money, and the text inputted as a parameter.

If input 1 or 2 exists then their respective button will be shown to allow the player to click on it. If the dice roll parameter is true then two random times for each dice roll are chosen which are decided using the roll time constants. The two roll values are also created.

```
while True:
    #If move player exists then the players position is increased by 1 at even intervals and the board in re blitted
    #until they reach their destination where the loop is ended
    if movePlayer != None:
        #Time is independent of fps
        if time % (MOVE_TIME * FPS/30) == 0:
            players[current_player].position += (1 if not movePlayer[1] else -1)
            if players[current_player].position > 39:
                players[current_player].position -= 40
                players[current_player].money += 200
                players[current_player].passed_go = True
            if players[current_player].position == movePlayer[0]:
                return
            self.drawBoard(text)
            self.blitCard()

    #Event loop used to collect inputs
    events = pygame.event.get()
    for event in events:

        #If the user tries to close the window, pygame and python will stop
        if event.type == pygame.QUIT:
            pygame.quit()
            exit()

        #If the user clicks the mouse button the code will check if the mouse is hovered over a button
        if event.type == MOUSEBUTTONDOWN:
            #Checks each tile position if mouse is over it
            for i in range(len(pixels)):
                tile_rect = pygame.rect.Rect(pixels[i][0], pixels[i][1], SQUARE_SIZE, SQUARE_SIZE)
                if tile_rect.collidepoint((pygame.mouse.get_pos())):
                    #Shows info about the clicked on tile
                    self.blitted_card = i
                    self.drawBoard(text)
                    self.blitCard()
                    #If the position can be upgraded then the upgrade button is shown and the clicked on function is defined to upgrade the property
                    if self.canUpgrade():
                        self.upgradeButton.show()
                        self.upgradeButton.onClick = lambda: self.upgradeCard(self.blitted_card, text)
```

Within the while True loop, the code to create the move player animation exists. If the ‘movePlayer’ flag has a value and is not none then every given amount of time, the players position will increase by one until it reaches its destination. Each time the players position increases the ‘drawBoard’ method is called which redraws the players positions which gives the illusion of the player moving.

The code for when a tile is clicked on is the same as before but it now uses methods to blit the card instead of repeating code. If the tile is a property that can have a house on it, then the upgrade button is shown and the ‘onClick’ argument is set. I couldn’t have given the button an ‘onClick’ function when it was constructed as I needed to set the parameter for the method inside it.

```
#If dice roll is true a random dice roll will be generated
if diceRoll:
    #At even intervals the dices are updated with a random value
    if time % (3 * FPS/30) == 0:
        if time < roll_time1:
            roll1 = random.randint(1, 6)
        if time < roll_time2:
            roll2 = random.randint(1, 6)
        #The dice corresponding to the values are blitted
        screen.blit(dices[roll1 - 1], (551, 721))
        screen.blit(dices[roll2 - 1], (776, 721))
        #Boxes are drawn aroung the box to make it consistant
        pygame.draw.rect(screen, BLACK, (551, 721, 200, 200), LINEWIDTH)
        pygame.draw.rect(screen, BLACK, (776, 721, 200, 200), LINEWIDTH)
    #Once the time to roll is up for both dices the values are returned
    if time > roll_time1 and time > roll_time2:
        return (roll1, roll2)
#If dice roll is false the previous dice roll will be displayed
if diceRoll == False:
    screen.blit(dices[players[current_player].roll1 - 1], (551, 721))
    screen.blit(dices[players[current_player].roll2 - 1], (776, 721))
    pygame.draw.rect(screen, BLACK, (551, 721, 200, 200), LINEWIDTH)
    pygame.draw.rect(screen, BLACK, (776, 721, 200, 200), LINEWIDTH)
```

Below that is the code responsible for the dice roll. If the ‘diceRoll’ value is true, at regular time intervals a random dice number will be generated then that dice image will be blitted so it looks like a dice rolling. This happens for both dice simultaneously. Once the ‘time’ variable has exceeded both of their roll times then the whatever dice rolls are left will be returned.

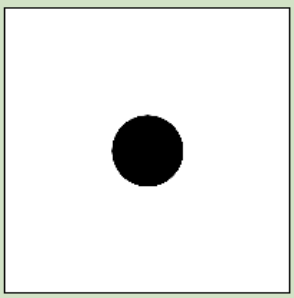
If the ‘diceRoll’ value is false then the players previous dice roll will be blitted.

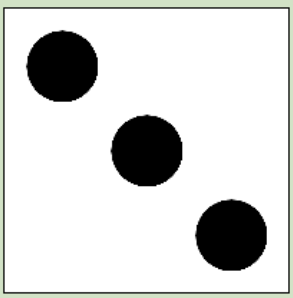
Dice images swap constantly while rolling:

Mikey's Turn

Money: 1500

Rolling Dice...





```
#If out has a value then the value will be returned and all of the buttons will be hidden
if self.out != None:
    self.button1.hide()
    self.button2.hide()
    self.xButton.hide()
    self.upgradeButton.hide()
    #If the move player animation was interrupted then the player will skip to their destination
    if movePlayer != None:
        players[current_player].position = movePlayer[0]
    return self.out

#If the player is a computer all buttons but the x button will be hidden
if isinstance(players[current_player], Computer):
    self.button1.hide()
    self.button2.hide()
    self.upgradeButton.hide()
    # Once the time exceeds the computer time and there isnt a dice rolling or player moving then the computer will return
    if time > COMPUTER_TIME and diceRoll != True and movePlayer == None:
        #Before it finishes the computer has the chance to buy houses
        players[current_player].buyHouses()
        return computerOutput
#Used to update pygame widgets
pygame_widgets.update(events)
#Updates the screen
pygame.display.flip()
#The time is increased at the end of each loop
time += 1
#Limits the framerate to 30 frames per second
clock.tick(FPS)
```

At the end of the main loop, we check if the attribute ‘out’ has a value. If so then that value is returned. This means that when we click on button 1 or 2 the value of ‘out’ is set and the loop ends. In addition if the move player animation was happening then the players position will be instantly set to its destination allowing the players to skip the animation.

If the current player is a computer player then all buttons other than the ‘xButton’ are hidden. This is because all of the other buttons are used to affect the game whereas the ‘xButton is just used to close the displayed card which a human player might be looking at while the computer is having their turn. Once the ‘time’ value exceeds the chosen ‘COMPUTER_TIME’ then assuming the move player or dice roll animation isn’t taking place then the loop will end. This means that the computers automatically progress text. The output returned is a variable called computer output. This can be chosen as a kwarg but it is usually set to its default value. It allows the computers output to be decided outside of the graphics class and outputted here in the same place the players output is outputted.

The function ‘pygame_widgets.update(events)’ is used to update all of the widgets and make them work.

```
#Draws the detail for each section of the board and blits it onto the main screen surface
def drawBoard(self, text):

    #Fills the background
    screen.fill(LIGHTGREEN)
    #Creates the 4 surfaces that make up the board
    bottom_surf = pygame.surface.Surface((SQUARE_SIZE * 10, SQUARE_SIZE))
    left_surf = pygame.surface.Surface((SQUARE_SIZE * 10, SQUARE_SIZE))
    top_surf = pygame.surface.Surface((SQUARE_SIZE * 10, SQUARE_SIZE))
    right_surf = pygame.surface.Surface((SQUARE_SIZE * 10, SQUARE_SIZE))

    #Creates the common base features for each surface
    def drawBoardBase(surf):
        surf.fill((LIGHTGREEN))
        pygame.draw.rect(surf, BLACK, (0, 0, SQUARE_SIZE * 10, SQUARE_SIZE), LINEWIDTH)
        for i in range(0, 10):
            pygame.draw.rect(surf, BLACK, (SQUARE_SIZE * i, 0, SQUARE_SIZE, SQUARE_SIZE), LINEWIDTH)

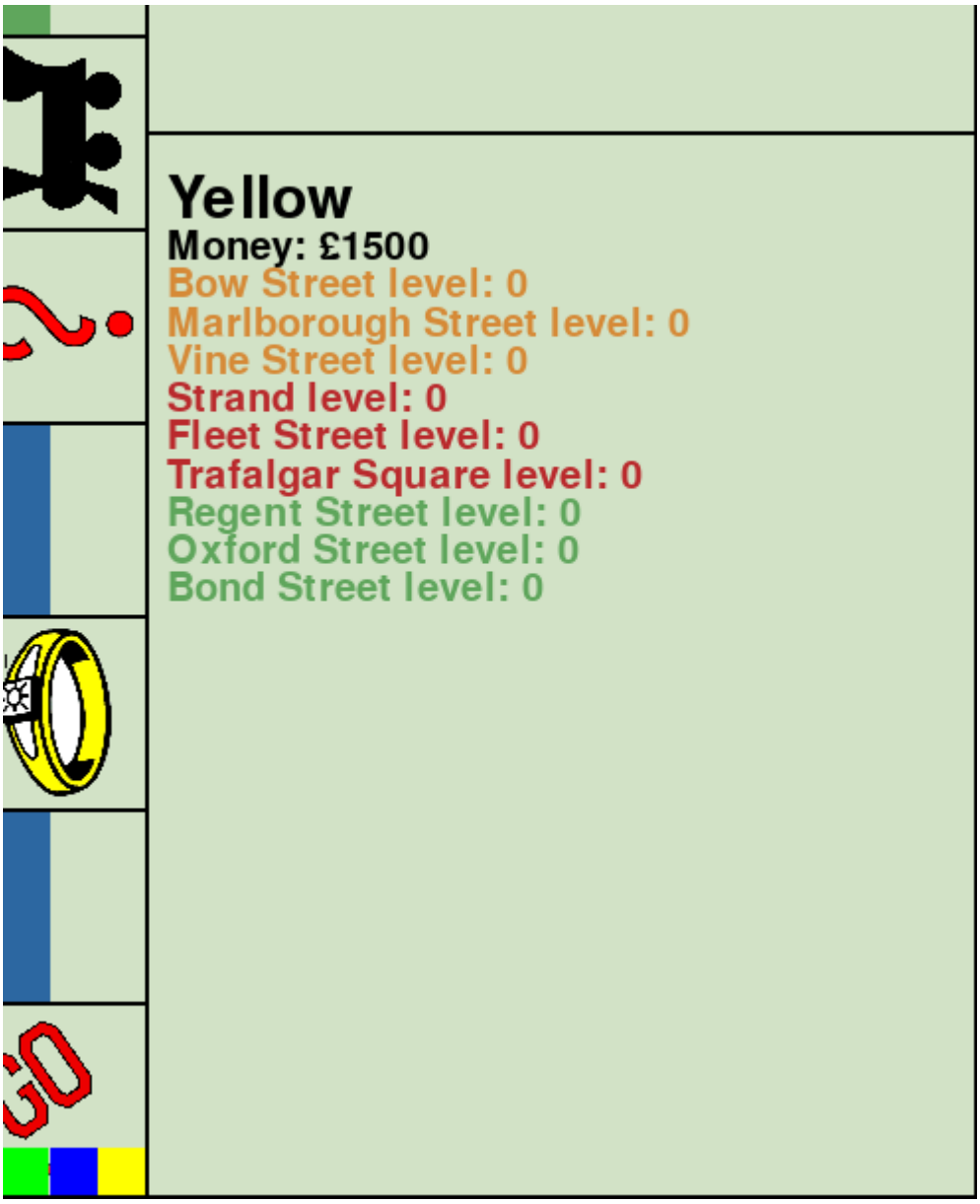
    #Adds the squares the 4 surfaces
    drawBoardBase(bottom_surf)
    drawBoardBase(left_surf)
    drawBoardBase(top_surf)
    drawBoardBase(right_surf)

    #Draws all the coloured parts for each property and blits the images
    pygame.draw.rect(screen, (0,0,0), (SQUARE_SIZE + 421, SQUARE_SIZE + 1, BOARDWIDTH - (2 * SQUARE_SIZE), BOARDWIDTH - (2 * SQUARE_SIZE)), LINEWIDTH)
    #Go
    bottom_surf.blit(go_image, ((SQUARE_SIZE * 9) + LINEWIDTH - 10, 0 + LINEWIDTH))
    #Old Kent Road
    pygame.draw.rect(bottom_surf, BROWN, ((SQUARE_SIZE * 8) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Community Chest 1
    bottom_surf.blit(community_chest_image, ((SQUARE_SIZE * 7) + LINEWIDTH, 0 + LINEWIDTH))
    #Whitechapel Road
    pygame.draw.rect(bottom_surf, BROWN, ((SQUARE_SIZE * 6) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Income Tax
    bottom_surf.blit(income_tax_image, ((SQUARE_SIZE * 5) + LINEWIDTH, 0 + LINEWIDTH))
    #Kings Cross Station
    bottom_surf.blit(station_image, ((SQUARE_SIZE * 4) + LINEWIDTH, 0 + LINEWIDTH))
    #The Angel Islington
    pygame.draw.rect(bottom_surf, SILVER, ((SQUARE_SIZE * 3) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Chance 1
    bottom_surf.blit(chance_image, ((SQUARE_SIZE * 2) + LINEWIDTH, 0 + LINEWIDTH))
    #Euston Road
    pygame.draw.rect(bottom_surf, SILVER, ((SQUARE_SIZE * 1) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
    #Pentonville Road
    pygame.draw.rect(bottom_surf, SILVER, ((SQUARE_SIZE * 0) + LINEWIDTH, 0 + LINEWIDTH, SQUARE_SIZE - (LINEWIDTH * 2), (SQUARE_SIZE / 2) - LINEWIDTH) )
```

‘drawBoard’ is now a method in the graphics class. It also now blits all of the extra images for the other tiles.

Image of board with new icons:




Yellow
Money: £1500
Bow Street level: 0
Marlborough Street level: 0
Vine Street level: 0
Strand level: 0
Fleet Street level: 0
Trafalgar Square level: 0
Regent Street level: 0
Oxford Street level: 0
Bond Street level: 0

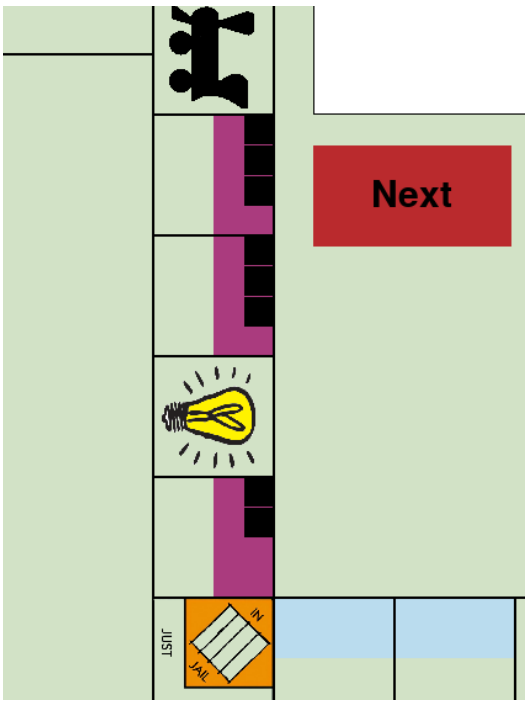
The status cards now have colour coding which is based on the colour values given to the tiles. This makes it much easier to recognise when you have a set and to differentiate between different tiles.

```
#Drawing Houses code
for pos in range (len(board)):
    if isinstance(board[pos], Sets):
        #Variable to show how offset the house must be to avoid overlap
        offset = 0

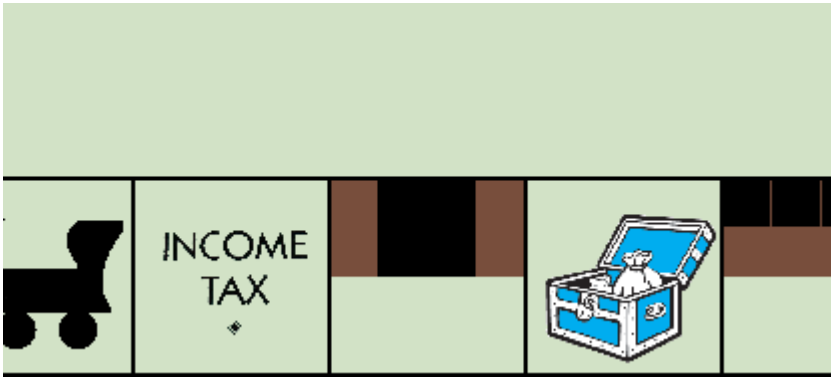
        #Decides which surface it needs to go on
        if pos < 10:
            surf_to_use = bottom_surf
            pos2 = pos
        elif pos < 20:
            surf_to_use = left_surf
            pos2 = pos - 10
        elif pos < 30:
            surf_to_use = top_surf
            pos2 = pos - 20
        else:
            surf_to_use = right_surf
            pos2 = pos - 30
        #If it is a hotel one big house is created
        if board[pos].num_of_houses == 5:
            pygame.draw.rect(surf_to_use, BLACK, ((SQUARE_SIZE * (9 - pos2)) + (SQUARE_SIZE / 4), 0, SQUARE_SIZE/2, SQUARE_SIZE/2), 0)
        else:
            #Loops through all of the houses after each iteration the offset is increased to avoid house overlap
            for i in range(board[pos].num_of_houses):
                pygame.draw.rect(surf_to_use, BLACK, ((SQUARE_SIZE * (9 - pos2)) + ((i * int(PLAYER_SIZE)) + (i)), 0, PLAYER_SIZE, PLAYER_SIZE), 0)
```

The draw board function also has a section for creating small squares for the houses on a space. It works similarly to the player blitting code. For a hotel it is a big square.

Houses:



Hotels:




```
#Draws boxes for all the text
pygame.draw.rect(screen, WHITE, (551, 116, 425, 83)) #Box and outline for  player turn
pygame.draw.rect(screen, BLACK, (551, 116, 425, 83), LINEWIDTH)
pygame.draw.rect(screen, WHITE, (551, 224, 425, 83)) #Box and outline for money
pygame.draw.rect(screen, BLACK, (551, 224, 425, 83), LINEWIDTH)
pygame.draw.rect(screen, WHITE, (551, 332, 425, 257)) #Box and outline for text
pygame.draw.rect(screen, BLACK, (551, 332, 425, 257), LINEWIDTH)

#Text for whose turn it is
drawText(screen, f"{players[current_player].name}'s Turn", BLACK, (551, 116 + 5, 424, 83), FONT40, ANTI_ALIASING)
#Text for how much money the current player has
drawText(screen, f"Money: {players[current_player].money}", BLACK, (551, 224 + 5, 424, 83), FONT40, ANTI_ALIASING)
#Text for what is happening at the moment
drawText(screen, text, BLACK, (551, 332 + 5, 400, 257), FONT40, ANTI_ALIASING)
```

This method also blits the text as mentioned earlier. The text it blits is passed to it as a parameter.

```
#Updates the out variable
def setOut(self, bool):
    self.out = bool

def canUpgrade(self):
    if isinstance(board[self.blitted_card], Sets):
        if board[self.blitted_card].id_of_owner == current_player and board[self.blitted_card].can_buy_houses and players[current_player].passed_go and
            return True
    return False
```

```
and not isinstance(players[current_player], Computer) and players[current_player].money > board[self.blitted_card].house_price:
```

The set out method is used by the buttons to change the value of the 'out' attribute. Since the widgets only take lambda statements which can change variables directly, this method is necessary.

The can upgrade method calculates if the current player is able to upgrade a given property at that time. The 'can_buy_houses' attribute only checks if the pre conditions for buying a house

are met, this method also checks if the player is the owner and if they can afford it. If the player is a computer player then it will return false as it is only used for players and computers upgrade their houses in a different way.

```
#Blits the info card about a tile onto the screen
def blitCard(self):
    if self.blitted_card != None:
        #Gets the surface with all of the info blitted on it from the show info method
        surface = board[self.blitted_card].showInfo()
        #Adds text saying to upgrade house if it is possible to upgrade
        if isinstance(board[self.blitted_card], Sets):
            if self.canUpgrade():
                surface.blit(FONT20.render(f"Add house for £{board[self.blitted_card].house_price}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * 10))
        #Blits the surface into the correct place
        screen.blit(surface, (421 + (6 * SQUARE_SIZE), SQUARE_SIZE + 1))
        self.xButton.show()

#Upgrades the set and re draws the board so the house is blitted and the players money is updated
def upgradeCard(self, pos, text):
    board[pos].upgrade()
    self.drawBoard(text)
    self.blitCard()
    #If the house is no longer able to be upgraded the upgrade button is hidden
    if not self.canUpgrade():
        self.upgradeButton.hide()

#Unblits the card by removing the value of the blitted card attribute and covering up the blitted card with a rect the same colour as the background
def stopCardBlit(self):
    self.blitted_card = None
    self.xButton.hide()
    self.upgradeButton.hide()
    pygame.draw.rect(screen, LIGHTGREEN, (421 + (6 * SQUARE_SIZE) -1, SQUARE_SIZE + 1 + 1, CARD_WIDTH, CARD_HEIGHT))
```

These last 3 methods are all used for the card blit system.

‘blitCard’ will check if a card should be blitted, if so it will get the surface from the ‘showInfo’ method and blit it. If the tile can be upgraded then it will add text saying to do so. It will also ‘show’ the ‘xButton’.

‘upgradeCard’ is called when the ‘upgradeButton’ is clicked on. It will upgrade the tile with its method and redraw all screen elements. This is so the house icons on the board are updated and the players money is updated. If the tile can no longer be updated afterwards then the ‘upgradeButton’ is hidden.

‘stopCardBlit’ is called when the ‘xButton’ is clicked. It sets ‘blitted_card’ to none and hides the ‘xButton’ and ‘upgradeButton’. Afterwards it draws a rectangle over the card which is the same colour as the background in order to hide it without having to redraw the whole board again.

```
#Checks if houses can be built on that space by checking that the whole set is owned by one person and the houses are spread equally
def checkHouseEligability():
    for i in sets:
        if isinstance(board[i[0]], Sets):
            lowest = board[i[0]].num_of_houses
            for j in i:
                if board[j].num_in_set == board[j].size_of_set:
                    if board[j].num_of_houses < lowest:
                        lowest = board[j].num_of_houses
            for j in i:
                if (board[j].num_of_houses - lowest) < 1 and board[j].num_of_houses <5 and board[j].num_in_set == board[j].size_of_set:
                    board[j].can_buy_houses = True
                else:
                    board[j].can_buy_houses = False
```

Outside of any classes I define the 'checkHouseEligablity' procedure. It loops through all coloured sets and checks if they can have houses on them. They must have the houses spread evenly to buy them and the property must be a part of a whole set.

```
#The names of the different players, will be created by the menu
names = ["Red", "Green", "Blue", "Yellow"]
#Number of players in the game
num_of_players = 4
#Number players that are computers, cant be greater than number of players
num_of_computers = 0

#Array of all player
players = [None] * num_of_players

#instantiates players using above data
for i in range(num_of_players):
    if i > num_of_players - num_of_computers -1 :
        players[i] = Computer(i)
    else:
        players[i] = Player(names[i], i)
```

Above is the code used to create all of the players. The three variables at the top will eventually be assigned in the main menu by the user but for now they are written in the code. A 'num_of_players' number of players are created by the for loop and 'num_of_computers' of which will be computer players.

```
#Used to allow a quick game where all of the sets are randomly distributed to each player
QUICK_GAME = True
if QUICK_GAME:
    sets2 = list(sets)
    random.shuffle(sets2)
    num = random.randint(0, num_of_players - 1)
    for i in sets2:
        num += 1
        num = num % num_of_players
        for j in i:
            board[j].getOwned(players[num])
```

Quick game is an option that is useful for testing housing related things but it will likely be implemented into the full game as it is quite simple to implement. If it is enabled then all of the property sets will be randomly distributed between all of the players equally.

```
#Defining graphics class
gc = Graphics()


#Loops through all players and make them take their turn
while True:

    players[current_player].takeTurn()
    current_player += 1
    current_player = current_player % num_of_players
    players_left = 0
    for i in players:
        if not i.dead:
            players_left += 1
    if players_left == 1:
        break

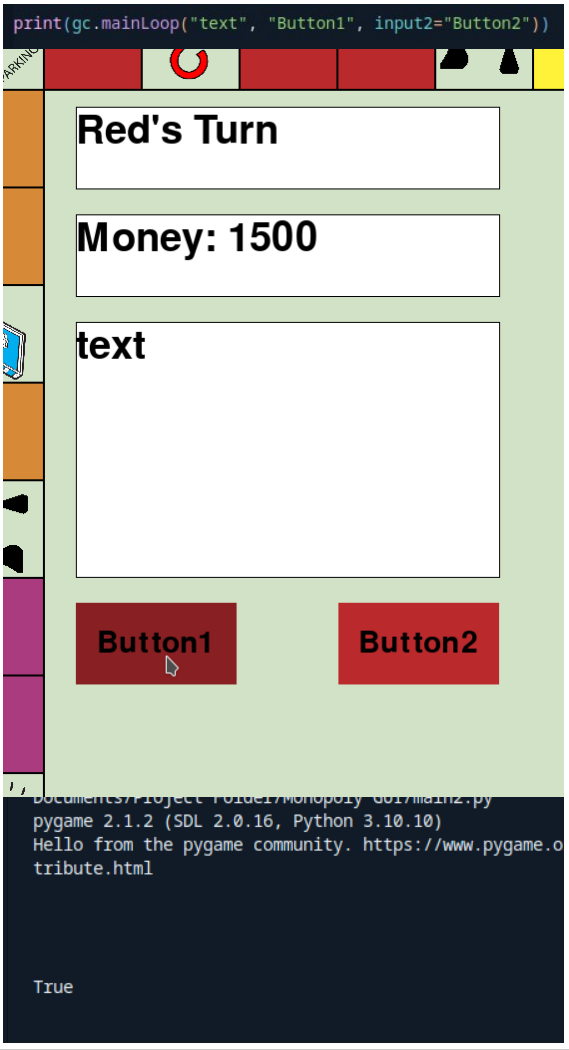

#Finds out who is left and congratulates them
for i in players:
    if not i.dead:
        gc.mainLoop(f"{i.name} Wins!!", "Close")

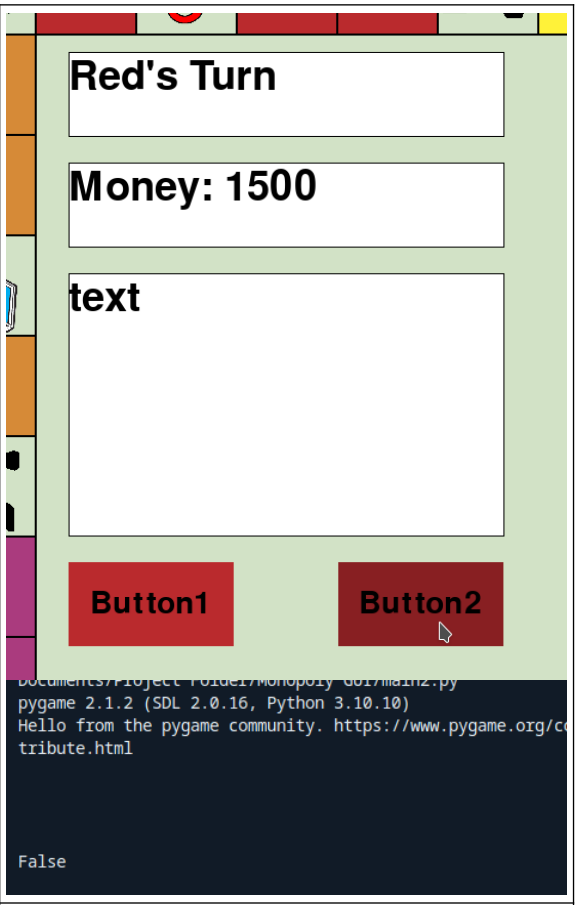
pygame.quit()
```

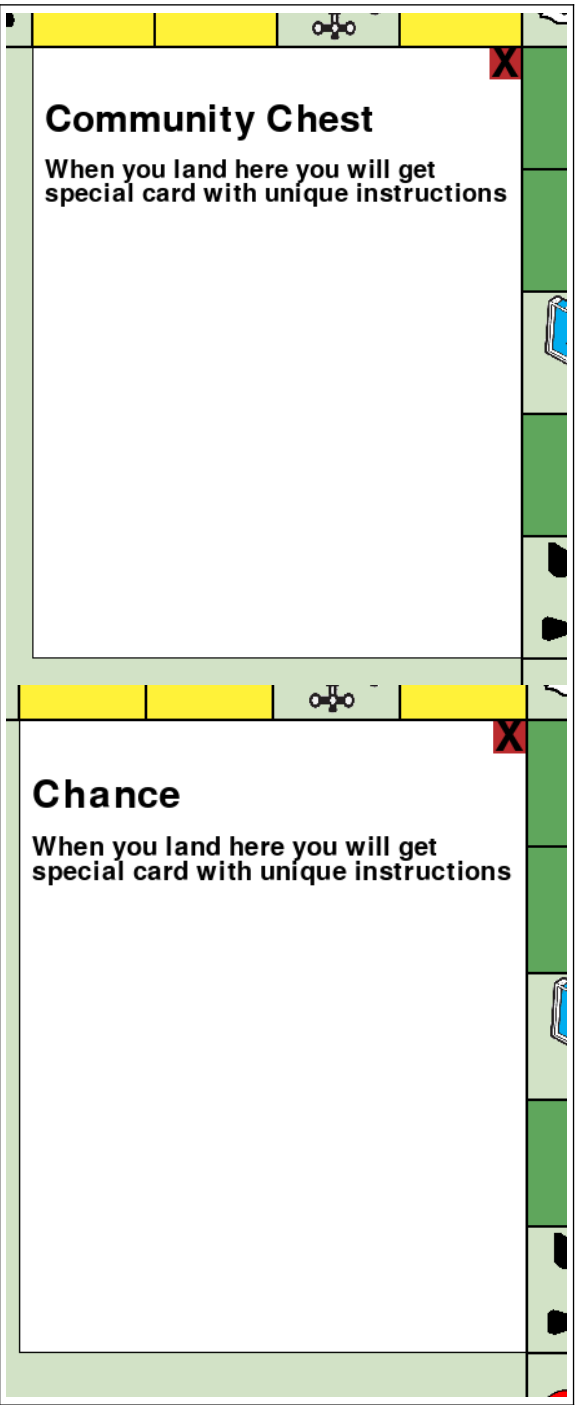
Finally at the end of the code the graphics object is instantiated and the game player turn loop begins until only one player is left.

Testing of Second Iteration:

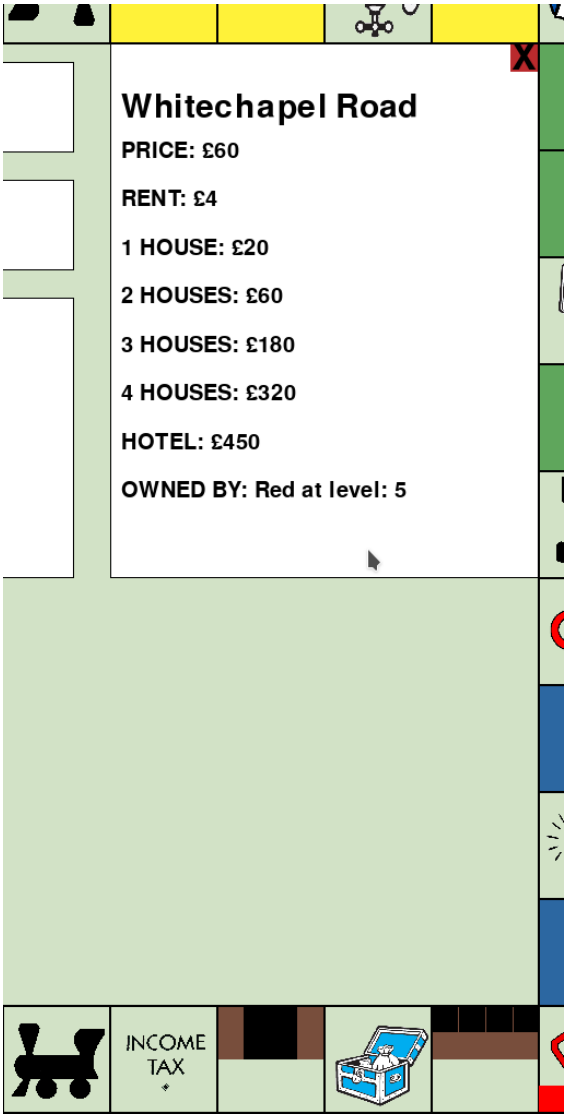
Previous Tests:

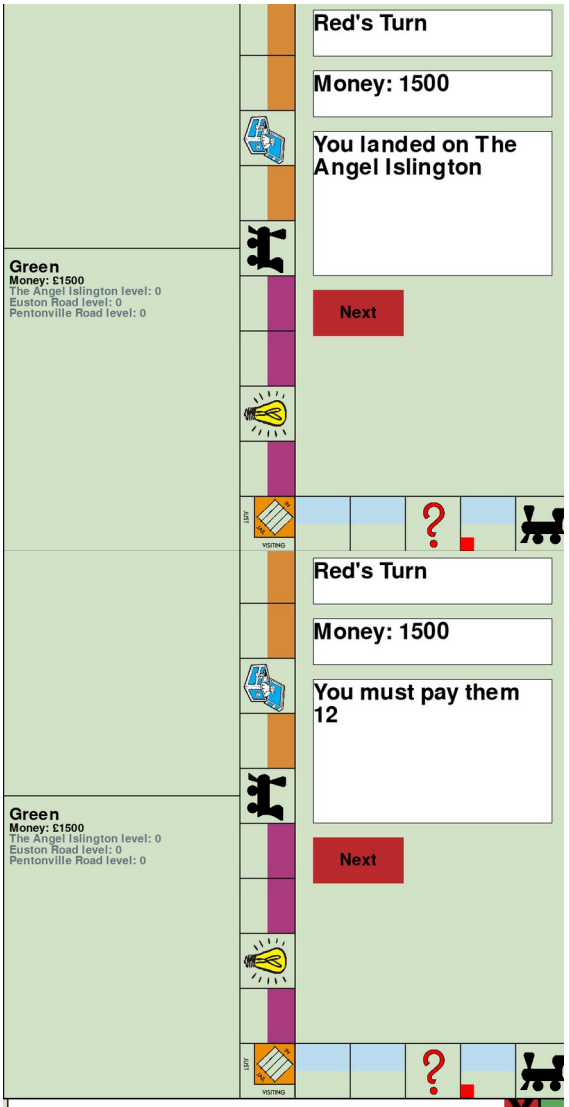
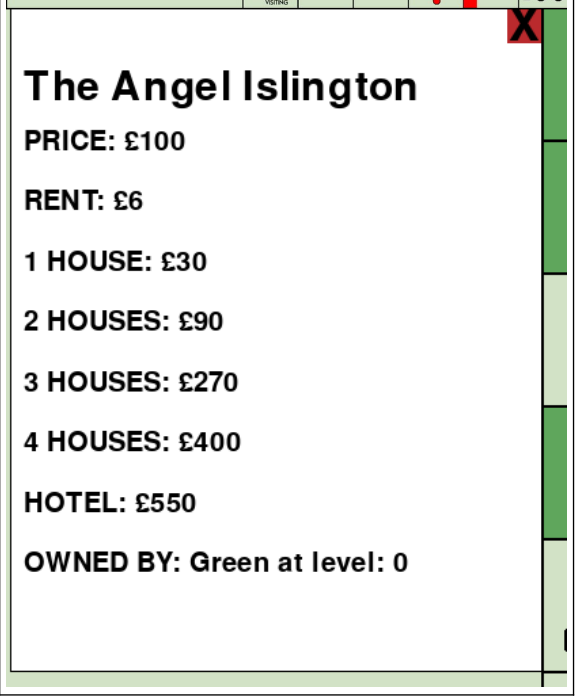
Test Number	Test Description	Passed?	Evidence Description	Evidence
101	When clicking the first button in the main loop the function should return True	Yes	New widget buttons work the same	

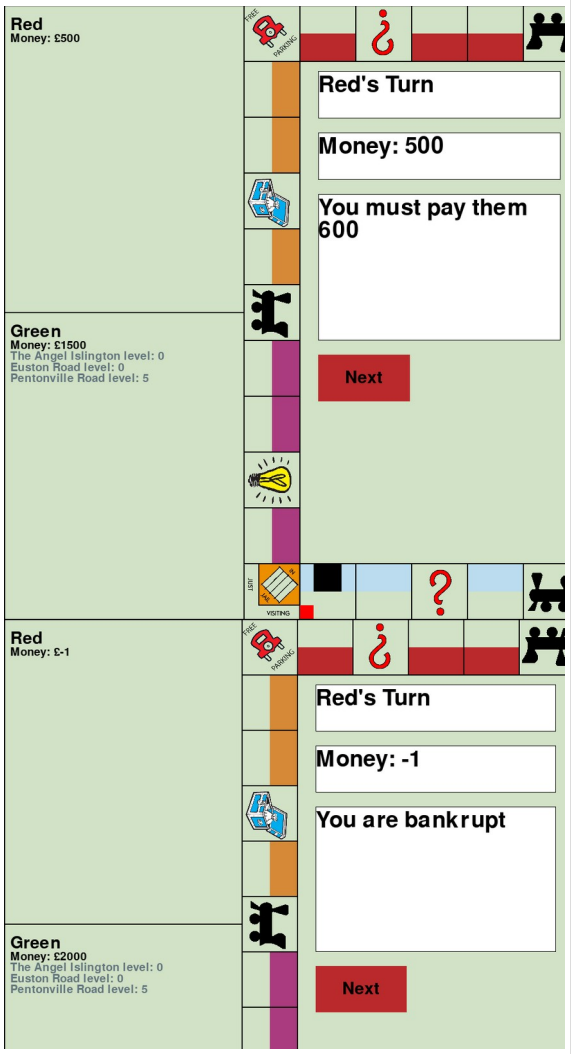
102	When clicking the second button in the main loop the function should return False	Yes	New widget buttons work the same	
103	When window is attempted to be closed, pygame stops running	Yes		Same as previous

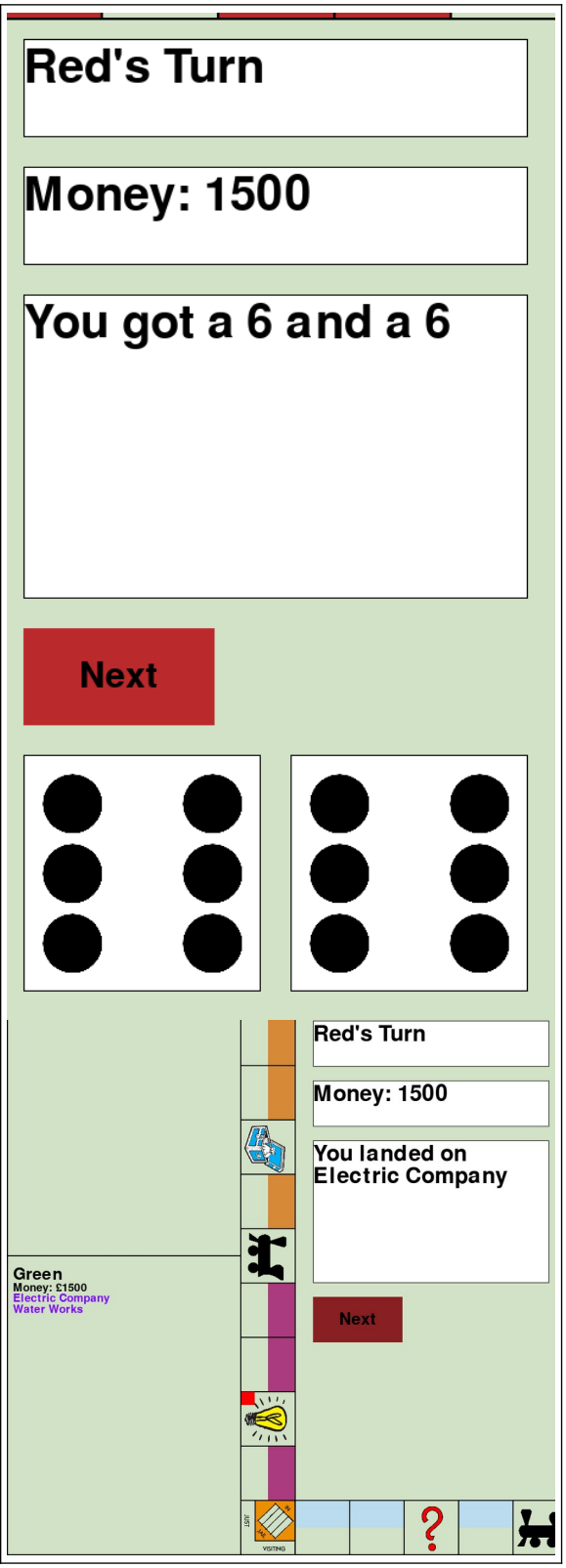
104	Clicking on all the different tiles on the screen should display a box with information about the tile	Yes	Community chest and Chance tiles now have tile cards	
105	When the player clicks on a property that they own but are not eligible to place houses on then they should not have the option to do so	Yes		Same as previous

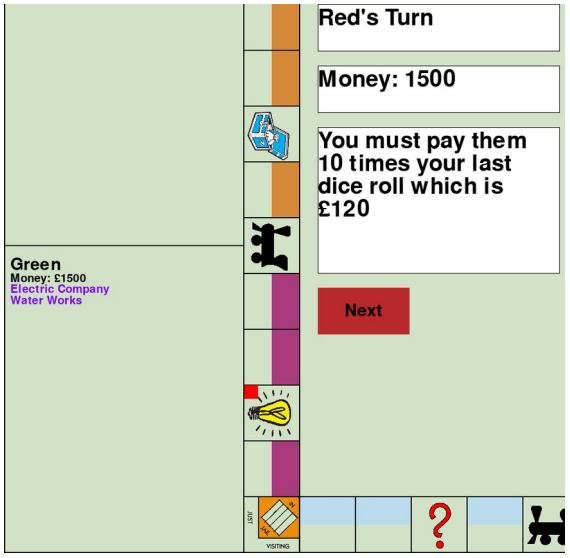
106	When the player clicks on a property that they own and are eligible to purchasing houses on it then they should have the option to do so	Yes		Same as previous
107	When the a player clicks the button to purchase a house, the property should gain an extra level for the extra house	Yes		Same as previous

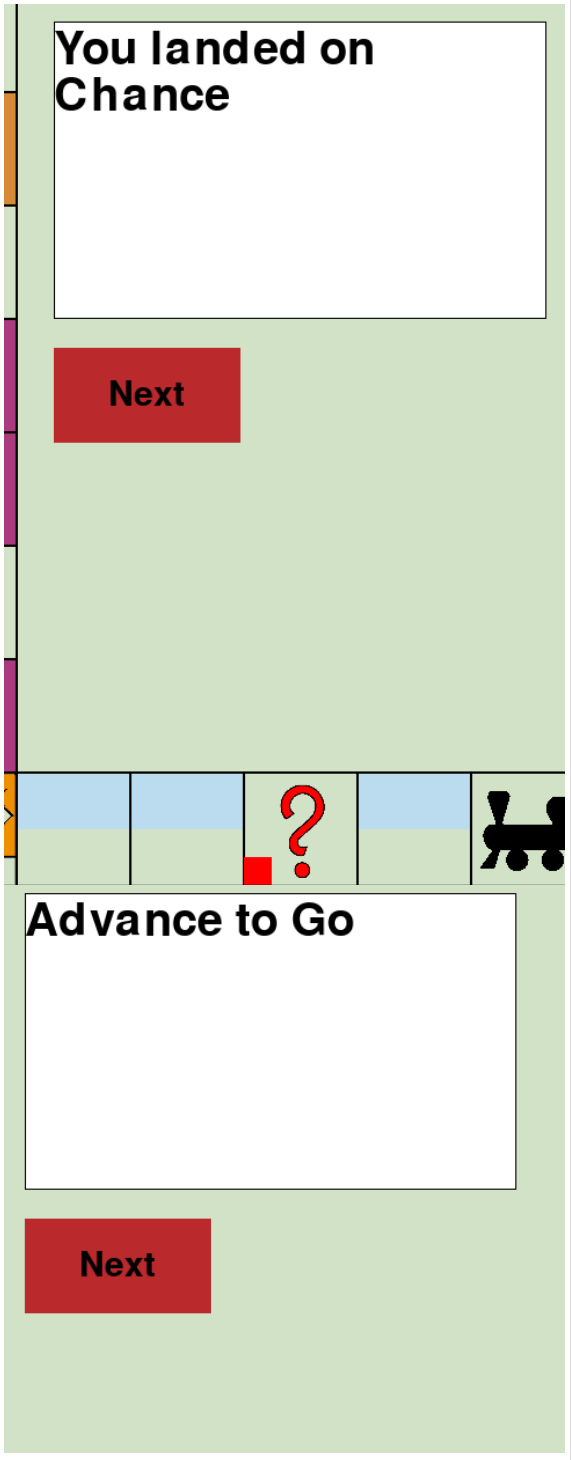
108	The player should not be able to upgrade a set more than 5 times	Yes	Button and text to upgrade no longer show up	
109	The player should be able to buy a property when it is unowned and they can afford to. When they purchase it the money should be taken from their wealth.	Yes		Same as previous
110	The player should be prevented from purchasing an unowned property if they cant afford it	Yes		Same as previous

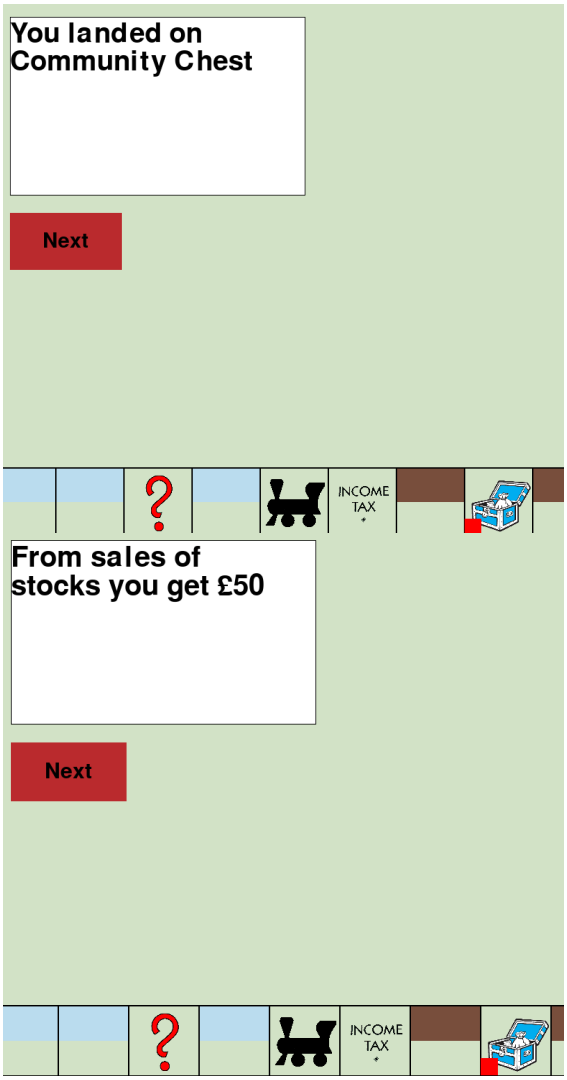
111	When a player lands on a property owned by somebody else they should pay them the exact amount required	Yes	The payments work as expected and the double rent when full set is owned is now working properly	 

112	When a player lands on a property owned by someone else and they cant afford to pay the other person. The other person should get the remainder of their cash and the player should go bankrupt	Yes	‘Red’ owes ‘Green’ £600 but they only have £500 so after the transaction ‘Red’ is bankrupt and ‘Green’ gets their remaining £500	

113	When the player lands on a utility where both are owned by the same person they should have to pay the owner 10 times their previous dice roll	Yes	The utility error is fixed and 'Red' has to pay 10 times the dice roll as both utilities are owned by 'Green'	

				
114	When the player lands on a station which is owned by another player who owns multiple stations they should have to pay them more depending on how many they own	Yes		Same as previous
115	When the player rolls a double they should get an extra turn	Yes		Same as previous
116	When the player rolls 3 doubles in a row they should go to jail	Yes		Same as previous

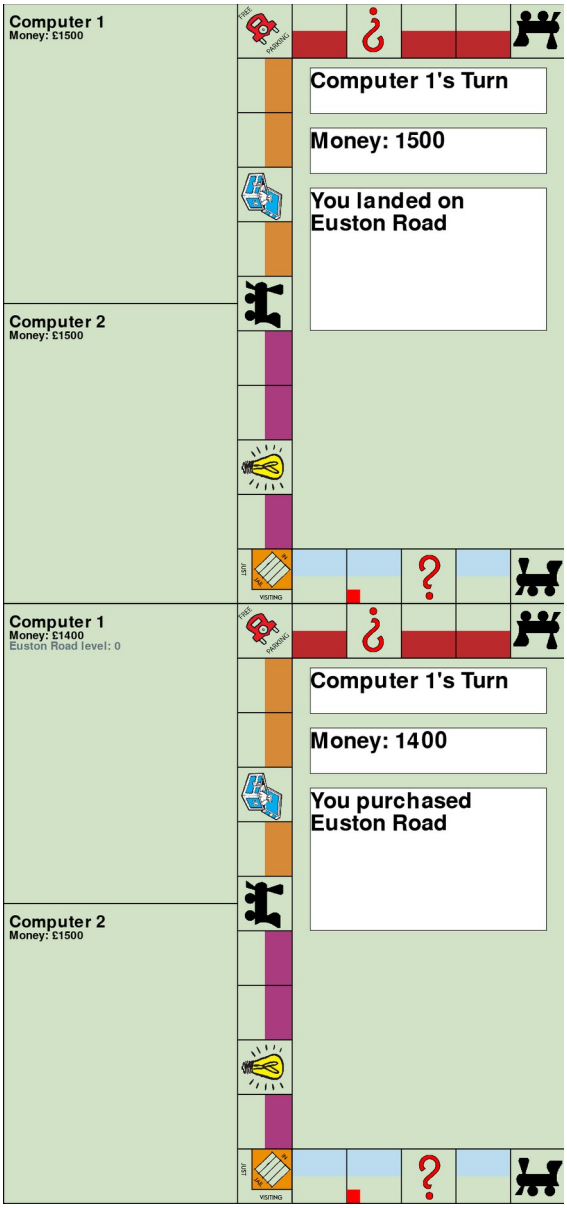
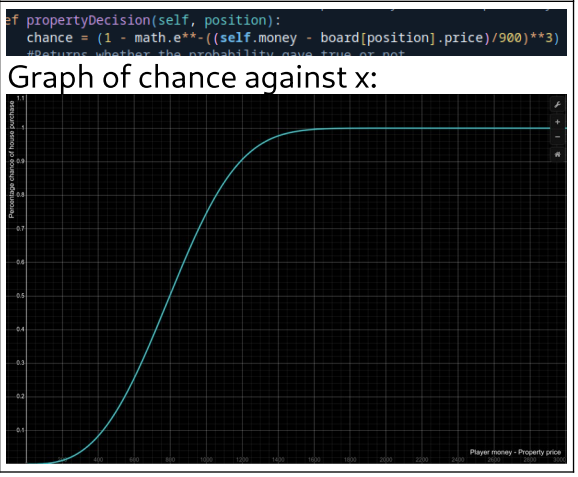
117	When the player lands on a chance or community chest space they should receive a card with instructions on it which should be followed correctly by the program	Yes	All cards work as they should	

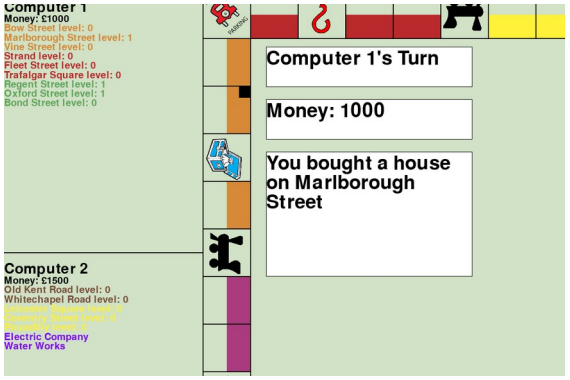
				
118	When the player moves past Go they should receive £200	Yes		Same as previous
119	When the player lands on a tax space they pay the money specified to the free parking space	Yes		Same as previous

120	When the player lands on Free Parking they should get all of the money in the space and there should be £0 in free parking afterwards	Yes		Same as previous
121	When the player lands on Go to Jail they should be immediately sent to jail	Yes		Same as previous
122	When all players but one go bankrupt the remaining player should be declared the winner and be congratulated	Yes		Same as previous

New tests:

Test Number	Test Description	Pass?	Evidence Description	Evidence
123	Computer Players progress text automatically	Yes		Cant show in images

124	Computer Players buy properties automatically	Yes	Computer 1 lands on the property and purchases it	
125	Computer Players wont buy properties they can't afford	Yes	Chance value will always be zero if the computer cant afford the property	

126	Computer players can buy houses automatically when they are eligible	Yes	Computer 1 has the full set and the houses are spread out equally so they put a house there automatically	

127	Computer players wont buy houses they cant afford	Yes	The value of x will always be below 0 if the house price is greater than the players money meaning the value of chance will be 0 or below as seen on the graph	Code for chance of buying house:
				`x = (self.money - (2 * board[j].house_price)) * (((board[j].house_values[board[j].num_of_houses + 1] - board[j].house_values[board[j].num_of_houses])/1000) + 0.5) + random.randint(-300, 100)`
				
				`chance = (1 - math.e**-((x)/900)**3)`
				
				Graph of chance against x:
				 

Review of Second Iteration:

What has been done? I have added all of the features that make up the gameplay. The chance card and community chest system is fully in place. There are now computer players. A graphics class was made to simplify the process of creating the graphics and handling user inputs. There is now a dice roll and player move animation. The board graphic is now fully complete with all of the images in place. Many quality of life changes and code improvements.

What went well? The graphics class makes the main loop much simpler. The chance and community chest class worked out well as I was able to simplify about 20 different card instructions into a few common functions.

What went poorly? The computer players still behave in a strange way in regards to how they decide to buy properties and the decision making could probably be improved.

What improvements to the code could be made?

- The text inputted into the main loop as a parameter could be assigned to an attribute of the class so I don't need to keep passing it around through the class methods as a parameter.
- The when the tiles are instantiated they should be assigned straight to the 'board' array instead of being assigned to variables first.
- When deciding to purchase a property the computer player should take into account if they already own some of the set.

What improvements to the usability could be made?

- The player icons could appear in the jail cell when they are in jail
- There could be a small icon next the players name on the status card to show if they are in jail
- There could be a coloured icon on the status cards to show the player colours

What success criteria were met?

- The game is fully playable
- There are additional UI elements to improve the user experience
- There are optional AI players

What success criteria were not met?

- There is no main menu
- There is no way to save the game

Conclusion. This second iteration was a big success. Many key features and quality of life features were implemented, the were no failed tests and the previous code was improved in many ways.

Third Iteration:

Break Down of Third Iteration:

This iteration saw the addition of the start menu and the save system. In addition many small quality of life additions and bug fixes were made to polish the game.

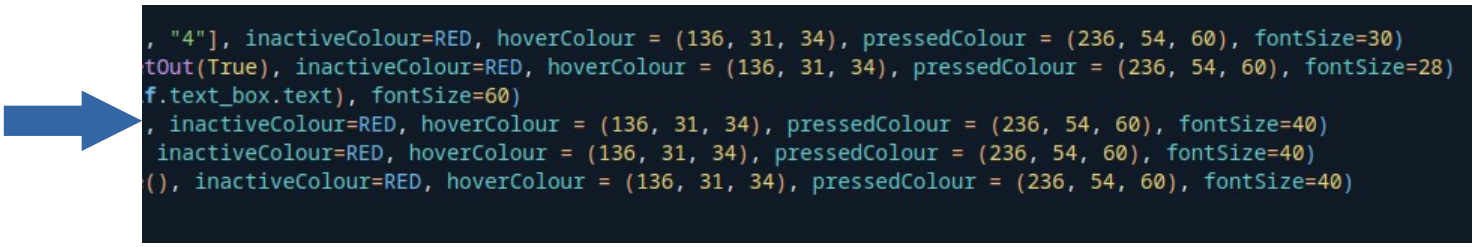
```
import random
import pygame
from pygame.locals import *
import pygame_widgets
from pygame_widgets.button import Button
from pygame_widgets.textbox import TextBox
from pygame_widgets.dropdown import Dropdown
import math
import pickle
```

To create the save system I needed to import a new module called 'pickle'. It allows the serialisation of objects in the game into a binary file. It can then deserialise them back into the game even after it has been closed. I also required additional widgets for the main menu system so I had to import them for pygame widgets.

```
#Procedure to quit the game and close python
def quitGame():
    pygame.quit()
    exit()
```

Here I define a procedure that allows me to quickly quit pygame and exit python which is useful for the new close button I have created which helps players close the game which is usually in full screen.





In order to create the main menu I defined a class. It has an attribute for each widget that I use with. It also has an attribute corresponding to each game setting such as the number of players. The 'out' and 'text_output' attributes are just used to help the input loop understand when an input has been made.

```
#Method that goes through the menu options
def menu(self):

    #Asks user if they want to load their previous save
    if self.menuLoop("Load Save?", "buttons"):
        self.load_game = True
        return

    #Asks user if they want to play a quick game
    if self.menuLoop("Play a quick game?", "buttons"):
        self.quick_game = True

    #Asks user how many players they want
    self.num_of_players = int(self.menuLoop("How many players?", "drop_down"))

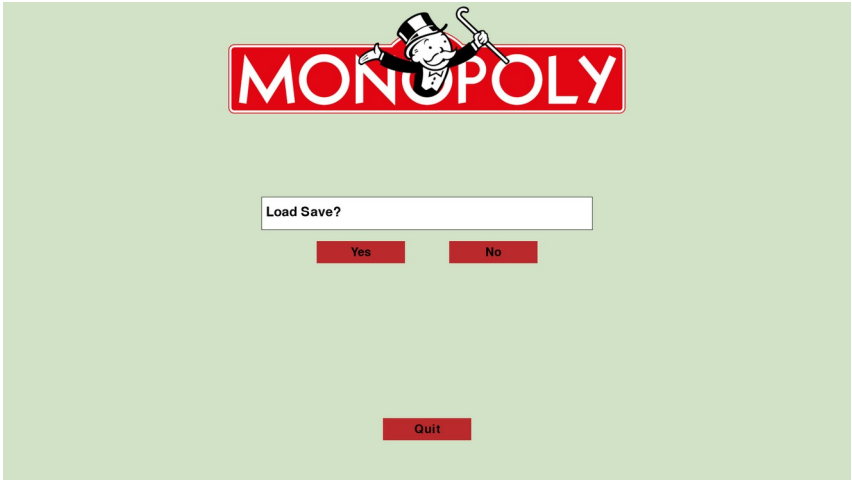
    #Creates array of choices for how many computers
    computer_choices = []
    for i in range(self.num_of_players + 1):
        computer_choices.append(str(i))
    self.drop_down_button.hide()
    self.drop_down_button = Dropdown(screen, SCREENWIDTH/2 - 200, SCREENHEIGHT / 2, 150, 50, name="Select Number",choices=computer_choices,

    #Asks user how many computer players they want
    self.num_of_computers = int(self.menuLoop("How many players should be computers?", "drop_down"))
```

```
,choices=computer_choices, inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60),fontSize=30)
```

This is the first half of the menu method. It is used to take the player through the menu options. It uses another method called 'menuLoop' to display the options. These two methods work similar to the 'takeTurn' method and the 'mainLoop' method.

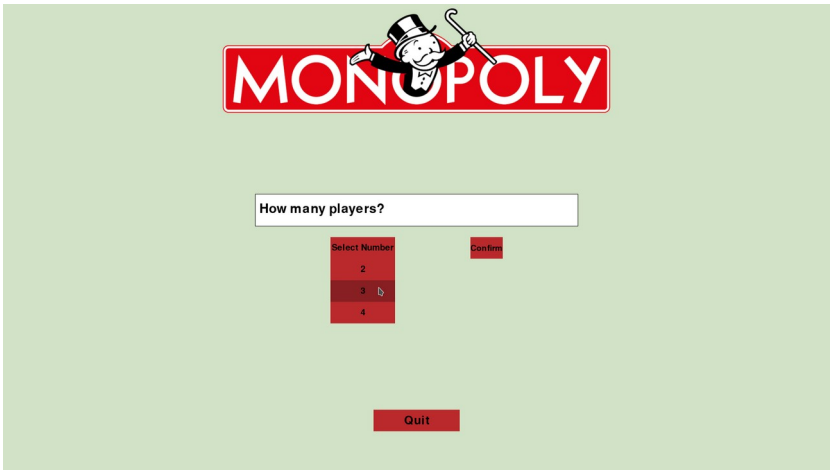
First it asks the user if they would like to load the previous game. If so then the 'load_game' attribute is set to true and a return statement is called to force end the method.



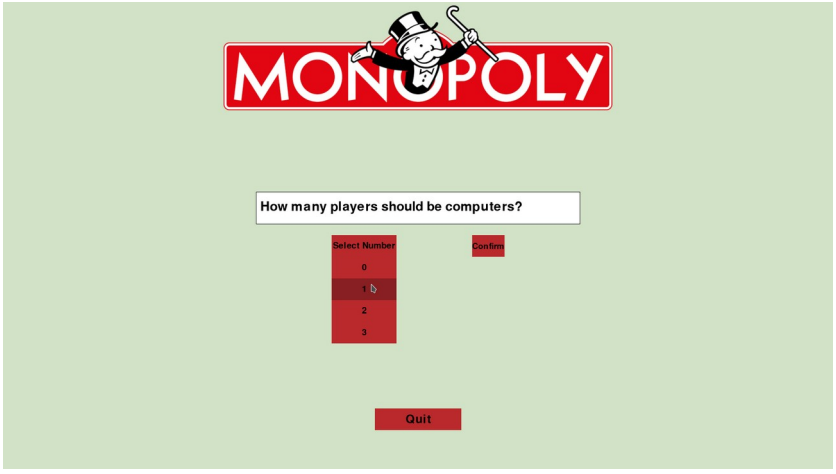
After it asks the user if it wants to play a quick game in the same way.



To find out the number of players a drop down button is used to provide the choices.



Once the number of players is chosen, the choice for the number of computer players needs to be from 0 to the number of players so a for loop is used to create the array of choices. Since the drop down button choices cannot be changed after instantiating the object, the 'drop_down_button' object needs to be re instantiated to show the choices for the number of computer players.



```
#Code for inputting and validating the names of the players
if (self.num_of_players - self.num_of_computers) > 0:

    #Empty array of names
    self.names = [None] * (self.num_of_players - self.num_of_computers)
    for i in range(len(self.names)):

        #Variable if a name isn't valid
        not_valid = True
        #Error message if a name isn't valid
        error_message = ""
        while not_valid:
            not_valid = False
            self.names[i] = "".join(self.menuLoop(f"Name of player {i + 1}. Press enter when done", "text_box", error_message = error_message))

            #If more than one person has that name then you need to chose a new name
            if i > 0:
                if self.names.count(self.names[i]) > 1:
                    not_valid = True
                    error_message = "Your name is the same as a previous name"

            #If the name has no characters
            all_space = True
            for char in self.names[i]:
                if ord(char) != 32:
                    all_space = False
            if all_space:
                not_valid = True
                error_message = "Your name can't only have spaces"

            #If the name is longer than 12 characters then you need to choose a new name
            if len(self.names[i]) > 12:
                not_valid = True
                error_message = "Your name is above 12 characters"

            #If the name is empty
            if len(self.names[i]) < 1:
                not_valid = True
                error_message = "Input something into the box"

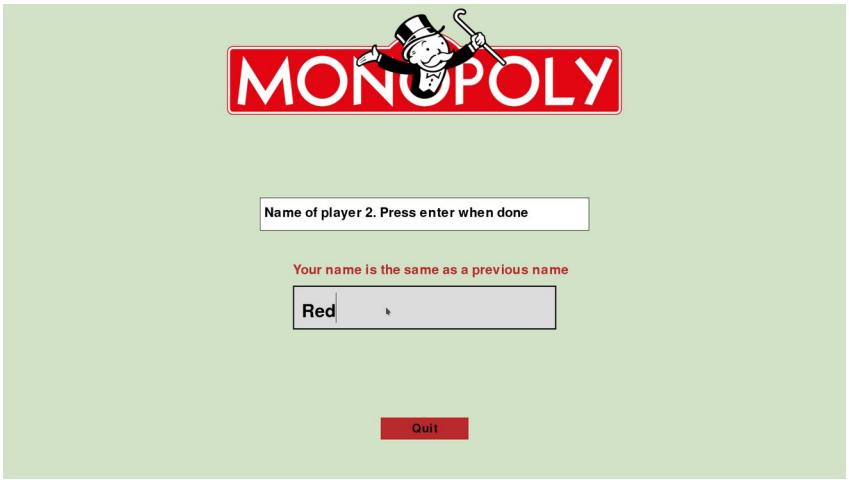
        #Reset the text box
        self.text_box.setText("")
```

This is the second half of the method is for inputting and validating the names. First it checks if any of the players are human players because only human players are given names. A for loop is made to loop through each of the names that needs to be made. Inside the for loop is a while loop which makes the player input the name and re input it if it isn't valid using the 'not_valid' variable.

To collect the inputted name the 'menuLoop' is called with the argument that makes it display the text box for the player to input. The output of this is then put into the names array. The variable 'error_message' is used to display a message to the user if they inputted an invalid name and tell them what was wrong with it.



If the name inputted isn't the first name inputted, it is checked to see if it identical to any of the previous names before hand. If the name only contains spaces then it is invalid. And if the name is not from 1 to 12 characters long. Whenever one of these is not met, a string is given to the 'error_message' variable to be displayed. After each name is successfully inputted, 'setText' is called on the text box widget in order to reset the inputted text on it.




```
#Method that displays menu options
def menuLoop(self, text, widget_to_use, **kwargs):
    #Reset output variables
    self.out = None
    self.text_output = None
    self.hideButtons()
    error_message = kwargs.get("error_message", None)

    #Showing the buttons needed
    self.quitButton.show()

    if widget_to_use == "drop_down":
        self.drop_down_button.show()
        self.drop_down_confirm.show()

    elif widget_to_use == "text_box":
        self.text_box.show()

    elif widget_to_use == "buttons":
        self.button_1.show()
        self.button_2.show()


    while True:
```

This the first part of the menu loop method. It starts by hiding all of the buttons and only showing the ones needed which are selected by the 'widget_to_use' parameter.

```
        while True:
            #Adding graphics
            screen.fill(LIGHTGREEN)
            pygame.draw.rect(screen, WHITE, (SCREENWIDTH/2 - 750 / 2, SCREENHEIGHT/2 - 100, 750, 75))
            pygame.draw.rect(screen, BLACK, (SCREENWIDTH/2 - 750 / 2, SCREENHEIGHT/2 - 100, 750, 75 ), LINEWIDTH)
            screen.blit(monopoly_title_image, (510, 10))

            #Adding text
            screen.blit(FONT30.render(text, True, BLACK), (SCREENWIDTH / 2 - 750 / 2 + 10, SCREENHEIGHT/2 - 100 + 20))
            if error_message != "":
                screen.blit(FONT30.render(error_message, True, RED), (SCREENWIDTH / 2 - 600 / 2, SCREENHEIGHT/2 + 50))

            #Checking if user closes game
            events = pygame.event.get()
            for event in events:
                if event.type == pygame.QUIT:
                    #Close the game
                    quitGame()
            #If the text box has a value then end the loop
            if widget_to_use == "text_box":
                if self.text_output != None:
                    self.hideButtons()
                    return self.text_output

            #If the out attribute has value then end the loop
            if self.out != None:
                if widget_to_use == "drop_down":
                    if self.drop_down_button.getSelected() != None:
                        self.hideButtons()
                        return self.drop_down_button.getSelected()
                    else:
                        self.out = None
                else:
                    self.hideButtons()
                    return self.out

            #Update the widgets and the screen
            pygame_widgets.update(events)
            pygame.display.update()
            #Caps the framerate
            clock.tick(FPS)
```

Within the while loop the background filled in with the monopoly logo placed at the top of the screen. If the widget in use is a text box, if the 'text_output' attribute has a value then the buttons are hidden and that value is returned. The attribute is given the value of the text in the text box whenever the enter button is pressed on it.

If the widget in use is a drop down button then it will return the value of the drop down button if the confirm button is pressed, which sets the 'out' attribute to true, and if a value is selected in the drop down button. If no value is selected then nothing will happen.

If it isn't in drop down mode then when the value of 'out' is changed then that value will be outputted. This is used when the two buttons are displayed.

```
    #Changes the value of text_output
    def setTextOutput(self, text):
        self.text_output = text

    #Changes the value of out
    def setOut(self, bool):
        self.out = bool

    #Hides all of the buttons
    def hideButtons(self):
        self.drop_down_button.hide()
        self.text_box.hide()
        self.drop_down_confirm.hide()
        self.button_1.hide()
        self.button_2.hide()
        self.quitButton.hide()

    #Shows the winner
    def winner(self, player):
        self.menuLoop(f"Congratulations {player.name}, you win!!!", None)


main_menu = MainMenu()

main_menu.menu()
```

Here the final methods are defined. The first two are responsible for changing the output attributes and are called through the widgets. 'hideButtons' is used to easily hide all of the buttons so they can be turned on individually.

The 'winner' method is used to display who the winner is and is called when someone wins. The menu is used instead of the graphics class as it is neater and more easily distinguishable when a player wins.

Below that the 'main_menu' object is created and the 'menu' method is called.

```
class Player():
    def __init__(self, given_name, given_id):
        #What space they are on
        self.position = 0
        #What name they were given
        self.name = given_name
        #How much money they have
        self.money = 1500
        #Identifies which player they are outside of the name and matches the index of the players array
        self.id = given_id
        #If they are bankrupt and out of the game
        self.dead = False
        #What their last rolls were
        self.roll1 = None
        self.roll2 = None
        #The number of turns they have left in jail
        self.jailed = 0
        #If they just got a double
        self.double = False
        #How many doubles they got in a row
        self.double_count = 0
        #Chooses colour
        if self.id == 0:
            self.colour = P_RED
        elif self.id == 1:
            self.colour = P_GREEN
        elif self.id == 2:
            self.colour = P_BLUE
        elif self.id == 3:
            self.colour = P_YELLOW
        #Keeps track of if they have passed go this game
        self.passed_go = False
        #Keeps track of how many get out of jail free cards they have
        self.get_out_of_jail_free = 0
```

Within the player class I made a new attribute to track how many get out of jail free cards a player possesses which is a new chance and community chest card added to the pool. When the player has one they can escape from jail for free and it is used up.

```
def takeTurn(self):
    #Ends turn immediatly if player is out of the game
    if self.dead:
        return
    self.double_count = 0
    gc.mainLoop(f"It is {self.name}'s go", save="True")
    #Checks if player is in jail and gives them option to pay to get out, if not the jail counter goes down by one
    if self.inJail():
        return

    self.double = True
    #Repeats turn while player has a double
    while self.double:
        self.double = False
        #Gives input for player to roll dice
        gc.mainLoop("Click to roll", "Roll")
        self.rollDice()
        #If the player gets 3 doubles in a row then they are sent to jail and the turn ends
        if self.double_count == 3:
            gc.mainLoop("You got 3 doubles in a row and are now sent to jail")
            self.jail()
            return
        #Function is used to increase the position as it provides animation
        self.increasePos(self.roll1 + self.roll2)
        #Checks if player is bankrupt
        if self.money < 0:
            self.bankrupt()
            return
    gc.mainLoop("Your turn has ended")
```

Within the 'takeTurn' method the code for checking a players jail status has been moved to a separate method. This allows this method to be overwritten by the computer class to allow the computers to have their own logic for being in jail without having to overwrite the whole take turn method.

Since the save system cannot save how far a player has gone through their 'takeTurn' method, bugs can be caused by a player changing their attributes throughout their turn, then saving and restarting their turn with the changed attributes. To fix this only the first 'mainLoop' call where it shows whose go it is in the 'takeTurn' method can be saved on. This is indicated by the argument 'save="True"'.

```
#Sets player status to jailed for 3 turns and move player to jail position
def jail(self):
    self.jailed = 3
    self.position = 10
    self.double = False
    self.double_count = 0
```

The jail method now resets a players double values. This is because I encountered a bug where a player was sent to jail by a chance card after having rolled a double and were granted another roll while in jail which causes errors.

```
def inJail(self):
    if self.jailed > 0:
        if self.get_out_of_jail_free > 0:
            if gc.mainLoop(f"You are in jail with {self.jailed} turns left, you can use a get out of jail free card to get out", "Use card", input2="Dont Pay"):
                self.jailed = 0
                self.get_out_of_jail_free -= 1
                return False
        elif self.money > 50:
            if gc.mainLoop(f"You are in jail with {self.jailed} turns left, you can pay 50 to get out", "Pay", input2="Dont Pay"):
                    self.jailed = 0
                    self.money -= 50
                    board[20].free_parking_cash += 50
                    return False

        self.jailed -= 1
        return True
    else:
        return False
```

The 'inJail' method is responsible for checking if a player is in jail and if so, deciding what happens. If it returns true then the player is in jail and if it returns false they are not. If a player is in jail and they have a get out of jail free card they are given the option to use it or stay in jail. If they are in jail and don't have a get out of jail free card but does have over £50 then the player is given the option to pay to get out of jail or stay. If they are in jail but don't meet any of the other requirements then they stay in jail. Whenever the player stays in jail the jail count goes down by one. If they aren't in jail then the method returns false.

```
#Class for computer players
class Computer(Player):
    #Initialises the computer with the name being computer and the id of it
    def __init__(self, given_id):
        super().__init__(f"Computer {given_id + 1}", given_id)



    #Uses mathematical function to decide the probablity of the computer buying the property
    def propertyDecision(self, position):
        num_owned_already = 1
        for i in sets:
            if position in i:
                for j in i:
                    if board[j].id_of_owner == self.id:
                        num_owned_already += 1
        x = (self.money - board[position].price) * num_owned_already
        chance = (1 - math.e**-((x)/900)**3)
        #Returns whether the probability gave true or not
        return random.random() < chance
```

The computers decision making when buying properties has been improved. Now when the computer owns part of the set the property belongs to already then the chance of it purchasing it will be greatly increased. This encourages computers to get sets more easily as previous feedback suggested it was too difficult for computer players to gain full sets.

```
#Checks if a computer is in jail and decides what happens to them
def inJail(self):
    if self.jailed > 0:
        #If the computer has a get out of jail free card they will use it to get out of jail
        if self.get_out_of_jail_free > 0:
            self.jailed = 0
            self.get_out_of_jail_free -= 1
            gc.mainLoop("You used a get out of jail free card")
            return False
        else:
            #If the computer has over £250 they will spend £50 to get out of jail
            if (self.money - 50) > 200:
                self.jailed = 0
                self.money -= 50
                board[20].free_parking_cash += 50
                gc.mainLoop("You paid to get out of jail")
                return False
            #Otherwise they will stay in jail
            else:
                gc.mainLoop("You didn't pay to get out of jail")
                self.jailed -= 1
                return True
    else:
        return False
```

If a computer is in jail they are given the same options as the players, but the computer choses itself and the main loop is called to display to the other players what choice was made.

```
def landOn(self, player):
    #Switch case statement for random card to decide what happens
    match random.choice(self.chances if self.name == "Chance" else self.com_chests):
            #Chance Cards
        case "AdvToGo":
            self.advanceTo(board[0].name, 0, player)
        case "AdvToTrafalgar":
            self.advanceTo(board[24].name, 24, player)
        case "AdvToPall":
            self.advanceTo(board[11].name, 11, player)
        case "AdvNearUtil":
            self.advanceTo("Nearest Utility",self.findNearest(player.position, Utilities) , player)
        case "AdvNearRail":
            self.advanceTo("Nearest Railway Station",self.findNearest(player.position, Station) , player)
        case "Get50":
            self.getMoney("Bank pays you a dividend of £50", 50, player)
        case "GoBack3":
            player.increasePos(-3)
        case "GoToJail":
            gc.mainLoop("Go to Jail. Do not pass Go, do not collect £200")
            player.jail()
        case "GeneralRepairs":
            self.propertyRepairs(f"Make general repairs on all of your property. For each house pay £25, for each hotel pay £100", 25, 100, player)
        case "AdvToKingsCross":
            self.advanceTo(board[5].name, 5, player)
        case "AdvToMayfair":
            self.advanceTo(board[39].name, 39, player)
        case "PayEachPlayer50":
            self.moneyFromEach("You have been elected chairman of the board. Pay each player £50", -50, player)
        case "Get150":
            self.getMoney("Your loan matures. Recieve £150", 150 , player)
        case "GetOutOfJail":
            gc.mainLoop("You get a get out of jail free card")
            player.get_out_of_jail_free += 1

            #Community Chest cards
        case "Get200":
```

Within the ''ChanceChests' class there were two new cards created. First is the get out of jail free card which gives the player a get out of jail free card.

```
            self.getMoney("You inherit £100", 100, player)
        case "AdvToUsefulProp":
            gc.mainLoop("Advance to a property you need. If none exist advance to a random property")
            best_set = None
            highest = 0
            for i in range(len(sets)):
                count = 0
                not_player = False
                unowned_property = False
                for j in sets[i]:
                    if board[j].id_of_owner == player.id:
                        count += 1
                    if board[j].id_of_owner != player.id and board[j].id_of_owner != None:
                        not_player = True
                    if board[j].id_of_owner == None:
                        unowned_property = True
                if (not not_player) and count > highest and unowned_property:
                    highest = count
                    best_set = i
            if best_set != None:
                for i in sets[best_set]:
                    if board[i].id_of_owner == None:
                        self.advanceTo(board[i].name, i, player)
                        return

            pos = list(range(39))
            random.shuffle(pos)
            for i in pos:
                if isinstance(board[i], Properties):
                    if board[i].id_of_owner == None:
                        print(board[i].name)
                        self.advanceTo(board[i].name, i, player)
                        return
            random.shuffle(pos)
            for i in pos:
                if isinstance(board[i], Properties):
                    print(board[i].name)
                    self.advanceTo(board[i].name, i, player)
                    return
```

The second was one which allows the player to advance to a useful property. It works by checking which set the player is closest to completing and sends the player there to a unowned property their to buy it. If one of the properties in the set is owned by somebody else then it wont be considered. If there are no useful sets to complete then it will bring the player to a random unowned property. If all properties are owned then it will send the player to a random owned property. This fixes two issues I heard from feedback. The first was that it was very hard to get a full set which meant the game would often never end as players would continuously accumulate money with nothing to spend it on. This remedies this by helping players complete their sets to buy houses. The other issue was that games would be boring as nobody would land on other people properties. This helps solve this issue by providing another way for a player to be put on another players property.

```
    #Performs common task of purchasing properties if unowned
    def landOn(self, player):
        if self.id_of_owner == None:
            if not player.passed_go:
                gc.mainLoop("You can't buy properties until you have passed go once")
            elif self.price <= player.money:
                if isinstance(player, Computer):
                    output = player.propertyDecision(self.position)
                else:
                    output = None
                if gc.mainLoop(f"Do you want to buy it for £{self.price}?", "Yes", input2="No", output = output):
                    self.getOwned(player)
                    player.money -= self.price
                    gc.mainLoop(f"You purchased {self.name}")
                else:
                    gc.mainLoop(f"You didn't purchase {self.name}")
            else:
                gc.mainLoop("It is unowned but you are too poor to afford it")
        else:
            return True
```

The 'landOn' method for the 'Properties' class has been changed so that players cannot purchase any property before passing go. This is how it has always worked in the real game.

```
#Class to handle all pygame graphics processing
class Graphics():
    def __init__(self):
        #Keeps track of which card is blitted
        self.blitted_card = None
        #Variable to tell the main loop method to end and what to return
        self.out = None
        #Variable to tell drawBoard what text to blit
        self.text = None


        #Defining the pygame widget buttons so they dont have to be defined as global variables
        #adds 50 hue to colour for pressed colour and subtract 50 for the hover colour
        self.button1 = Button(screen, 551, 614, 161, 82, text = None, inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60), fontSi
        self.button2 = Button(screen, 814, 614, 161, 82, text = None, inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60), fontSi
        self.xButton = Button(screen, CARD_WIDTH - 25 - LINEWIDTH + (421 + (6 * SQUARE_SIZE)), LINEWIDTH + SQUARE_SIZE + 1, 25, 25, text = "X", inactiveColo
        self.upgradeButton = Button(screen, 425 + 200 + (6 * SQUARE_SIZE), 1 + SQUARE_SIZE + (CARD_HEIGHT / 11 * 10) - 5, 100, 30, text = "Yes", inactiveCol
        self.saveButton = Button(screen, 1208, 614, 161, 82, text = "Save", inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60),
        self.quitButton = Button(screen, 1208, 874, 161, 82, text = "Quit", inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60),


        #Hides all buttons until they are needed
        self.hideButtons()
```

```
(236, 54, 60), fontSize = 50, onClick = lambda: self.setOut(True))
(236, 54, 60), fontSize = 50, onClick = lambda: self.setOut(False))
xt = "X", inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60), fontSize = 50, onClick = lambda: self.stopCardBlit())
 = "Yes", inactiveColour=RED, hoverColour = (136, 31, 34), pressedColour = (236, 54, 60), fontSize = 40, onClick = lambda: self.upgradeCard())
lour = (236, 54, 60), fontSize = 50, onClick=lambda: save())
lour = (236, 54, 60), fontSize = 50, onClick=lambda: quitGame())
```

The text that is inputted into the main loop is now been made into an attribute which means it doesn't have to be passed through as a parameter constantly. A 'hideButtons' method has also been created for this class in order to easily hide all of buttons similarly to the 'MainMenu' class.

A save button has been added which allows the user to save their game and re open it where they left off.

To increase the usability of this game, I added a quit button which closes the game. Since the game always requires full screen, it may be difficult for some players to close their game when they are done.

```
#Main game loop with different parameters to do different things
#Other args used to provide less used optional arguements that dont need to be a regular parameter
def mainLoop(self, text, input1="Next", **other_args):
    self.out = None

    self.text = text

    #The input2 variable is gotten from an optional arguement
    input2 = other_args.get("input2", None)

    #A tuple with info about move player animation, the first element holds the destination, the second decides if it should go backwards or not
    movePlayer = other_args.get("movePlayer", None)
    #A variable which if true plays a dice roll animation and returns the result, if false it displays the previous roll, if none nothing happens
    diceRoll = other_args.get("diceRoll", None)
    #Variable which decides which output should be given by a computer player
    computerOutput = other_args.get("output", True)

    canSave = other_args.get("save", False)
    #shows how many frames have been displayed during the loop
    time = 0

    #Draws the board and blits the card if it needs to be blitted
    self.drawBoard()
    self.blitCard()
    self.quitButton.show()

    #If input 1 exits then it is rendered and the button is shown
    if input1 != None:
        self.button1.text = FONT30.render(input1, ANTI_ALIASING, BLACK)
        self.button1.show()

    #If input 2 exits then it is rendered and the button is shown
    if input2 != None:
        self.button2.text = FONT30.render(input2, ANTI_ALIASING, BLACK)
        self.button2.show()

    if canSave:
        self.saveButton.show()

    #If a dice roll is happening, two random roll times are created and two roll variables are defined
    if diceRoll:
        roll_time1 = random.randint(ROLL_TIME_1, ROLL_TIME_2)
        roll_time2 = random.randint(ROLL_TIME_1, ROLL_TIME_2)
        roll1 = 0
        roll2 = 0



    while True:
```

Due to the change with the text attribute, now in the main loop the text parameter is simply assigned to the text attribute. Another kwarg called ‘canSave’ was added which simply decides whether the save button is shown.

```
while True:
    #If move player exists then the players position is increased by 1 at even intervals and the board in re blitted
    #until they reach their destination where the loop is ended
    if movePlayer != None:
        #Time is independent of fps
        if time % (MOVE_TIME * FPS/30) == 0:

            players[current_player].position += (1 if not movePlayer[1] else -1)
            if players[current_player].position > 39:
                players[current_player].position -= 40
                players[current_player].money += 200
                players[current_player].passed_go = True
            if players[current_player].position == movePlayer[0]:
                return
            self.drawBoard()
            self.blitCard()

    #Event loop used to collect inputs
    events = pygame.event.get()
    for event in events:

        #If the user tries to close the window, pygame and python will stop
        if event.type == pygame.QUIT:
            quitGame()

        #If the user clicks the mouse button the code will check if the mouse is hovered over a button
        if event.type == MOUSEBUTTONDOWN:
            #Checks each tile position if mouse is over it
            for i in range(len(pixels)):
                tile_rect = pygame.rect.Rect(pixels[i][0], pixels[i][1], SQUARE_SIZE, SQUARE_SIZE)
                if tile_rect.collidepoint((pygame.mouse.get_pos())):
                    #Shows info about the clicked on tile
                    self.blitted_card = i
                    self.drawBoard()
                    self.blitCard()
```

Now when a new tile is blitted, there is no code to check whether the upgrade button should be shown or not shown. This is because that is all checked within the 'blitCard' method.

```
    #If out has a value then the value will be returned and all of the buttons will be hidden
    if self.out != None:
        self.hideButtons()
        #If the move player animation was interrupted then the player will skip to their destination
        if movePlayer != None:
            if players[current_player].position > movePlayer[0]:
                players[current_player].money += 200
                players[current_player].passed_go = True
            players[current_player].position = movePlayer[0]

        return self.out

    #If the player is a computer all buttons but the x button will be hidden
    if isinstance(players[current_player], Computer):
        self.button1.hide()
        self.button2.hide()
        self.upgradeButton.hide()

        # Once the time exceeds the computer time and there isnt a dice rolling or player moving then the computer will return
        if time > COMPUTER_TIME and diceRoll != True and movePlayer == None:
            #Before it finishes the computer has the chance to buy houses
            players[current_player].buyHouses()
            self.hideButtons()
            return computerOutput
    #Used to update pygame widgets
    pygame_widgets.update(events)
    #Updates the screen
    pygame.display.flip()
    #The time is increased at the end of each loop
    time += 1
    #Limits the framerate to 30 frames per second
    clock.tick(FPS)
```

At the end of the 'mainLoop' method I changed some of the output code due a bug I found. When the move player animation is playing and the destination is past go, if I click the button

to skip it before the icon moves past go the player wont get money for passing go and the 'passed_go' attribute won't change to true. To fix this, when the animation is skipped I check if the players destination is passed go. If it is then the money is given and the attribute is set to true.

```
#Draws the detail for each section of the board and blits it onto the main screen surface
def drawBoard(self):
```

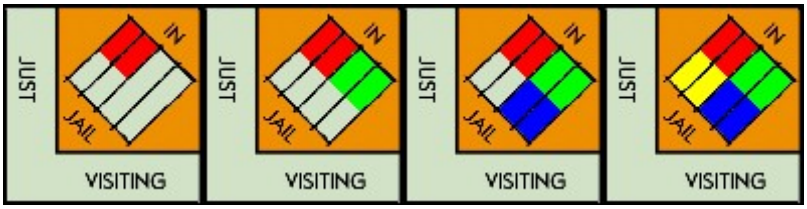
```
#Counts the number of players on a space with an id lower than the chosen player
def getNumOnSpaceBefore(pos,id):
    count = 0
    for i in range(id+1):
        if pos == players[i].position and not players[i].dead:
            count += 1
    return count

for player in players:
    if player.jailed > 0:
        const = ((SQUARE_SIZE * 9) + LINEWIDTH)
        match getNumOnSpaceBefore(player.position, player.id):
            case 1:
                pygame.draw.polygon(left_surf, player.colour, ([const + 3, 33 + 1], [const + 18, 18 + 1], [const + 34, 35 + 1], [const + 19, 51 + 1]))
            case 2:
                pygame.draw.polygon(left_surf, player.colour, ([const + 18, 18 + 1], [const + 35, 1 + 1], [const + 52, 18 + 1], [const + 34, 35 + 1]))
            case 3:
                pygame.draw.polygon(left_surf, player.colour, ([const + 52, 18 + 1], [const + 66, 34 + 1], [const + 50, 51 + 1], [const + 34, 35 + 1]))
            case 4:
                pygame.draw.polygon(left_surf, player.colour, ([const + 50, 51 + 1], [const + 34, 65 + 1], [const + 19, 51 + 1], [const + 34, 35 + 1]))
left_surf.blit(jail_image, ((SQUARE_SIZE * 9) + LINEWIDTH, 0 + LINEWIDTH))



#Code used to blit player squares onto board
for player in players:
    #Gets the number of players before it in the player list to decide how much the current player needs to be moved by to make room
    num_on_space = getNumOnSpaceBefore(player.position, player.id)
    #Value of how much player must be moved by
    offset = 0
    if player.jailed < 1 and not player.dead:
        match num_on_space:
            case 2:
                offset = PLAYER_SIZE
            case 3:
                offset = PLAYER_SIZE * 2
            case 4:
                offset = PLAYER_SIZE * 3
```

Within the 'drawBoard' method an addition was made to the blit player code. If the player is in jail they will have their icon appear in the jail cell on the space. The jail space image is then blitted afterwards to have the bars appear over the player icon. This uses the 'draw.polygon' method instead of 'draw.rect'. This is because the jail cell is angled so a rect wouldn't. This method draws a polygon by taking input of each of the points.



In addition, there was a bug where a players icon would not disappear when that player had gone bankrupt, now before blitting the player, it checks if the player is in the game or not.

```
#Code for creating player status cards

#Array of each status card surface
status_cards = [None] * num_of_players

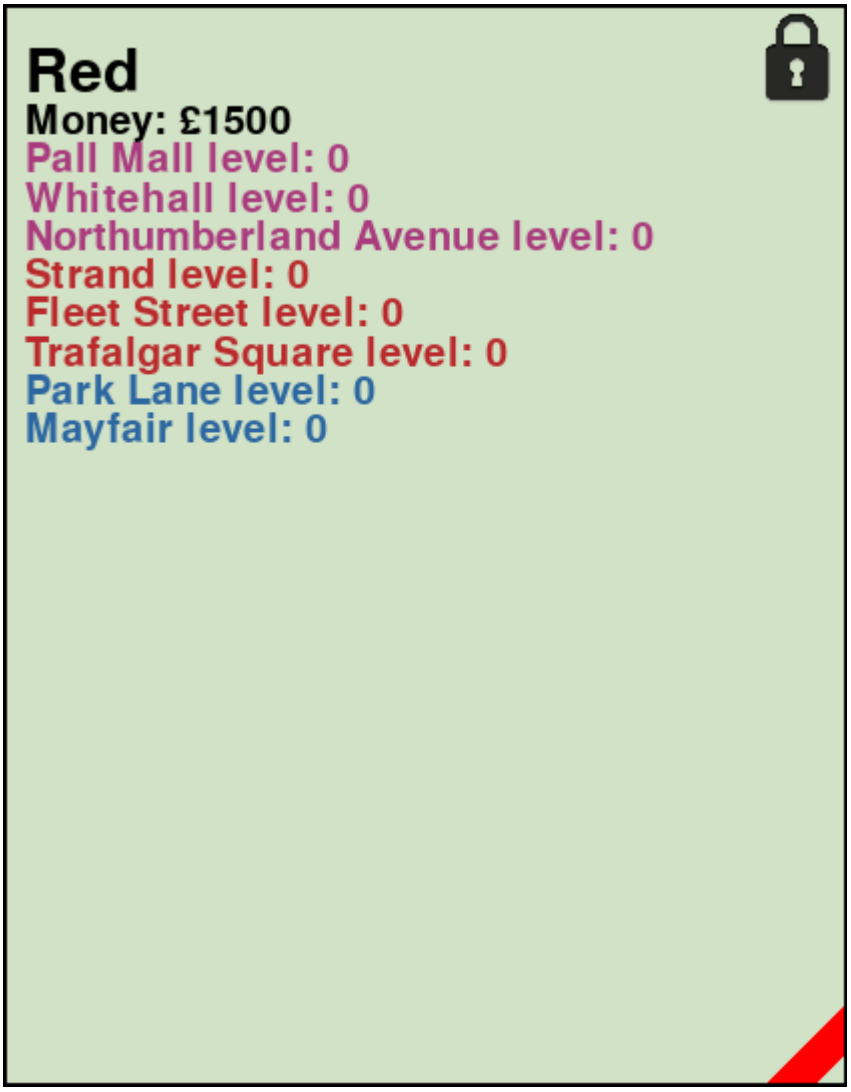
#Goes through each player
for i in range(num_of_players):
    #Creates surface for the text to be blitted onto
    status_cards[i] = pygame.surface.Surface((STATUS_CARD_WIDTH, STATUS_CARD_HEIGHT))
    status_cards[i].fill(LIGHTGREEN)

    #Ribbon showing players colour
    pygame.draw.line(status_cards[i], players[i].colour, (330, 600), (430, 500), 25)
    pygame.draw.rect(status_cards[i], BLACK, (0, 0, STATUS_CARD_WIDTH, STATUS_CARD_HEIGHT), LINEWIDTH)
    #If the player is out of the game the text dead will appear over their card
    if players[i].dead:
        dead_text = FONT40.render("DEAD", ANTI_ALIASING, RED)
        dead_text = pygame.transform.scale2x(dead_text)
        dead_text = pygame.transform.scale2x(dead_text)
        dead_text = pygame.transform.rotate(dead_text, -45)
        status_cards[i].blit(dead_text, (0,0))
        continue
    #Blits the players name and their money count
    status_cards[i].blit(FONT30.render(players[i].name, ANTI_ALIASING, BLACK), (10, STATUS_CARD_HEIGHT / 28 * 1))
    status_cards[i].blit(FONT20.render(f"Money: £{players[i].money}", ANTI_ALIASING, BLACK), (10, (STATUS_CARD_HEIGHT / 28 * 2) + 10))
    if players[i].jailed > 0:
        status_cards[i].blit(padlock_image, (370, 0))
    if players[i].get_out_of_jail_free > 0:
        status_cards[i].blit(get_out_of_jail_free_image, (220, 10))
        status_cards[i].blit(FONT30.render(f": {players[i].get_out_of_jail_free}", ANTI_ALIASING, BLACK), (220 + 60, 10))
```

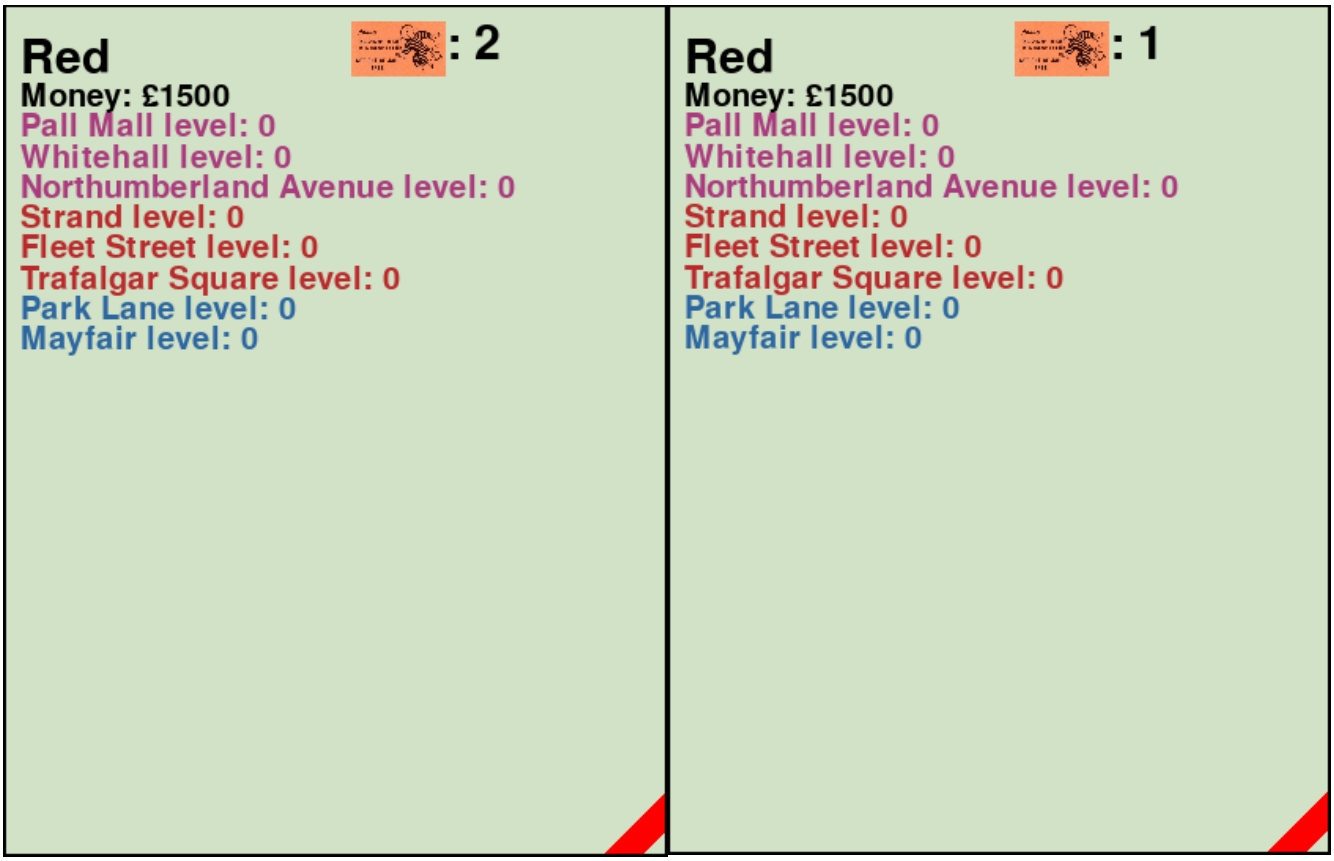
The status cards have had some additions which increase the playability of the game. Now each status cards displays the colour of the corresponding player with a coloured ribbon at the bottom right of the card. This was made using the 'draw.line' method to draw a very thick line across the bottom right corner.



There is also a little padlock icon which displays if a player is in jail.



When a player possesses get out of jail free cards, an icon of the card will be blitted along with the number they possess.



```
#Blits the info card about a tile onto the screen
def blitCard(self):
    if self.blitted_card != None:
        #Gets the surface with all of the info blitted on it from the show info method
        surface = board[self.blitted_card].showInfo()
        #Adds text saying to upgrade house if it is possible to upgrade
        if self.canUpgrade():
            surface.blit(FONT20.render(f"Add house for £{board[self.blitted_card].house_price}", ANTI_ALIASING, BLACK), (10, CARD_HEIGHT / 11 * 10))
            self.upgradeButton.show()
        else:
            self.upgradeButton.hide()
        #Blits the surface into the correct place
        screen.blit(surface, (421 + (6 * SQUARE_SIZE), SQUARE_SIZE + 1))
        self.xButton.show()

#Upgrades the set and re draws the board so the house is blitted and the players money is updated
def upgradeCard(self):
    board[self.blitted_card].upgrade()
    self.drawBoard()
    self.blitCard()
    #If the house is no longer able to be upgraded the upgrade button is hidden
    if not self.canUpgrade():
        self.upgradeButton.hide()

#Unblits the card by removing the value of the blitted card attribute and covering up the blitted card with a rect the same colour as the background
def stopCardBlit(self):
    self.blitted_card = None
    self.xButton.hide()
    self.upgradeButton.hide()
    pygame.draw.rect(screen, LIGHTGREEN, (421 + (6 * SQUARE_SIZE) -1, SQUARE_SIZE + 1 + 1, CARD_WIDTH, CARD_HEIGHT))

def hideButtons(self):
    self.button1.hide()
    self.button2.hide()
    self.xButton.hide()
    self.upgradeButton.hide()
    self.saveButton.hide()
    self.quitButton.hide()
```

The ‘blitCard’ method now shows the upgrade button if it needs to be shown. The ‘upgradeCard’ method now used the class attributes instead of parameters to upgrade the property. This means that since it has no parameters, the ‘onClick’ function for the upgrade button only has to be defined once instead of every time the blitted card changes.

```
def save():
    file = open("save", "wb")
    pickle.dump(num_of_players, file)
    pickle.dump(num_of_computers, file)
    pickle.dump(current_player, file)
    for i in board:
        pickle.dump(i, file)
    for j in players:
        pickle.dump(j, file)
    file.close()
```

The save function is not part of any class and is used to save the game state. Firstly, it opens the save file in write binary mode. The 'pickle.dump' method is used to dump that objects data into the file. Firstly it dumps the variables for the number of players, computers and the current player. Then it goes through the board array and dumps each tile object into the file. Finally it goes through the player array and dumps each player in one by one. The file is then closed.

```
#Defining of all of the tiles of the board
board = [
Go(                          "Go", 0),
Sets(              "Old Kent Road", 1, 60 , 2, 10,    30,  90,  160,   250, BROWN),
ChanceChests(    "Community Chest", 2),
Sets(            "Whitechapel Road", 3 , 60, 4, 20,    60,  180,  320,  450, BROWN),
Tax(                  "Income Tax", 4),
Station(     "Kings Cross Station", 5 , 200, BLACK),
Sets(        "The Angel Islington", 6 , 100, 6, 30,   90,  270,  400,  550, (99, 118, 127)),
ChanceChests(             "Chance", 7),
Sets(                "Euston Road", 8 , 100, 6, 30,   90,  270,  400,  550, (99, 118, 127)),
Sets(            "Pentonville Road", 9 , 120, 8, 40,   100, 300,  450,  600, (99, 118, 127)),
Jail(                       "Jail", 10),
Sets(                  "Pall Mall", 11, 140, 10, 50,  150, 450,  625,  750, PINK),
Utilities(      "Electric Company", 12, 150, P_PURPLE),
Sets(                  "Whitehall", 13, 160, 10, 50,  150, 450,  625,  750, PINK),
Sets(      "Northumberland Avenue", 14, 160, 12, 60,  180, 500,  700,  900, PINK),
Station(      "Marylebone Station", 15, 200, BLACK),
Sets(                  "Bow Street", 16, 180, 14, 70,  200, 550,  750,  950, ORANGE),
ChanceChests(    "Community Chest", 17),
Sets(          "Marlborough Street", 18, 180, 14, 70,  200, 550,  750,  950, ORANGE),
Sets(                "Vine Street", 19, 200, 16, 80,  220, 600,  800,  1000, ORANGE),
FreeParking(         "Free Parking", 20),
Sets(                     "Strand", 21, 220, 18, 90,  250, 700,  875,  1050, RED),
ChanceChests(             "Chance", 22),
Sets(               "Fleet Street", 23, 220, 18, 90,  250, 700,  875,  1050, RED),
Sets(            "Trafalgar Square", 24, 240, 20, 100, 300, 750,  925,  1100, RED),
Station("Fenchurch Street Station", 25, 200, BLACK),
Sets(            "Leicester Square", 26, 260, 22, 110, 330, 800,  975,  1150, (170, 161, 38)),
Sets(             "Coventry Street", 27, 260, 22, 110, 330, 800,  975,  1150, (170, 161, 38)),
Utilities(            "Water Works", 28, 150, P_PURPLE),
Sets(                  "Piccadilly", 29, 280, 24, 120, 360, 850,  1025, 1200, (170, 161, 38)),
GoToJail(              "Go to Jail", 30),
Sets(              "Regent Street", 31, 300, 26, 130, 390, 900,  1100, 1275, GREEN),
Sets(              "Oxford Street", 32, 300, 26, 130, 390, 900,  1100, 1275, GREEN),
ChanceChests(    "Community Chest", 33),
Sets(                 "Bond Street", 34, 320, 28, 150, 450, 1000, 1200, 1400, GREEN),
Station("Liverpool Street Station", 35, 200, BLACK),
ChanceChests(             "Chance", 36,),
Sets(                   "Park Lane", 37, 350, 35, 175, 500, 1100, 1300, 1500, BLUE),
Tax(                    "Super Tax", 38),
Sets(                     "Mayfair", 39, 400, 50, 200, 600, 1400, 1700, 2000, BLUE),
]
```

Instead of defining each tile object as a variable and placing it in an array, which was unnecessary, I instead now directly define them as a part of the array.

```
#Loads the game
if main_menu.load_game:
    file = open("save", "rb")
    num_of_players = pickle.load(file)
    num_of_computers = pickle.load(file)
    current_player = pickle.load(file)
    for i in range(len(board)):
        board[i] = pickle.load(file)
    for i in range(num_of_players):
        players.append(pickle.load(file))
    file.close()
else:
    #The names of the different players, will be created by the menu
    names = main_menu.names
    #Number of players in the game
    num_of_players = main_menu.num_of_players
    #Number players that are computers, cant be greater than number of players
    num_of_computers = main_menu.num_of_computers

    #Array of all player
    #instantiates players using above data
    for i in range(num_of_players):
        print(i)
        if i > num_of_players - num_of_computers - 1 :
            players.append(Computer(i))
        else:
            players.append(Player(names[i], i))

    #Used to allow a quick game where all of the sets are randomly distributed to each player
    if main_menu.quick_game:
        sets2 = list(sets)
        random.shuffle(sets2)
        num = random.randint(0, num_of_players - 1)
        for i in sets2:
            num += 1
            num = num % num_of_players
            for j in i:
                board[j].getOwned(players[num])
```

If the 'load_game' attribute of the main menu is true then the load code is ran. The different stored objects in the file are loaded in the order they were saved in. A for loop is used to load the players and the number of iterations is equal to the 'num_of_players' variable which was loaded in which ensures that it doesn't try to load in too many players.

If 'load_game' is false, the game setup information like the number of players are gathered from the main menu and used to create all of the players and allocate the sets if a quick game is chosen.

```
#Finds out who is left and congratulates them
for i in players:
    if not i.dead:
        main_menu.winner(i)
pygame.quit()
```

If a player has won the 'winner' method is called with the player as the argument.



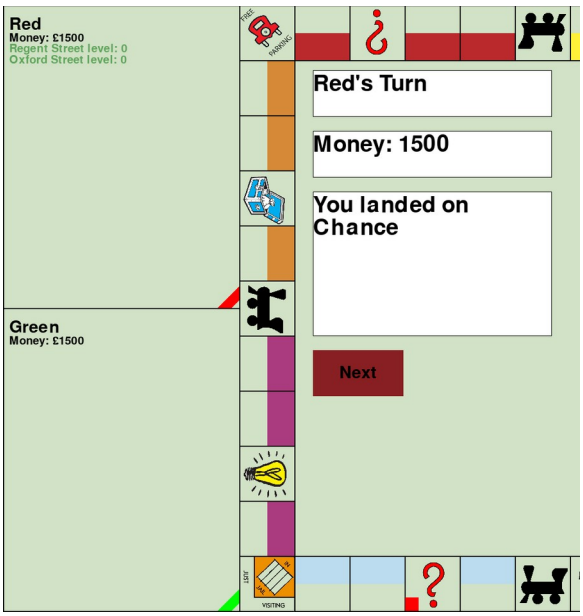
Testing of Third Iteration:

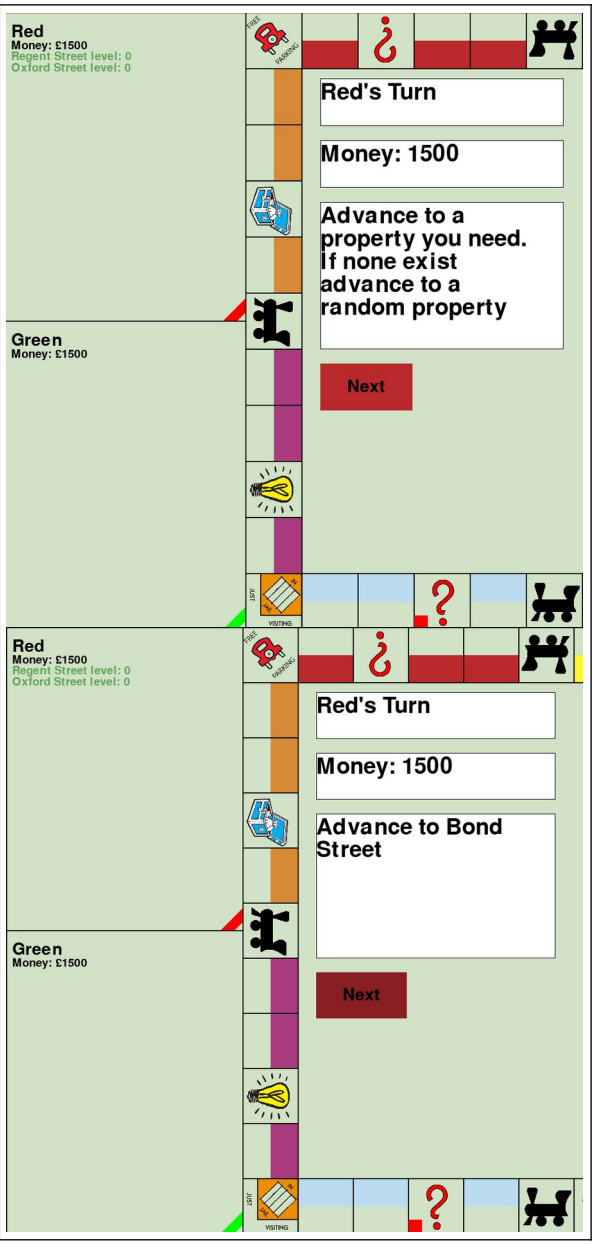
Previous Tests:

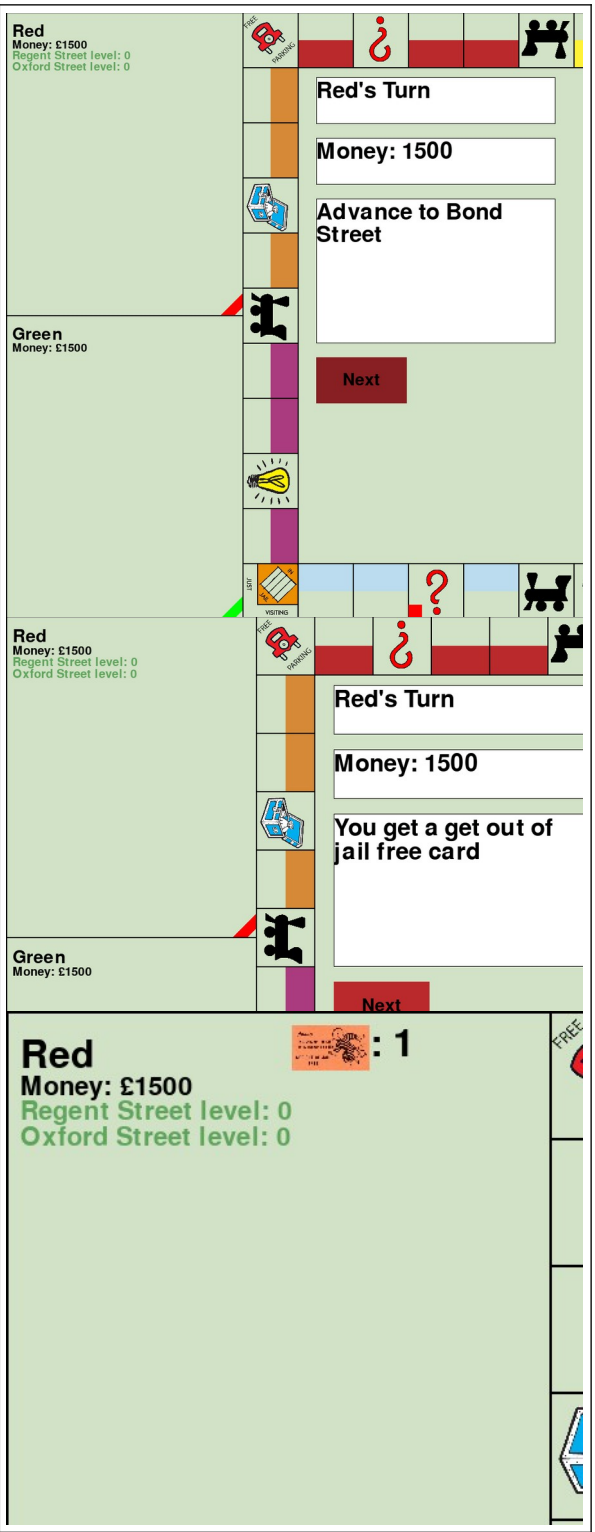
Test Number	Test Description	Passed?	Evidence Description	Evidence
101	When clicking the first button in the main loop the function should return True	Yes		Same as previous

102	When clicking the second button in the main loop the function should return False	Yes		Same as previous
103	When window is attempted to be closed, pygame stops running	Yes		Same as previous
104	Clicking on all the different tiles on the screen should display a box with information about the tile	Yes		Same as previous
105	When the player clicks on a property that they own but are not eligible to place houses on then they should not have the option to do so	Yes		Same as previous
106	When the player clicks on a property that they own and are eligible to purchasing houses on it then they should have the option to do so	Yes		Same as previous
107	When the a player clicks the button to purchase a house, the property should gain an extra level for the extra house	Yes		Same as previous
108	The player should not be able to upgrade a set more than 5 times	Yes		Same as previous

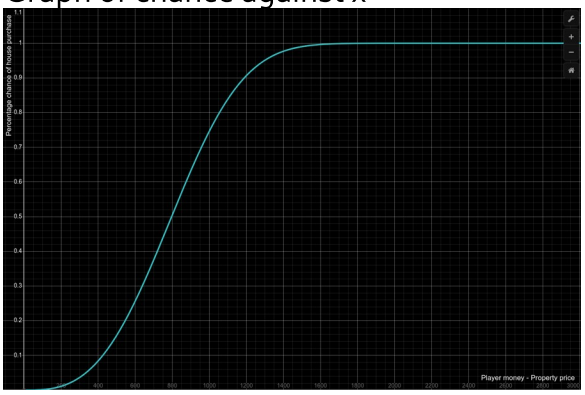
109	The player should be able to buy a property when it is unowned and they can afford to. When they purchase it the money should be taken from their wealth.	Yes		Same as previous
111	When a player lands on a property owned by somebody else they should pay them the exact amount required	Yes		Same as previous
112	When a player lands on a property owned by someone else and they cant afford to pay the other person. The other person should get the remainder of their cash and the player should go bankrupt	Yes		Same as previous
113	When the player lands on a utility where both are owned by the same person they should have to pay the owner 10 times their previous dice roll	Yes		Same as previous

114	When the player lands on a station which is owned by another player who owns multiple stations they should have to pay them more depending on how many they own	Yes		Same as previous
115	When the player rolls a double they should get an extra turn	Yes		Same as previous
116	When the player rolls 3 doubles in a row they should go to jail	Yes		Same as previous
117	When the player lands on a chance or community chest space they should receive a card with instructions on it which should be followed correctly by the program	Yes	The advance to useful property chance card works. 'Red' needs bond street to complete their set and the chance card send them there. The get out of jail free card also works. When 'Red' lands on the space they are given a get out of jail free card	

				 Red Money: £1500 Regent Street level: 0 Oxford Street level: 0 Red's Turn Money: 1500 Advance to a property you need. If none exist advance to a random property Next Green Money: £1500 Red Money: £1500 Regent Street level: 0 Oxford Street level: 0 Red's Turn Money: 1500 Advance to Bond Street Next Green Money: £1500

				
118	When the player moves past Go they should receive £200	Yes		Same as previous

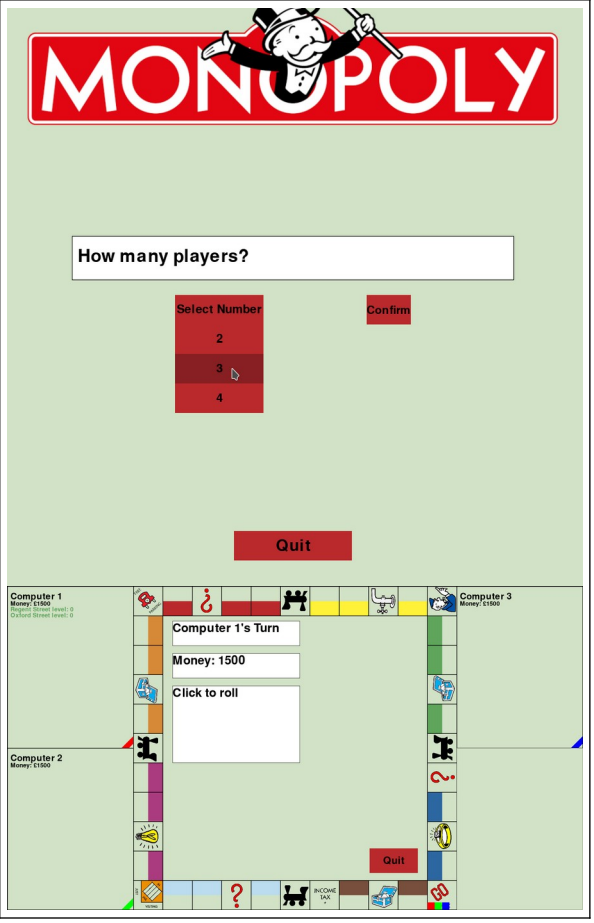
119	When the player lands on a tax space they pay the money specified to the free parking space	Yes		Same as previous
120	When the player lands on Free Parking they should get all of the money in the space and there should be £0 in free parking afterwards	Yes		Same as previous
121	When the player lands on Go to Jail they should be immediately sent to jail	Yes		Same as previous
122	When all players but one go bankrupt the remaining player should be declared the winner and be congratulated	Yes		Same as previous
123	Computer Players progress text automatically	Yes		Same as previous
124	Computer Players buy properties automatically	Yes		Same as previous

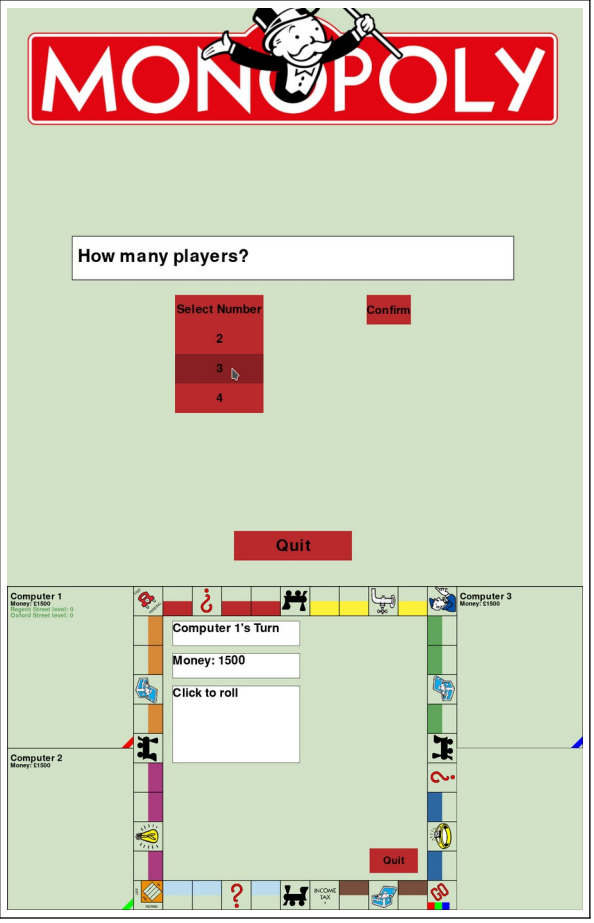
125	Computer Players wont buy properties they can't afford	Yes	The value of x will always be negative if the price is greater than the players money so the chance equation will return 0 or below even if the player owns part of that set already	Code for property decision:
				`num_owned_already += 1`
				`x = (self.money - board[position].price) * num_owned_already`
				`chance = (1 - math.e**-((x)/900)**3)`
				`#Returns whether the probability gave true or not`
				`return random.random() < chance`
				Graph of chance against x
				 

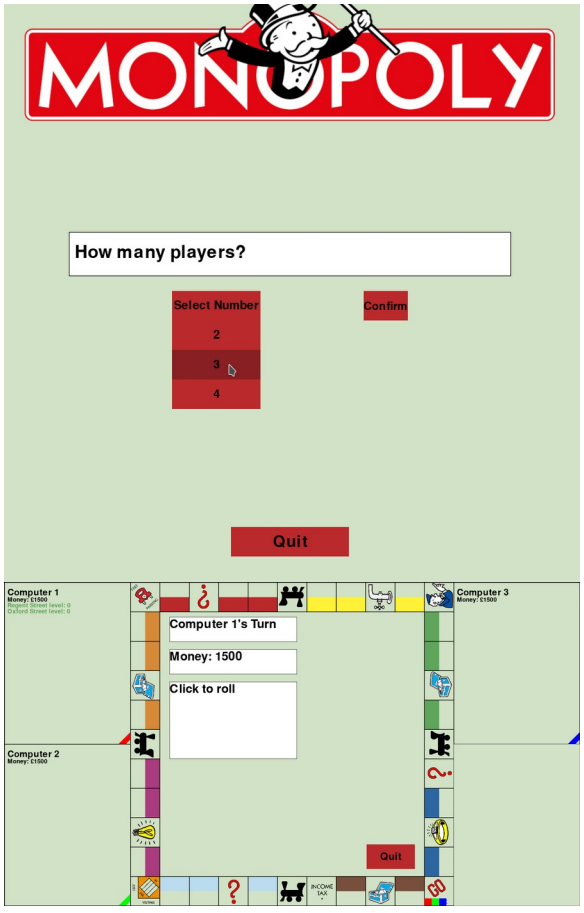
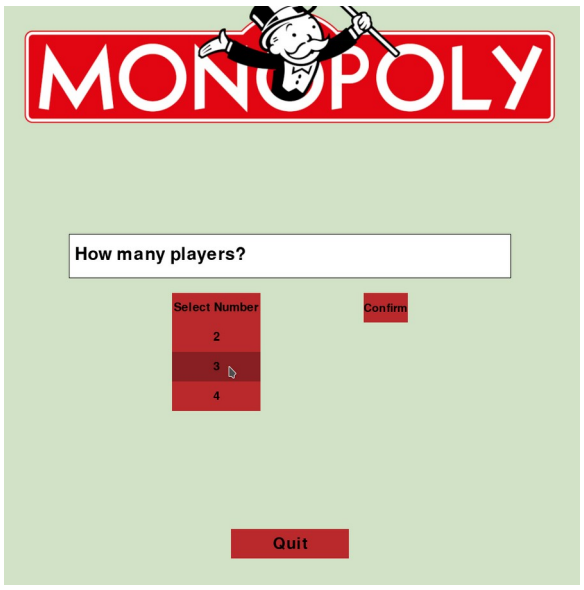
126	Computer players can buy houses automatically when they are eligible	Yes		Same as previous
127	Computer players wont buy houses they cant afford	Yes		Same as previous

Main Menu Tests:

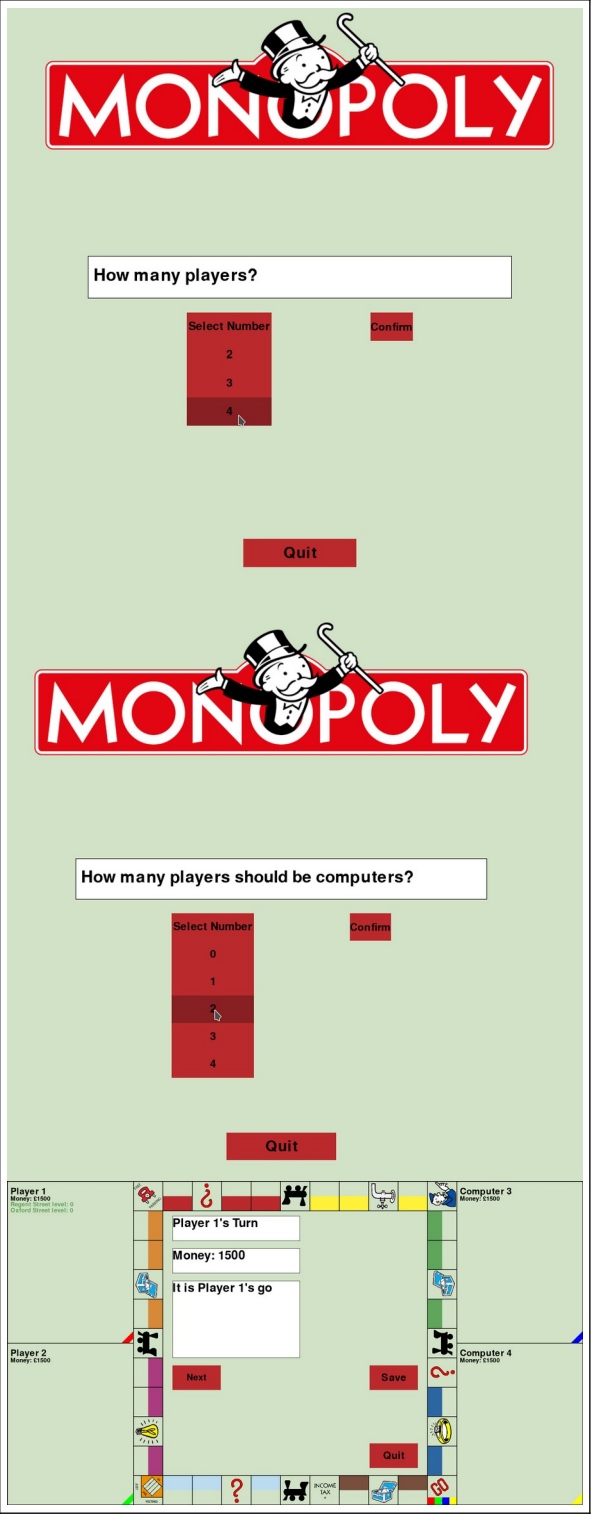
Test Number	Test Description	Passed?	Evidence Description	Evidence

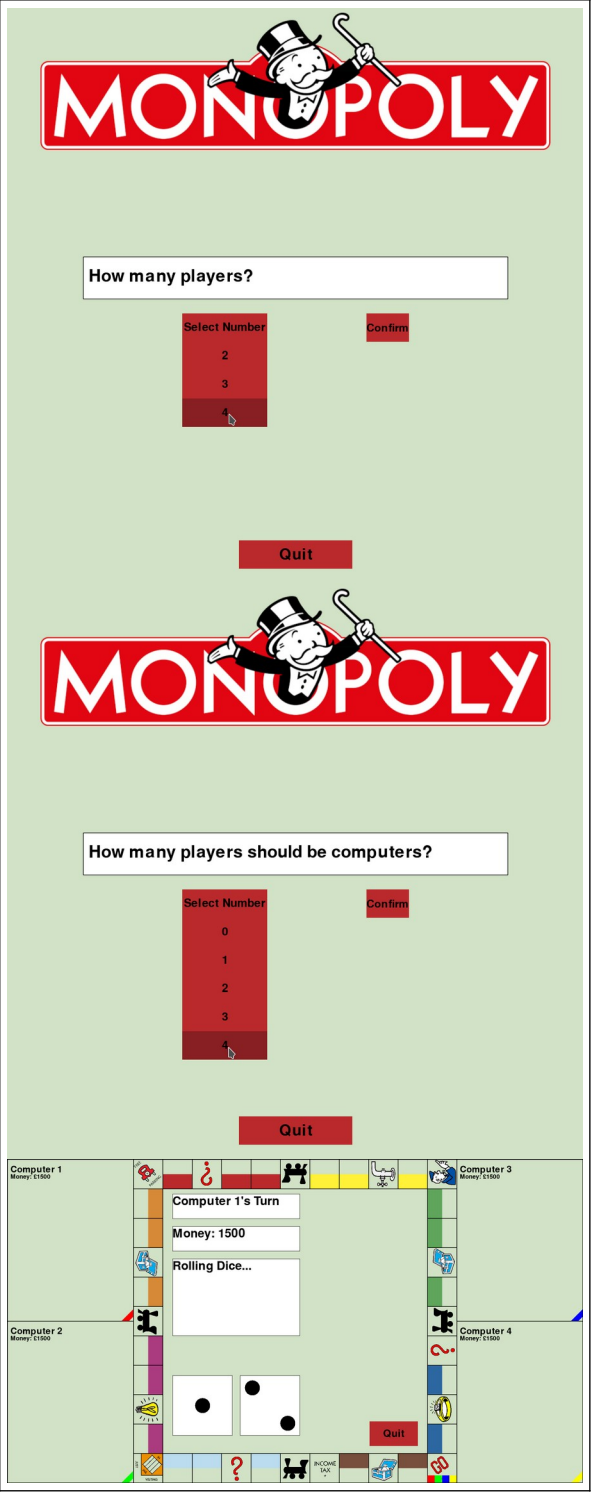
201	When 3 players are chosen 3 players are created	Yes	3 players are created after 3 are chosen	

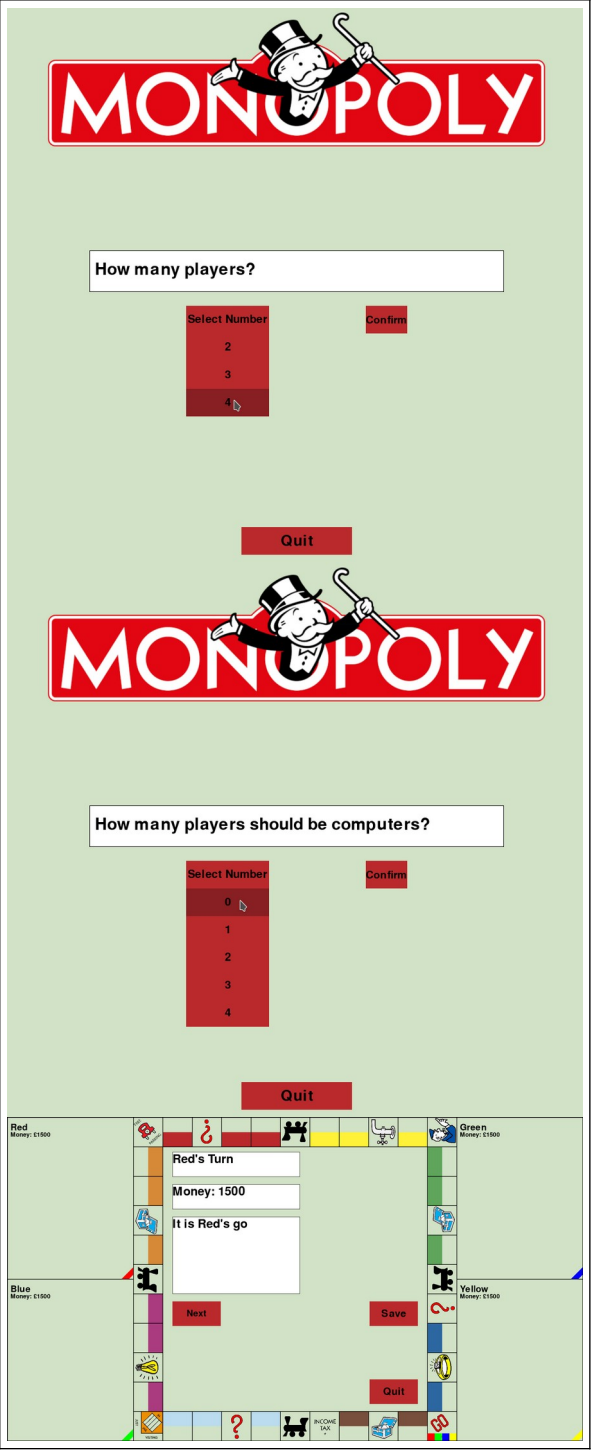
202	When 2 players are chosen 2 players are created	Yes	2 players are created after 2 are chosen	

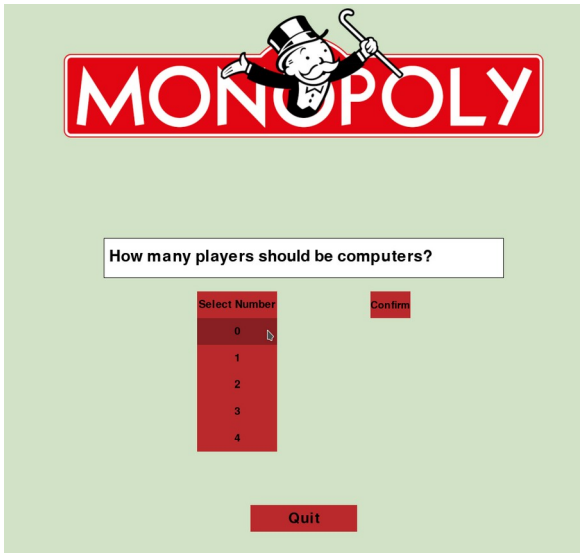
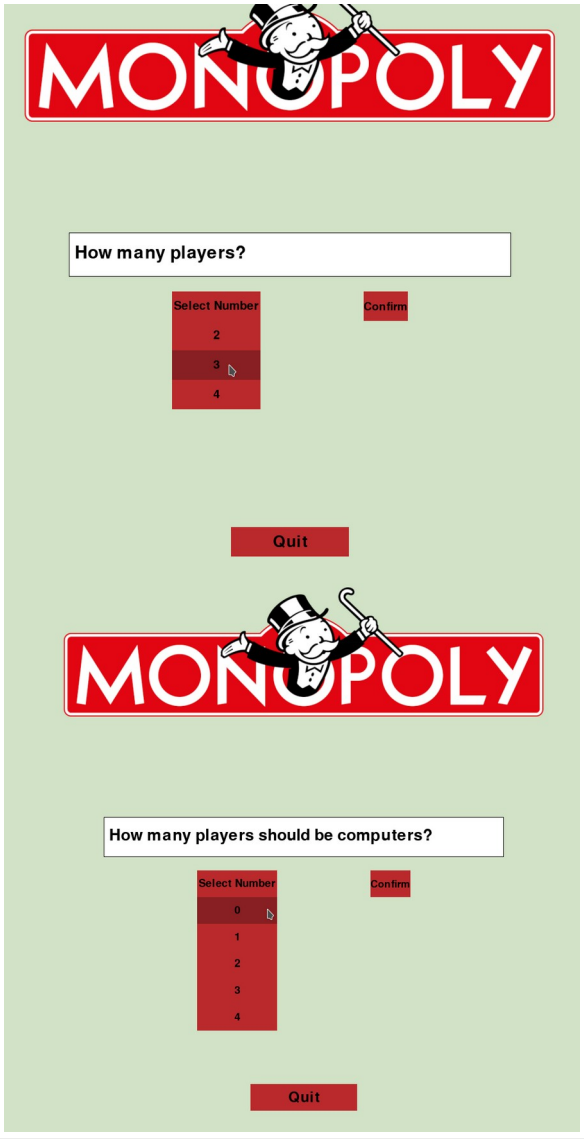
203	When 4 players are chosen 4 players are created	Yes	4 players are created after 4 are chosen	
204	The user won't be able to select 1 player as that is not enough players	Yes	1 Player is not an option to be chosen	
205	The user wont be able to select 5 players as that is too many players	Yes	5 Players is not an option to be chosen	Same as above

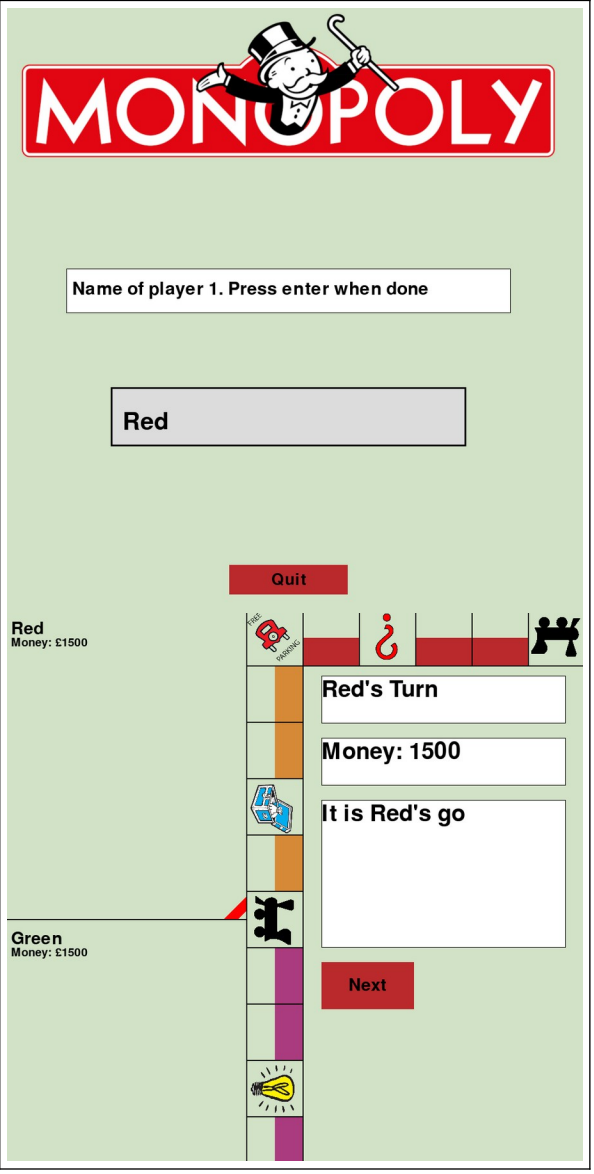
206	The user must not be able to input letters for the number of players as it should only take a number	Yes	You cannot input letters into the box as it is a drop down box without letters as any selections	Same as above

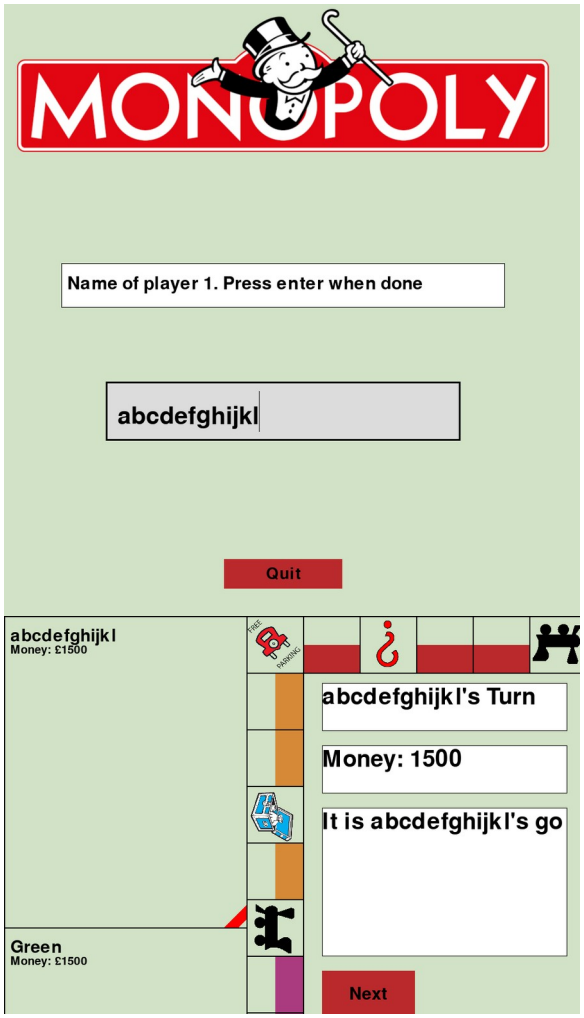
207	When the user choses 4 players and 2 computers there should be 2 human and 2 computer players made	Yes	When 4 players and 2 computers are chosen the game is made with 2 human players and 2 computer players	

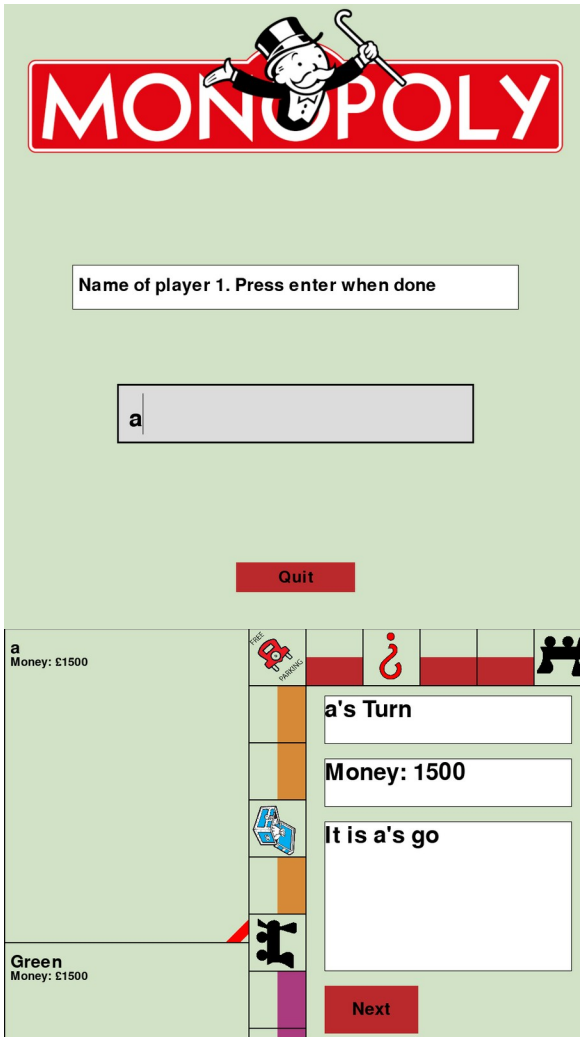
208	When 4 players and 4 computers are chosen, 4 players will be made all of which will be computer players	Yes	When 4 players and 4 computers are chosen the game is made with 4 computer players and no human players	

209	When 4 players and 0 computers are chosen, 4 players will be made all of which will be human players	Yes	When 4 players and 0 computers are chosen the game is made with 4 human players and no computer players	

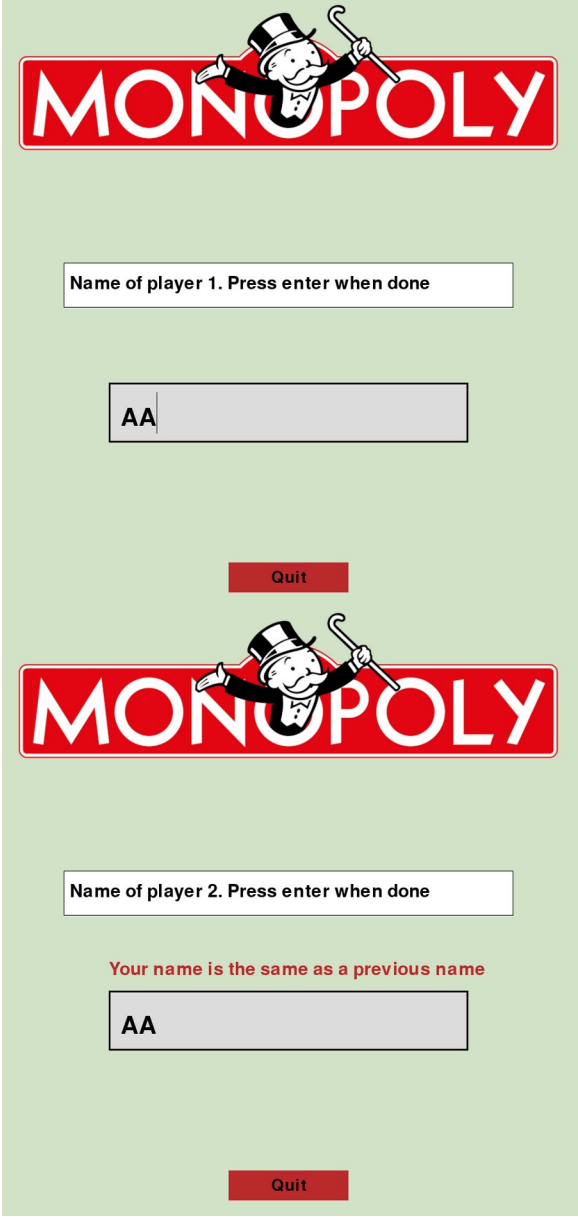
210	The user must not be able to input -1 computer players as there is a minimum of 0	Yes	The user cannot input -1 players as the options are limited from 0 to the number of players	
211	The user must not be able to more computer players than the number of players	Yes	There is no selection available that allows the user to chose a number of computer players larger than the number of players	

212	The user must not be able to input letters for the number of computers as it should only take a number	Yes	You cannot input letters into the box as it is a drop down box without letters as any selections	Same as above
213	The user inputs the name 'Red' and that name is given to a player to test if the name system accepts normal names	Yes	The name 'Red' is inputted and a player in the game is given that name	

214	The user inputs the name ‘abcdefghijkl’ and that name is given to a player	Yes	The name is given to the player as it is not above 12 characters long	

215	The user inputs the name 'a' and that name is given to a player	Yes	The name is given to the player as it is not below 1 character	
216	The user doesn't input a name into the name box and that name is declared invalid by the system	Yes	An error message shows, telling the user to input something into the box	

217	If the user inputs the name ‘abcdefghijklmnopqrst’ it will be declared invalid by the system for being to long	Yes	An error message shows, telling the user that their name is too long	

218	If two players are given the name 'AA' then the second input of the name will be declared invalid for being identical to a previous name	Yes	An error message shows, telling the user that their name is the same as a previous name	

Save System Tests:

Test Number	Test Description	Passed?	Evidence Description	Evidence

301	When saving and loading the game all player values are identical to how they were before the save	Yes	All of the properties owned by each player are identical before and after. The money each player has and their position on the board is the same. The types of players are identical with two human players and two computer players. The names of the human players are identical. The players turn is also saved.	Before save:  After save: 
302	When saving and loading the number of players will stay the same and the types of player will stay the same	Yes	The number of players remains the same before and after along with the type of player as there is still 1 human player and 1 computer player	Before:  After: 

303	When saving with players in jail and loading the players will still be in jail	Yes	Both ‘Red’ and ‘Yellow’ stay in jail after load	Before:  After: 
304	When saving, the houses and hotels on each of the properties will remain the same after loading again	Yes	All of the number of houses and hotels remain the same. The only change is the blitted card which is fine.	Before:  After: 

305	When saving, the players who a out of the game will remain the same	Yes	‘Blue’ and ‘Yellow’ remain out of the game after saving and loading.	Before:  After: 
306	When saving, the amount of money stored in the free parking space will remain the same after loading	Yes	The cash in free parking is £100 before and after saving. ‘Red’s number of get out of jail free cards is also saved	Before:  After: 

Review of Third Iteration:

What has been done? I have created the main menu system which allows users to choose different settings for the game. I have also implemented a saving system which lets users save their progress in their game to a file and load it at a later time.

What went well? The saving system worked very well. It passed all of my testing and appears to perfectly save each player and board tile with no unintended effects. The code is also quite simple and not overly complicated.

What went poorly? The main menu class